

ADVANTEST®

ADVANTEST CORPORATION

ASX/U-51
Hybrid Controller
Reference Manual

MANUAL NUMBER 8409600-07

Applicable Operating System
ASX/U-51

Trademark Information

The following are trademarks and registered trademarks used in this manual.

- ADVANTEST, ADVANsite, ATL, and ATLworks are trademarks of ADVANTEST Corporation.
- Sun, Sun Microsystems, the Sun logo, Ultra, Solaris, and NFS are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries
- All SPARC trademarks are used under license of SPARC International, Inc. and are also trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries. Products bearing SPARC trademarks are based on an architecture developed by Sun Microsystems, Inc.
- IBM, PowerPC, and the PowerPC architecture are trademarks or registered trademarks of International Business Machines Corp.
- UNIX is a registered trademark in the United States and other countries, licensed exclusively through The Open Group.
- VxWorks, Wind River Systems, the Wind River Systems logo, wind, CrossWind, Tornado, and WindSh are trademarks or registered trademarks of Wind River Systems, Inc.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Revision History

Rev.	Date	Notes
01	May 31/01	
02	Dec 25/01	
03	Jul 15/02	
04	Nov 1/02	
05	Nov 12/02	
06	May 15/03	
07	Jul 25/03	

Table of Contents

Preface	Preface-1
----------------------	-----------

About This Manual	Preface-1
-------------------------	-----------

Related Manuals	Preface-1
-----------------------	-----------

Typographic Convention	Convention-1
-------------------------------------	--------------

Typeface Conventions and Symbols	Convention-1
--	--------------

1. Basic System Library	1-1
--------------------------------------	-----

1. 1 Library Functions	1-1
1. 1. 1 AT_atexit	1-1
1. 1. 2 AT_calloc	1-3
1. 1. 3 AT_clock_gettime	1-4
1. 1. 4 AT_diff_timespec	1-7
1. 1. 5 AT_event	1-9
1. 1. 6 AT_evlongjmp	1-14
1. 1. 7 AT_evmask	1-17
1. 1. 8 AT_evwait	1-20
1. 1. 9 AT_fork	1-23
1. 1.10 AT_forkkill	1-26
1. 1.11 AT_forkwait	1-27
1. 1.12 AT_free	1-29
1. 1.13 AT_free_named	1-30
1. 1.14 AT_get_channel	1-31
1. 1.15 AT_get_pin	1-33
1. 1.16 AT_getchar	1-35
1. 1.17 AT_gets	1-36
1. 1.18 AT_malloc	1-37
1. 1.19 AT_malloc_named	1-38
1. 1.20 AT_perror	1-41
1. 1.21 AT_printf	1-42
1. 1.22 AT_printErrno	1-43
1. 1.23 AT_putchar	1-44
1. 1.24 AT_puts	1-45
1. 1.25 AT_scanf	1-46
1. 1.26 AT_strerror	1-47

1. 1.27	AT_strtor	1-48
1. 1.28	AT_strtorm	1-50
1. 1.29	ATL_exec	1-52
1. 1.30	ATL_get_char	1-60
1. 1.31	ATL_get_errinfo	1-62
1. 1.32	ATL_get_inte	1-64
1. 1.33	ATL_get_intem	1-66
1. 1.34	ATL_get_logi	1-68
1. 1.35	ATL_get_logim	1-70
1. 1.36	ATL_get_real	1-72
1. 1.37	ATL_get_realm	1-74
1. 1.38	ATL_init	1-76
1. 1.39	ATL_init2	1-79
1. 1.40	ATL_proc	1-81
1. 1.41	ATL_set_char	1-84
1. 1.42	ATL_set_inte	1-85
1. 1.43	ATL_set_intem	1-88
1. 1.44	ATL_set_logi	1-90
1. 1.45	ATL_set_logim	1-91
1. 1.46	ATL_set_real	1-93
1. 1.47	ATL_set_realm	1-95

2. Tester Access Library 2-1

2. 1	ATL Statement Correspondence Table	2-2
2. 2	Utility - Function Table	2-5
2. 3	Notes on Multi-dimensional Arrays of C and ATL	2-6
2. 4	Reference	2-7
2. 4. 1	AT_read_result	2-7
2. 4. 2	AT_read_fail	2-9
2. 4. 3	AT_read_fail_test_cnt	2-11
2. 4. 4	AT_read_cnt	2-13
2. 4. 5	AT_read_ctcnt	2-15
2. 4. 6	AT_read_wfcnt	2-18
2. 4. 7	AT_read_wfctcnt	2-21
2. 4. 8	AT_set_result	2-24
2. 4. 9	AT_reset_result	2-25
2. 4.10	AT_set_fail	2-26
2. 4.11	AT_reset_fail	2-27
2. 4.12	AT_set_cnt	2-28
2. 4.13	AT_reset_cnt	2-30
2. 4.14	AT_set_wfcnt	2-32

2. 4.15	AT_reset_wfcnt	2-33
2. 4.16	AT_set_sort	2-34
2. 4.17	AT_read_sort	2-35
2. 4.18	AT_set_stn	2-37
2. 4.19	AT_reset_stn	2-38
2. 4.20	AT_read_dct_pdata	2-39
2. 4.21	AT_read_dct_pdatam	2-41
2. 4.22	AT_read_dct_fail	2-43
2. 4.23	AT_read_dct_failm	2-45
2. 4.24	AT_read_dct_hfail	2-48
2. 4.25	AT_read_dct_hfailm	2-50
2. 4.26	AT_read_dct_lfail	2-53
2. 4.27	AT_read_dct_lfailm	2-55
2. 4.28	AT_read_dct_vs	2-58
2. 4.29	AT_read_dct_data	2-60
2. 4.30	AT_read_tpin	2-62
2. 4.31	AT_read_fpin	2-64
2. 4.32	AT_set_category	2-66
2. 4.33	AT_reset_category	2-67
2. 4.34	AT_read_category	2-68
2. 4.35	AT_set_fkmode	2-70
2. 4.36	AT_reset_fkmode	2-71
2. 4.37	AT_read_fk	2-72
2. 4.38	AT_read_stn	2-73
2. 4.39	AT_read_test	2-74
2. 4.40	AT_read_stncnt	2-75
2. 4.41	AT_read_dutcnt	2-76
2. 4.42	AT_send_gpib_function	2-77
2. 4.43	AT_send_gpib_data	2-79
2. 4.44	AT_read_gpib_srq	2-81
2. 4.45	AT_read_gpib_data	2-82
2. 4.46	AT_gpib_timeout	2-84
2. 4.47	AT_send_ext	2-85
2. 4.48	AT_read_ext	2-86
2. 4.49	AT_set_decision	2-87
2. 4.50	AT_set_rejection	2-89
2. 4.51	AT_set_rejectionm	2-91
2. 4.52	AT_set_jdut	2-93
2. 4.53	AT_set_jdutm	2-95
2. 4.54	AT_reset_jdut	2-98
2. 4.55	AT_set_rdut	2-99
2. 4.56	AT_read_ddut	2-100
2. 4.57	AT_read_sdut	2-102
2. 4.58	AT_read_mdut	2-104

2. 4.59	AT_read_edut	2-106
2. 4.60	AT_read_cdut	2-108
2. 4.61	AT_read_pdut	2-110
2. 4.62	AT_read_pdut_t	2-112
2. 4.63	AT_read_fdut	2-114
2. 4.64	AT_read_fdut_t	2-116
2. 4.65	AT_read_gfdut	2-118
2. 4.66	AT_set_sdut	2-120
2. 4.67	AT_set_sdutm	2-122
2. 4.68	AT_set_exclusion	2-124
2. 4.69	AT_set_exclusionm	2-126
2. 4.70	AT_reset_exclusion	2-129
2. 4.71	AT_reset_exclusionm	2-131
2. 4.72	AT_set_exclusion2	2-134
2. 4.73	AT_set_exclusionm2	2-136
2. 4.74	AT_reset_exclusion2	2-139
2. 4.75	AT_reset_exclusionm2	2-141
2. 4.76	AT_read_func	2-144
2. 4.77	AT_read_funcm	2-146
2. 4.78	AT_read_func_block	2-149
2. 4.79	AT_read_cumfunc	2-151
2. 4.80	AT_read_cumfuncm	2-153
2. 4.81	AT_read_cumfunc_block	2-156
2. 4.82	AT_read_ercode	2-158
2. 4.83	AT_write_afm	2-159
2. 4.84	AT_read_afm_confcnt	2-169
2. 4.85	AT_read_afm_confdat	2-172
2. 4.86	AT_read_afm_dutselect	2-175
2. 4.87	AT_read_afm_memory	2-177
2. 4.88	AT_read_afm_failaddr	2-182
2. 4.89	AT_read_afm_failcnt	2-191
2. 4.90	AT_read_afm_mcnt	2-200
2. 4.91	AT_read_cfm_memory	2-206
2. 4.92	AT_read_dbm	2-213
2. 4.93	AT_read_cfm_failcnt	2-215
2. 4.94	AT_read_cbm	2-220
2. 4.95	AT_set_pdfm	2-222
2. 4.96	AT_read_pdfm	2-224

3. ATL Library 3-1

3. 1 Useful Utility Programs 3-1

3. 1. 1 CLOCKGET 3-1

Index I-1

Preface

About This Manual

This manual describes details of the library functions provided by the hybrid controller.

Related Manuals

ASX/U-51 Hybrid Controller Users Manual(No. : 8409598)

ADVANsite Test System Software Installation/System Start User's Manual Vol1 Basic OS Installation(No. : 8227728)

ADVANsite Test System Software Installation/System Start User's Manual Vol2 Tester Environment Software Installation(No. : 8227729)

ATL-51 Test Plan Program Reference Manual(No. : 8431498)

XATL/U-51 Cross Compiler Users Manual(No. : 8256237)

T5500 Series Tester Utility Programs Reference Manual(No. : 8256295)

T5500 Series Controller Commands Reference Manual(No. : 8256297)

T5500 Series Memory Test System Control Box/Trigger Panel Users Manual(No. : 8256298)

Error Code Table Reference Manual(No. : 8141091)

ATLworks Tool Users Manual(No. : 8311174)

Typographic Convention

Typeface Conventions and Symbols

The typefaces and symbols listed below have special meanings in this manual, as explained below.

Item	Example
Simultaneous key input	[CTRL]-[C], [CTRL]-[X]
Name of manual to be referenced	<i>Tool Reference Manual</i>
Name of chapter or section to be referenced	1. "Overview"

In addition, the following terms express a specific meaning or objective.

Term	Meaning or Objective
Hybrid controller	The tester controller on which the functions explained in this manual are installed. This term is used to distinguish this tester controller from those earlier than ASX/U-51 Rev.6.00.
Controller	Tester controller software
Tester processor	A tester processor mounted on a test system. This manual refers to TP5 or TP6.
Host system Host	A workstation connected to an ADVANsite test system. The workstation that provides various services for the tester processor.
Target system Target	The tester processor mounted on a test system. This term is used to distinguish a tester processor from a host system.
C language	C language which complies with ANSI standards (ANSI/ISO/IEC 9899)
C++ language	C++ language which complies with ANSI standards (ISO/IEC 14882)
C program C++ program ATL program	A program written in the C, C++, or ATL language. These terms are used when it is not necessary to distinguish between source and object programs or when the program type can be determined from the context. Unless stated otherwise, "C program" can also refer to a C++ program.

1. Basic System Library

This chapter explains the library functions related to the basic system of the hybrid controller.

1. 1 Library Functions

1. 1. 1 AT_atexit

Function: This library function registers a function to be called when a program ends.

Syntax

```
#include "AT/hybrid.h"

int AT_atexit
(
    void      (*func) (void),          /* new function */
    void      (**pfunc) (void)        /* previous function */
);
```

Description

This function registers a function specified in the argument "func" as the function being called when the program ends. Use of this function enables a program to release system resources by itself even if it is forcibly terminated by [CTRL]-[C].

If the argument "func" is not NULL, the function specified in "func" is called with no argument when the program ends, and it is executed in the context of the system task tUsrProc. The function specified in "func" must not perform any processing other than releasing the system resources reserved at execution. Pay particular attention to exit(), abort(), and other functions that change task operations. If any of these functions are executed, the system may stop responding.

The function specified in the argument "func" must have the following format.

```
void func(void);
```

If the argument "func" is NULL, the previously registered function is canceled.

If the argument "pfunc" is not NULL, the pointer to the previously registered function is set to the area indicated by "pfunc". If no function has been specified, NULL is assigned to the area indicated by "pfunc".

When using this function, only one function can be registered at a time.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

Example

(1) Example of Registering the Cleanup_procedure Function

```
#include "AT/hybrid.h"
#include "stdlib.h"
#include "fcntl.h"
#include "unistd.h"

static int  fd = -1;

static void cleanup_procedure(void)
{
    if(fd >= 0){
        close(fd);
        fd = -1;
    }
    return;
}

int test_main(void)
{
    :
    /* close fd even when interrupted */
    AT_atexit(cleanup_procedure, NULL);

    if((fd = open("XXX", O_RDONLY, 0)) == -1){
        AT_printf("Error: open\n");
        exit(1);
    }
    :
    return (0);
}
```

1. 1. 2 AT_calloc

Function: This library function allocates dynamic memory.

Syntax

```
#include "AT/hybrid.h"

void *AT_calloc
(
    size_t    nelem,          /* number of elements */
    size_t    elsize         /* size of each element */
);
```

Description

This function allocates a memory block to an array with number of elements, nelem, of elemental size, elsize. The allocated area is initialized to 0.

The allocated memory space varies depending on the memory usage. A maximum of 96 MB can be physically allocated at a time.

➔ For more information, refer to [1. 1. 12 "AT_free" on page 1-29](#).

Return Value

If the function ends successfully, it returns the pointer to the allocated memory block.

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

ENOMEM

The memory area is insufficient.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

1. 1. 3 AT_clock_gettime

Function: This utility program reads the CPU time.

Syntax

```
#include "AT/hybrid.h"

int AT_clock_gettime
(
    struct timespec    *tdata    /* where to store CPU time */
);

struct timespec {
    time_t      tv_sec;          /* seconds      */
    long        tv_nsec;        /* nano seconds */
};
```

Description

This function reads the current CPU time.

In the argument "tdata", specify the pointer that is set to the struct timespec structure that stores the CPU time. Data in seconds is stored in the member "tv_sec" of the struct timespec structure and data in nanoseconds is stored in the member "tv_nsec". The CPU time is (tv_sec+tv_nsec x10⁻⁹). The range of the data stored in tv_nsec is 0 ≤ tv_nsec < 1,000,000,000.

➔ For more information, refer to [1. 1. 4 "AT_diff_timespec" on page 1-7](#) and [3. 1. 1 "CLOCKGET" on page 3-1](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

Example

(1) Example of a Program That Uses the Function to Measure the Execution Time

- C program

```

#include "AT/hybrid.h"

int test_main(void)
{
    struct timespec statm,endtm;
    struct timespec total;

    struct timespec from,to;
    struct timespec resulta,resultb;
    :
    /*reads the CPU time when the program starts */
    AT_clock_gettime(&statm);
    :
    /* loads the ATL program */
    ATL_init("SAMPLE");
    :
    :
    /* reads the CPU time when the program starts */
    AT_clock_gettime(&from);
    :
    :
    /* reads the CPU time when the program starts */
    AT_clock_gettime(&to);
    :
    /* calculates execution time of A */
    AT_diff_timespec(&from,
                    &to,
                    &resulta);
    :
    :
    /* executes ATL program */
    ATL_exec(ATL_GOSUB, "LB1", 4,
            INTE2(&from.tv_sec),
            INTE2(&from.tv_nsec),
            INTE2(&to.tv_sec),
            INTE2(&to.tv_nsec));
    :
    /* calculates execution time of B */
    AT_diff_timespec(&from,
                    &to,
                    &resultb);
    :
    :
    /* reads the CPU time when the program ends */
    AT_clock_gettime(&endtm);
    :
    /* calculates execution time of entire program */
    AT_diff_timespec(&statm,
                    &endtm,
                    &total);
    :
    return(0);
}

```

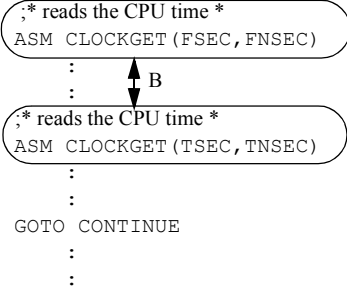
The execution time for process A is assigned to "resulta".

The execution time for process B is assigned to "resultb".

The total execution time for the program is assigned to "total".

- ATL program

```
PRO SAMPLE SOC1  
  
LB1: ARGUMENT (FSEC, FNSEC,  
              TSEC, TNSEC)  
  
    (* reads the CPU time *)  
    ASM CLOCKGET (FSEC, FNSEC)  
    :  
    :  
    :  
    (* reads the CPU time *)  
    ASM CLOCKGET (TSEC, TNSEC)  
    :  
    :  
    GOTO CONTINUE  
    :  
    :  
STOP  
END
```



1. 1. 4 AT_diff_timespec

Function: This library function calculates the difference between values stored in the struct timespec structures.

Syntax

```
#include "AT/hybrid.h"

int AT_diff_timespec
(
    struct timespec    *from    /* beginning time        */
    struct timespec    *to      /* end time          */
    struct timespec    *result  /* where to store result */
);

struct timespec {
    time_t    tv_sec;          /* seconds          */
    long      tv_nsec;         /* nano seconds    */
};
```

Description

This function calculates the difference between time values stored in the struct timespec structures.

In the arguments "from", "to", and "result", specify the pointers that are set to the struct timespec structures.

The argument "result" is used to store the absolute time value obtained by subtracting the data stored in the argument "from" from the data stored in the argument "to".

Note that the member "time_t tv_sec" of the struct timespec structure are an unsigned long type.

The member "long tv_nsec" of the struct timespec structure range is

$0 \leq \text{tv_nsec} < 1,000,000,000$.

If a value out of the range is specified, an error occurs.

➔ For more information, refer to [1. 1. 3 "AT_clock_gettime" on page 1-4](#) and [3. 1. 1 "CLOCKGET" on page 3-1](#).

➔ For more information on using this function, refer to the example in [1. 1. 3 "AT_clock_gettime" on page 1-4](#).

Return Value

If "from" is larger than "to", 2 is returned.

If "from" and "to" are the same value, 1 is returned.

If "from" is less than "to", 0 is returned.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_RANGE

The specified value is out of range.

1. 1. 5 AT_event

Function: This library function sets an event handler.

Syntax

```
#include "AT/hybrid.h"

int AT_event
(
    int          event,          /* target event code */
    void         (*handler)(int) /* pointer to handler function */
);
```

Description

This function sets an event handler.

Specify the event code corresponding to the target event in the argument "event". Event codes are defined as macros which are listed below.

Event Code	Event Class	Description
ATEV_DCTF	Synchronous	A failure occurred during the DC parametric test.
ATEV_FCTF	Synchronous	A failure occurred during a functional test.
ATEV_RSTE	Synchronous	A runtime error or system error occurred during execution of an ATL program.
ATEV_MRAANLY	Asynchronous	MRA4 analysis ended.
ATEV_MRARSLT	Asynchronous	Creation of the MRA4 repair analysis file finished.
ATEV_MRAUSRP	Asynchronous	The MRA4 user process terminated.

Each event code is always a value other than 0.

➔ For more information on events and event classes, refer to [ASX/U-51 Hybrid Controller Users Manual\(No. : 8409598\)](#).

The argument "handler" specifies the pointer to the function to be set as an event handler. The function specified in the argument "handler" must have the following format:

```
void handler
(
    int    event
);
```

The following predefined event handlers can also be specified in the argument "handler".

Event Handler	Description
ATEV_DFL	Specifies the default event handler.
ATEV_IGN	Specifies that the event is to be ignored.

Each event has a default event handler. If an event handler is not specified, the default event handler is executed. The default event handler for each event operates as shown below.

Event Code	Default Operation
ATEV_DCTF	Ignored.
ATEV_FCTF	Ignored.
ATEV_RSTF	Promptly terminates ATL program execution. The ATL_exec() function returns an error to the caller. An error is sent to the handler or prober, and the error lamp on the control box (C-BOX) goes on.
ATEV_MRAANLY	Ignored.
ATEV_MRARSLT	Ignored.
ATEV_MRAUSRP	Ignored.

The event handler function specified in the argument "handler" is called when the event specified in the argument "event" is delivered. The event code of the corresponding event that caused the event handler to be called is passed to the argument "event" of the handler() function.

The event handler is reset to the default event handler ATEV_DFL immediately before executing the specified event handler. To use the same event handler continuously, the event handler must be reset within the event handler each time it is executed. An event handler for a synchronous event cannot be set within an event handler for an asynchronous event.

Depending on the event class, the event handler for each event is set as follows during program execution.

Synchronous event

- When the ATL_init() function is executed, the event handler is implicitly reset to ATEV_DFL.
- When the ATL_exec() function executes an ATL external subroutine program, the event handler specified by the AT_event function becomes invalid.
- If this function and an ON statement of an ATL program are executed for the same event, the last setting becomes valid.
- If a synchronous event handler is set within a synchronous event handler, the specified event handler is enabled after the event handler is returned.

- In other cases, the event handler specified in this function is enabled when the function is executed.

Asynchronous event

- If an asynchronous event handler is specified within an asynchronous event handler, the specified event handler is enabled after the event handler returned.
- In other cases, the event handler specified in this function is enabled when the function is executed.

➔ For more information, refer to [1. 1. 6 "AT_evlongjmp" on page 1-14](#), [1. 1. 7 "AT_evmask" on page 1-17](#), and [1. 1. 8 "AT_evwait" on page 1-20](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Registering an Fctf_handler Function for an ATEV_FCTF Event

C program:

```
#include "AT/hybrid.h"

static void fctf_handler(int evcode)
{
    AT_printf("now in fctf_handler\n");
    :
    /* user defined process */
    :
    /* reset FCTF handler */
    AT_event(ATEV_FCTF, fctf_handler);

    return;
}

int test_main(void)
{
    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }
    /* set event handler for FCTF */
    AT_event(ATEV_FCTF, fctf_handler);

    if(ATL_exec(ATL_TEST, 100) != 0){
        AT_printf("Error: ATL_exec\n");
        exit(1);
    }

    return (0);
}
```

ATL program:

```
PRO SAMPLE
:
:
TEST 100
:
:
REG MPAT PC=#XX
MEAS MPAT
RETURNC

END
```

If the MEAS statement of the ATL program causes a failure during program execution, the following message is displayed on the VKT.

```
now in fctf_handler
```

1. 1. 6 AT_evlongjmp

Function: This library function executes a nonlocal jump from a synchronous event handler.

Syntax

```
#include "AT/hybrid.h"

void AT_evlongjmp
(
    jmp_buf  env,      /* environment saved by setjmp() */
    int      val       /* value to be returned by setjmp() */
);
```

Description

This function initializes the ATL program context to the status that no ATL program is being executed, and restores the environment saved by the latest `setjmp()` call with the same argument "env".

This function operates in the same way as the `longjmp()` function of the C standard library, except that this function initializes the ATL program context.

➔ For more information on the `longjmp()` function, refer to "VxWorks Reference Manual".

Use this function to execute a nonlocal jump from a synchronous event handler. If this function is used other than from a synchronous event handler, it works exactly the same as `longjmp()`.

Return Value

This function does not return to the caller. Instead, the `setjmp()` function corresponding to the argument "env" returns with the value of "val". If 0 is specified in the argument "val", the return value of the `setjmp()` function is 1.

➔ For more information, refer to [1. 1. 5 "AT_event" on page 1-9](#) and the description of `setjmp()` and `longjmp()` in the "VxWorks Reference Manual".

Example

- (1) Example of Skipping the Test and Going to the Next Test When the ATEV_FCTF Event Occurs

C program:

```
#include "AT/hybrid.h"

static jmp_buf    env;
static int        testno;

static void handler(int evcode)
{
    AT_evlongjmp(env, evcode);
}

int test_main(void)
{
    int    jumped_from_handler;

    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }

    /* set event handler for ATEV_FCTF */
    AT_event(ATEV_FCTF, handler);

    if((jumped_from_handler = setjmp(env)) == 0){
        for(testno = 100; testno < 110; testno++){
            if(ATL_exec(ATL_TEST, testno) != 0){
                AT_printf("Error: ATL_exec: testno = %d\n", testno);
                exit(1);
            }
        }
    } else {
        AT_printf("jumped from handler.\n");
        AT_printf("skipping TEST %d-109\n", testno);
    }

    /* next test process */
    :
    :
    return (0);
}
```

ATL program:

```
PRO SAMPLE
:
:
TEST 100
:
  REG MPAT PC=#XX
  MEAS MPAT
  RETURN

TEST 101
:
  REG MPAT PC=#XX
  MEAS MPAT
  RETURN
:
:
END
```

If the MEAS statement for TEST 104 in the ATL program causes a failure during program execution, the following message is displayed on the VKT.

```
jumped from handler
skipping TEST 104-109
```

1. 1. 7 AT_evmask

Function: This library function blocks or unblocks an asynchronous event.

Syntax

```
#include "AT/hybrid.h"

int AT_evmask
(
    int      event,    /* target event */
    int      how       /* to block or to unblock */
);
```

Description

This function blocks or unblocks the delivery of the specified asynchronous event.

Specify the event code of the target asynchronous event in the argument "event". Use one of the following macros to specify the argument "how" to block or unblock the event:

ATEV_BLOCK

Blocks the delivery of the asynchronous event specified in the argument "event". When a blocked event occurs, the event is placed in the event queue and remains there until it is unblocked. The event queue can hold up to 32 events. Events occurring after the event queue becomes full are discarded.

ATEV_UNBLOCK

Unblocks the delivery of the asynchronous event specified in the argument "event". If the event exists in the event queue when the function is called, the event is delivered before the function returns.

This function can also be called from an asynchronous event handler.

➔ For more information, refer to [1. 1. 5 "AT_event" on page 1-9](#) and [1. 1. 8 "AT_evwait" on page 1-20](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

Example

(1) Example of Blocking the Delivery of Asynchronous Events during Specific Processing

```
#include "AT/hybrid.h"

static void mra_handler(int evcode)
{
    switch(evcode){
        case ATEV_MRAANLY:    ... /* process for MRA analysis end */
                               break;
        case ATEV_MRARSLT:    ... /* process for MRA result ready */
                               break;
        case ATEV_MRAUSRP:    ... /* process for MRA user process end */
                               break;
    }
}

int test_main(void)
{
    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }
    /* set event handler for MRA events */
    AT_event(ATEV_MRAANLY, mra_handler);
    AT_event(ATEV_MRARSLT, mra_handler);

    /* block MRA events during TEST 100 */
    AT_evmask(ATEV_MRAANLY, ATEV_BLOCK);
    AT_evmask(ATEV_MRARSLT, ATEV_BLOCK);

    if(ATL_exec(ATL_TEST, 100) != 0){
        AT_printf("Error: ATL_exec\n");
        exit(1);
    }

    /* unblock MRA events */
    AT_evmask(ATEV_MRAANLY, ATEV_UNBLOCK);
    AT_evmask(ATEV_MRARSLT, ATEV_UNBLOCK);

    :

    return (0);
}
```

1. 1. 8 AT_evwait

Function: This library function waits for the occurrence of an asynchronous event.

Syntax

```
#include "AT/hybrid.h"

int AT_evwait
(
    int      *evcode,          /* where to set occurred event */
    int      timeout          /* timeout in msec */
);
```

Description

This function waits for the occurrence of an asynchronous event in the context of a normal program. It provides a mechanism to process asynchronous events synchronously.

When called, the function checks the pending events on the event queue. If an event is found, the function stores the corresponding event code in the area specified by the argument "evcode" and promptly returns. If no event is found, the calling task is blocked until a new event occurs within the wait time specified in the argument "timeout".

The function will always select the first-queued event, or the oldest event, in the event queue. The selected event is removed from the event queue. This event selection does not depend on the event mask setting. Waiting is possible for blocked events. If an event handler is specified for the event selected by the function, the event handler is not executed.

Specify the maximum wait time in milliseconds in the argument "timeout". The following macros that are defined in advance can be specified for the same purpose.

ATEV_NO_WAIT

Only checks the event queue and does not wait for an event.

ATEV_WAIT_FOREVER

Waits for an event with no time limit.

This function cannot be called from the asynchronous event handler.

➔ For more information, refer to [1. 1. 5 "AT_event" on page 1-9](#) and [1. 1. 7 "AT_evmask" on page 1-17](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_TIMEOUT

A timeout occurred while the function was waiting for an event.

Example

(1) Example of Specifying Waiting for an MRA Event before Execution of TEST 101 (TEST 101 Is Executed after Event Wait Processing Is Complete.)

```
#include "AT/hybrid.h"

int test_main(void)
{
    int    evcode;

    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }

    /* block ATEV_MRARSLT event not to be ignored */
    AT_evmask(ATEV_MRARSLT, ATEV_BLOCK);

    if(ATL_exec(ATL_TEST, 100) != 0){
        AT_printf("Error: ATL_exec: TEST 100\n");
        exit(1);
    }

    /* wait for ATEV_MRARSLT before proceeding to next test */
    while(1){
        if(AT_evwait(&evcode, ATEV_WAIT_FOREVER) != 0){
            AT_printf("Error: AT_evwait\n");
            exit(1);
        }
        if(evcode == ATEV_MRARST)
            break;
    }
    /* OK, here we got MRARSLT */
    if(ATL_exec(ATL_TEST, 101) != 0){
        AT_printf("Error: ATL_exec: TEST 101\n");
        exit(1);
    }

    :

    return (0);
}
```


1. 1. 9 AT_fork

Function: Execute a command on the host system.

Syntax

```
#include "AT/hybrid.h"

int AT_fork
(
    const char *command,      /* command string */
    int mode,                 /* execution mode */
    int *stat_loc             /* where to store exit status */
);
```

Description

This function generates a process on a host system and executes a shell-command specified in the argument "command". The maximum string length that can be specified in the argument "command" is 65536 characters including the terminating null character '\0'. The process in which a command is executed inherits the environment of the controller process which is a parent process.

Argument "mode" is used to determine that the a completion of command execution is waited or not. Any of following macro can be specified to "mode".

FORK_WAIT

WAIT mode

When this mode is specified, this function waits for a completion of command execution, and does not return until the execution is completed.

If the argument "stat_loc" is not NULL, a completion status of command is stored in the area specified by stat_loc.

FORK_NOWAIT

NOWAIT mode

When this mode is specified, this function returns without waiting for a completion of command execution.

Specification of argument "stat_loc" does not have meaning and it is ignored.

In the NOWAIT mode, this function returns a process ID of the generated process. AT_forkwait() and LIB FORKWAIT with the value allows a completion of command to be waited.

Since this function shares the process status table with LIB FORK, maximum 24 processes can be executed at the same time when the LIB FORK is used.

Process status table is used to maintain the completion status of process. The table is released by executing the `AT_forkwait()`, `LIB FORKWAIT`, and `LIB FORKLIST`. Therefore, to use this function or `LIB FORK` exceeding 24 processes, the process status table has to be released by executing the `AT_forkwait()`, `LIB FORKWAIT`, and `LIB FORKLIST`.

➔ For more information, refer to [1. 1. 11 "AT_forkwait" on page 1-27](#), [1. 1. 10 "AT_forkkill" on page 1-26](#), and [T5500 Series Tester Utility Programs Reference Manual\(No. : 8256295\)](#).

Return Value

If the function ends successfully, it returns 0 in the `WAIT` mode. In the `NOWAIT` mode, this function returns a process ID of the generated process. The process ID is a non-negative integer. If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_fork_OPENLOG

The error log file cannot be opened.

S_fork_NOSPACE

No space in the process status table.

S_fork_EXEC_ERROR

Error on the process generation.

Example

(1) Program Example of Execution for Command "sample -x" on the Host System

```
#include "AT/hybrid.h"

int test_main(void)
{
    int    status;

    if(AT_fork("sample -x", FORK_WAIT, &status) == -1){
        AT_perror("AT_fork");
        exit(1);
    }

    return (0);
}
```

1. 1.10 AT_forkkill

Function: The command is forcibly terminated.

Syntax

```
#include "AT/hybrid.h"

int AT_forkkill
(
    int      pid      /* process ID */
);
```

Description

This function forcibly terminates the process generated by the AT_fork() or LIB FORK in the NOWAIT mode.

For the argument "pid", specify a process ID of the process to be forcibly terminated.

➔ For more information, refer to [1. 1. 9 "AT_fork" on page 1-23](#), [1. 1. 11 "AT_forkwait" on page 1-27](#), and [T5500 Series Tester Utility Programs Reference Manual\(No. : 8256295\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

No process specified to the argument "pid" exists.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

1. 1.11 AT_forkwait

Function: Waits for a completion of command.

Syntax

```
#include "AT/hybrid.h"

int AT_forkwait
(
    int      pid,           /* process ID */
    int      *stat_loc      /* where to store exit status */
    int      timeout        /* timeout */
);
```

Description

This function waits for a completion of the process generated by the AT_fork() or LIB FORK in the NOWAIT mode.

For the argument "pid", specify a process ID of the process waiting for a completion.

Specify the maximum wait time in milliseconds in the argument "timeout". If the completion of the specified process is not confirmed within a specified wait time, this function will be failed. If the completion of specified process is confirmed, the process status table for the process is released. When the argument "stat_loc" is not NULL, this function stores completion status of the process specified in the area indicated by "stat_loc".

➔ For more information, refer to [1. 1. 9 "AT_fork" on page 1-23](#), [1. 1. 10 "AT_forkkill" on page 1-26](#), and [T5500 Series Tester Utility Programs Reference Manual\(No. : 8256295\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

No process specified to the argument "pid" exists.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_fork_TIMEOUT

The waiting time has passed.

1. 1.12 AT_free

Function: This library function frees the memory block.

Syntax

```
#include "AT/hybrid.h"

void AT_free
(
    void      *ptr      /* pointer to memory block to free */
);
```

Description

This function frees the memory block specified in the argument "ptr". The memory block specified in the argument "ptr" must be a block that was allocated by AT_malloc() or AT_calloc(). If "ptr" is not a block allocated by one of these functions or the block is overrun, the result is undefined.

➔ For more information, refer to [1. 1. 18 "AT_malloc" on page 1-37](#) and [1. 1. 2 "AT_calloc" on page 1-3](#).

Return Value

None

1. 1.13 AT_free_named

Function: This library function frees a named protected memory block.

Syntax

```
#include "AT/hybrid.h"

void AT_free_named
(
    void      *ptr      /* pointer to memory block to free */
);
```

Description

This function frees the memory block specified in the argument "ptr". The memory block specified in the argument "ptr" must be a block that was allocated by AT_malloc_named(). If the memory block specified in the argument "ptr" is not allocated using AT_malloc_named() or if the allocated memory block overruns, the result is undefined.

When this function frees the memory block, the memory block contents are not stored in a file.

➔ For more information, refer to [1. 1. 19 "AT_malloc_named" on page 1-38](#).

Return Value

None

1. 1.14 AT_get_channel

Function: DUT pin number is converted to the tester channel number.

Syntax

```
#include "AT/hybrid.h"

uint16_t AT_get_channel
(
    uint16_t pinno,      /* DUT pin kind and pin number */
);
```

Description

This function converts the DUT pin number to the tester channel number and returns it according to the channel connection relationship defined in the socket program.

Specify the type and number of the DUT pin for the tests in the argument "pinno". Use one of the following macros to specify the type and number of each DUT pin.

Macro	Types of DUT Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify an uint16_t-type DUT pin number (1 to 512) in macro argument "pin".

The function fails if:

- ATL_init() was not executed.
- DUT pin number is out of range.
- The value of DUT pin number exceeds the DR pin Max or CP pin Max defined in the socket program.

Details of conditions are as follows.

Macro Used for Specification of Argument "pinno"	Tester Channel after Socket Conversion	Conditions
AT_PD()	--	Exceeds the DR pin Max
AT_PC()	--	Exceeds the CP pin Max
AT_P()	I/O	Exceeds either of DR pin Max or CP pin Max
	Driver dedicated, DC dedicated	Exceeds the DR pin Max

- When the macro AT_P() is used to specify the argument "pinno", and tester channels connected to the specified DUT pins are different between driver pin and comparator pin.

Return Value

If the function ends successfully, it returns a tester channel number (1 to 512).

If the function fails, it returns 0 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_PRO_NOT_LOADED

The ATL test plan program is not loaded.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

1. 1.15 AT_get_pin

Function: The tester channel number is converted to the DUT pin number.

Syntax

```
#include "AT/hybrid.h"

uint16_t AT_get_pin
(
    uint16_t channelno, /* tester channel number */
    int      pinkind    /* kind of DUT pin number (driver/comparator) */
);
```

Description

This function converts the tester channel number to the DUT pin number and returns it according to the channel connection relationship defined in the socket program.

For the argument "channelno", specify the tester channel number between 1 to 512.

The DUT pin number returned by this function is either of driver pin number or comparator pin number. It can be selected by specifying to the argument "pinkind". Following macros can be specified to the argument "pinkind".

Macro	DUT Pin Number Returned by the Function
AT_DR	Driver pin number
AT_CP	Comparator pin number

The function fails if:

- ATL_init() was not executed.
- Tester channel number is out of range.
- The tester channel is connected to the DUT pin exceeding the DR pin Max or CP pin max defined in the socket program.
- The argument "pinkind" is specified with the macro which is not allowed for "pinkind".

Return Value

If the function ends successfully, it returns a DUT pin number (1 to 512).

If the function fails, it returns 0 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_PRO_NOT_LOADED

The ATL test plan program is not loaded.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example**(1) Example of Reading the Logical Sum of All STRB Failure Data of Tester Channel 33 in a 128 DUTs Parallel Test**

```
#define DUTNO 128

int foo(void)
{
    uint32_t dest[DUTNO];

    if(AT_read_func(1, AT_PC(AT_get_pin(33, AT_CP)), DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_func()\n");
        exit(1);
    }

    return(0);
}
```

1. 1.16 AT_getchar

Function: This library function reads the next input character from the VKT.

Syntax

```
#include "AT/hybrid.h"

int AT_getchar(void);
```

Description

This function reads the next character input from the VKT, converts it to the int type, and returns it.

Return Value

If the function ends successfully, it returns the read character.

If the function fails, it returns EOF.

1. 1.17 AT_gets

Function: This library function reads the next input line from the VKT.

Syntax

```
#include "AT/hybrid.h"

char *AT_gets
(
    char *buf          /* input buffer */
);
```

Description

This function reads characters from the VKT until a line feed character is encountered, then stores the characters in the area specified in the argument "buf". The terminal line feed character is ignored, and the read character string is terminated by the null character ('\0').

The area specified in the argument "buf" must be large enough to accommodate the input line, including the null character. If the size of the input line exceeds the size of "buf", the result is unpredictable.

➔ For more information, refer to the description of gets() and fgets().

Return Value

If the function ends successfully, it returns buf.

If the function fails, it returns NULL.

1. 1.18 AT_malloc

Function: This library function allocates dynamic memory.

Syntax

```
#include "AT/hybrid.h"
void *AT_malloc
(
    size_t      size      /* number of bytes to allocate */
);
```

Description

This function allocates a memory block of at least "size" bytes from the user memory area, and returns a pointer to the memory block. The pointer returned by this function is aligned on the appropriate boundary for arbitrary use.

The allocated memory space varies depending on the memory usage. A maximum of 96 MB can be physically allocated at a time.

➔ For more information, refer to [1. 1. 12 "AT_free" on page 1-29](#).

Return Value

If the function ends successfully, it returns the pointer to the allocated memory block.

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

ENOMEM

The memory area is insufficient.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

1. 1.19 AT_malloc_named

Function: This library function allocates named dynamic memory.

Syntax

```
#include "AT/hybrid.h"

void *AT_malloc_named
(
    size_t      size      /* number of bytes to allocate */
    const char  *name     /* name of memory block */
);
```

Description

This function allocates a memory block of at least "size" bytes from the user memory area, and returns a pointer to the memory block. The pointer returned by this function is aligned on the appropriate boundary for arbitrary use. This allocated memory block is not freed even if the user program ends.

A maximum of 32 characters can be used for the allocated memory block "name". If "name" is not a null pointer, the contents of the memory block are saved in the file "name" when the user program is unloaded or the controller is terminated.

➔ For more information on the user program unloading conditions, refer to Section 3.3, "Loading and Unloading C-program" in [ASX/U-51 Hybrid Controller Users Manual\(No. : 8409598\)](#).

The file is created in the current customer directory (usually where the controller is started). If the same file name exists, the old file is overwritten with the new one.

The contents of the memory block are dumped to the "name" file in binary format. Sometimes the file size is larger than the actual memory block "size", so the file size and allocated memory "size" do not always match. To recover the allocated memory block, refer to the examples shown below.

If this function fails to save the contents, the message shown below is displayed on the VKT:

```
PROTECTION MEMORY DATA SAVE FAIL : XXXX
```

The memory name is displayed where XXXX is indicated.

The memory block is automatically freed, regardless of whether the data in the memory is saved or not.

If the "name" is not a null pointer, a new memory block with the same "name" cannot be allocated. If a memory block with the same "name" already exists and the size is larger than "size", the pointer to the first allocated memory block is returned. If the size is less than "size", this function fails.

If "name" is a null pointer, a new memory block without a name is allocated. However, the memory block contents are not saved in the file.

The allocated memory space varies depending on the memory usage. A maximum of 96 MB can be physically allocated at a time.

➔ For more information, refer to [1. 1. 13 "AT_free_named" on page 1-30](#) and Section 3.3, "Loading and Unloading C-program" in [ASX/U-51 Hybrid Controller Users Manual\(No. : 8409598\)](#).

Return Value

If the function ends successfully, it returns the pointer to the allocated memory block.

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

ENOMEM

The memory area is insufficient.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

Example

(1) Example of Allocating the Memory Block for 32 Variables of Type Int32_t and Naming the Area Datafield.int32_t.32

```
int foo1(void)
{
    int32_t    *data;
    char       fname[] = "datafield.int32_t.32";

    if((data = AT_malloc_named(sizeof(int32_t) * 32, fname)) == NULL) {
        AT_perror("memory allocation error");
        exit(1);
    }

    return (0);
}
```

(2) Example of Recovering the Contents of the Memory Block That Was Allocated in the Above Example Using Another User Program

```
int foo2(void)
{
    int32_t    *data;
    char        fname[] = "anotherfield.int32_t.32";
    char        fname_saved[] = "datafield.int32_t.32";

    if((data = AT_malloc_named(sizeof(int32_t) * 32, fname)) == NULL) {
        AT_perror("memory allocation error");
        exit(1);
    }
    if(load_memfile(fname_saved, sizeof(int32_t), 32, data)) {
        AT_printf("memory recovery error\n");
        exit(1);
    }

    return (0);
}

int load_memfile(char *fname, size_t size, size_t nelem, void *dest)
{
    FILE        *fp;

    if((fp = fopen(fname, "rb")) == NULL) {
        AT_printf("file open error\n");
        return (1);
    }
    if(fread(dest, size, nelem, fp) != nelem) {
        AT_printf("file read error\n");
        fclose(fp);
        return (1);
    }
    fclose(fp);

    return (0);
}
```

1. 1.20 AT_perror

Function: This library function displays the error message corresponding to "errno".

Syntax

```
#include "AT/hybrid.h"

void AT_perror
(
    const char *errstr    /* message header string */
);
```

Description

This function outputs the character string specified in the argument together with an error message corresponding to the errno value of the calling task to the VKT and to the standard error output of the calling task.

The error message is output in one of the following formats according to the string specified in the argument "errstr".

- If the argument "errstr" is not NULL
The character string specified in the argument "errstr" is output followed by a colon ":" and a space character, which are then followed by the error message corresponding to "errno". A line feed character is output last.
- If the argument "errstr" is NULL
The error message corresponding to "errno" is output followed by a line feed character.

The contents of the error message are the same as those returned by the AT_strerror() function.

➔ For more information, refer to [1. 1. 26 "AT_strerror" on page 1-47](#).

Return Value

None

1. 1.21 AT_printf

Function: This library function outputs formatted data to the VKT.

Syntax

```
#include "AT/hybrid.h"

int AT_printf
(
    const char *fmt,    /* format string */
    ...                /* arbitrary arguments to write */
);
```

Description

This function outputs data under the same argument specifications as the `printf()` function of the ANSI-C standard library to the VKT.

➔ For more information, refer to the description of `printf()`.

Return Value

If the function ends successfully, it returns the number of output characters.

If the function fails, it returns a negative value.

1. 1.22 AT_printErrno

Function: This library function displays the error message corresponding to the specified error number.

Syntax

```
#include "AT/hybrid.h"

void AT_printErrno
(
    int    errno    /* error number */
);
```

Description

This function outputs the error message corresponding to the error number specified in the argument "errno" to the VKT and the standard error output of the calling task.

Either the error number defined by VxWorks or the error number defined by the hybrid controller can be specified in the argument "errno". If an undefined error number is specified, an error message is displayed in the following format.

```
Unknown errno = 0xXXXX
```

0xXXXX is the specified error number.

➔ For more information, refer to [1. 1. 20 "AT_perror" on page 1-41](#) and [1. 1. 26 "AT_strerror" on page 1-47](#).

Return Value

None

1. 1.23 AT_putchar

Function: This library function outputs a character to the VKT.

Syntax

```
#include "AT/hybrid.h"

int AT_putchar
(
    int c      /* character to write */
);
```

Description

This function outputs the character specified in the argument "c" to the VKT.

➔ For more information, refer to the description of putchar().

Return Value

If the function ends successfully, it returns the specified character.

If the function fails, it returns EOF.

1. 1.24 AT_puts

Function: This library function outputs a character string to the VKT.

Syntax

```
#include "AT/hybrid.h"

int AT_puts
(
    char *s    /* string to write */
);
```

Description

This function outputs the character string specified in the argument "s", together with a line feed character, to the VKT. It does not output the null character indicating the end of a character string.

➔ For more information, refer to the description of puts().

Return Value

If the function ends successfully, it returns the number of output characters.

If the function fails, it returns EOF.

1. 1.25 AT_scanf

Function: This library function reads formatted input from the VKT.

Syntax

```
#include "AT/hybrid.h"

int AT_scanf
(
    const char *fmt,    /* format string */
    ...                /* arbitrary arguments for output */
);
```

Description

This function reads input characters from the VKT according to the format specified by the argument "fmt". The function argument specifications are the same as those used for the scanf() function of the ANSI-C standard library.

➔ For more information, refer to the description of scanf().

Return Value

If the function ends successfully, it returns the number of converted formats.

If the function fails, it returns EOF.

1. 1.26 AT_strerror

Function: This library function obtains the error message corresponding to "errno".

Syntax

```
#include "AT/hybrid.h"

char *AT_strerror
(
    int    errnum    /* error number */
);
```

Description

This function returns a pointer to the error message string corresponding to the error number specified in the argument "errnum".

Either the error number defined by VxWorks or the error number defined by the hybrid controller can be specified in the argument "errnum".

This function is not reentrant because it returns a pointer to a static area of the library function.

➔ For more information, refer to [1. 1. 22 "AT_printErrno" on page 1-43](#) and [1. 1. 20 "AT_perror" on page 1-41](#).

Return Value

This function returns a pointer to the area holding the error message.

1. 1.27 AT_strtor

Function: This library function converts a string expression of a real number with or without a unit into the struct atreal structure.

Syntax

```
#include "AT/hybrid.h"

struct atreal *AT_strtor
(
    struct atreal      *dest,    /* where to put result */
    const char         *str      /* string expr of real num */
);
```

Description

This function converts a string expression of a real number, with or without a unit, specified in the argument "str" into struct atreal, and saves the results in the area indicated by the argument "dest".

The area specified in the argument "dest" must be provided by the caller. A character string consisting of a real number concatenated with a unit can be specified in the argument "str". The unit is optional. If no unit is specified, the specified value is handled as a real number without a unit. Character strings that can be specified as units are listed below. There is no differentiation between uppercase and lowercase letters.

Type	String to be Spec-ified	Unit
Voltage	V	Volt
	MV	Millivolt
Current	A	Ampere
	MA	Milliampere
	UA	Microampere
Time	S	Seconds
	MS	Millisecond
	US	Microsecond
	NS	Nanosecond
	PS	Picosecond

The converted value is obtained by first converting the character string representing the value of the argument "str" (using strtod() for the conversion) and then multiplying the results by the multiplier of the basic unit of the specified unit. If an overflow or underflow occurs during the conversion, the function fails.

➔ For more information, refer to the description of strtod().

Return Value

If the function ends successfully, it returns "dest".

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

ERANGE

Conversion to a floating-point number failed.

Example

(1) Example of Converting the Character String "20NS"

```
struct atreal v;

if(AT_strtor(&v, "20NS") == NULL){
    AT_printf("Error: AT_strtor\n");
    exit(1);
}
```

(2) Example of Using this Function with Another Function that Handles the Struct Areal Structure

```
struct atreal v;

/* setting 20NS to HB0025 */
if(ATL_set_real(SV_HB(25), AT_strtor(&v, "20NS")) != 0){
    AT_printf("Error: ATL_set_real\n");
    exit(1);
}
```

1. 1.28 AT_strtorm

Function: This library function converts a string expression of a real number with multiple units into the struct atreal structure.

Syntax

```
#include "AT/hybrid.h"

struct atreal *AT_strtorm
(
    struct atreal    *dest,    /* array to put result */
    size_t          nelem,    /* number of following arguments */
    ...              /* string expr of real nums */
);
```

Description

This function is a variation of the AT_strtor() function. It converts the string expressions of as many real numbers as specified in "nelem" in the variable argument part, and it saves the conversion results in the array specified in the argument "dest".

In the argument "dest", specify the pointer that is set to an array of the struct atreal structure that has at least the number of elements specified by the argument "nelem" or more. This area must be provided by the caller. In the variable argument part, specify string expressions with at least as many real numbers to be converted as the number specified in "nelem", delimiting each string with a comma. The method for specifying these strings is the same as that for AT_strtor().

The calling side is responsible for matching the size of the area specified in the argument "dest", the argument "nelem", and the number of variable arguments. If they are inconsistent, the behavior is undefined.

The converted results are saved from the beginning of the array specified in the argument "dest", in the sequence specified in the variable argument part. If the conversion fails, the function stops processing and returns an error.

➔ For more information, refer to [1. 1. 27 "AT_strtor" on page 1-48](#).

Return Value

If the function ends successfully, it returns "dest".

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

ERANGE

Conversion to a floating-point number failed.

Example

- (1) Example of Converting Character Strings "10NS", "20NS", "30NS", and "40NS" and Then Assigning Them to the Array "tim"

```
struct atreal tim[4];

if(AT_strtorm(tim, 4, "10NS", "20NS", "30NS", "40NS") == NULL){
    AT_printf("Error: AT_strtorm\n");
    exit(1);
}
```

1. 1.29 ATL_exec

Function: This library function executes an ATL program.

Syntax

```
#include "AT/hybrid.h"

int ATL_exec
(
    int      command,      /* command */
    ...      /* arguments to command */
);
```

Description

This function executes an ATL program as specified in the argument "command".

In the argument "command", specify a command to indicate how the ATL program is to be handled. The following commands can be specified.

Command	Description
ATL_GOSUB	Executes an internal subroutine of the ATL test plan program loaded by the ATL_init() function.
ATL_TEST	Executes a TEST statement of the ATL test plan program loaded by the ATL_init() function.
ATL_CALL	Executes an ATL external subroutine program.

Each command receives in the variable argument part the data required for command execution. The number and the type of arguments required for each command depend on the type of the command. If the number of specified arguments and their types do not satisfy the argument requirements of the specified command, the behavior of the function is undefined. The argument requirements for individual commands are explained in subsequent sections.

If a runtime or system error is caused by the ATL program executed by issuing this function with any of the commands, the function returns an error. The function also returns an error if a runtime or system error have occurred in the ATL program executed before this function is called. This function cannot be called from the asynchronous event handler. Other error conditions are explained in subsequent sections, separately for individual commands.

ATL_GOSUB

This command executes an internal subroutine of the ATL test plan program loaded by the ATL_init() function. In the variable argument part, specify the label name "label" for the branch destination of the internal subroutine, the number "narg", and the number "narg" of arguments which are passed to the subroutine.

```
ATL_exec(ATL_GOSUB, char *label, int narg, ...);
```

➔ For more information on how to specify the arguments to be passed to the subroutine, refer to ["Passing Arguments to a Subroutine" on page 1-54](#).

This command returns to the caller if the GOTO CONTINUE statement corresponding to this function call is executed in the ATL program. The command fails if:

- ATL_init() was not executed.
- Invalid data is specified in the argument "label".
- The label specified in the argument "label" does not exist.
- The value specified in the argument "narg" is a negative number or exceeds 36.
- The specification of an argument to be passed to the subroutine is invalid.
- The function failed to allocate memory space for arguments to be passed to the subroutine.

ATL_TEST

This command executes the TEST statement in the ATL test plan program loaded by the ATL_init() function. Specify the test number, testno, of the branch destination in the variable argument part.

```
ATL_exec(ATL_TEST, int testno);
```

This command returns to the caller if the RETURN statement is executed in the ATL program. The command fails if:

- ATL_init() was not executed.
- The test number specified in the argument "testno" does not exist.

ATL_CALL

This command calls an ATL external subroutine program. In the variable argument part, specify the external subroutine program name, `subr_name`, the number of argument "narg" to be passed to the external subroutine, and as many arguments to be passed to the subroutine as specified in "narg".

```
ATL_exec(ATL_CALL, char *subr_name, int narg, ...);
```

➔ For more information on how to specify the arguments to be passed to the subroutine, refer to ["Passing Arguments to a Subroutine" on page 1-54](#).

This command returns to the caller if the RETURN statement is executed in the ATL external subroutine program. The command fails if:

- The specification of the argument "subr_name" is invalid.
- The value specified in the argument "narg" is a negative number or exceeds 36.
- The specification of an argument to be passed to the subroutine is invalid.
- The function failed to allocate memory space for arguments to be passed to the subroutine.

Passing Arguments to a Subroutine

Specify the arguments to be passed to a subroutine by using the pre-defined macros with arguments shown below according to the argument type. The specified arguments are passed to an ATL program in the sequence they are specified. The calling side is responsible for matching between the macro used and the type of data or the values specified in the macro arguments. If these specifications are inconsistent, the behavior of the function is undefined. The value specified in the argument "narg" of this function must match the number of macros specified in the subsequent variable argument part.

When using a macro that passes an array or character string, a value equal to or greater than 65536 cannot be specified as the number of array elements or the size of the character string (value to be specified in the argument "sz" of the macro).

➔ For more information on precautions about passing floating-point numbers, refer to ["Notes on Passing a Floating-point Number" on page 1-57](#).

INTE1(int16_t *p)

This macro passes a signed 16-bit integer. In the the argument "p", specify the pointer that is set to an int16_t type object. The actual argument passed to the subroutine is an INTE[1] type variable without DIM. The argument is passed by reference.

INTE(int32_t *p)**INTE2(int32_t *p)**

Each of these macros passes a signed 32-bit integer. In the argument "p", specify the pointer that is set to an int32_t type object. The actual argument passed to the subroutine is an INTE[2] type variable without DIM. The argument is passed by reference.

DINTE1(int16_t *p, size_t sz)

This macro is used to pass an array of signed 16-bit integers. In the argument "p", specify the pointer that is set to an int16_t type array of at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of an INTE[1] type one-dimensional array of sz elements. The argument is passed by reference.

DINTE(int32_t *p, size_t sz)

DINTE2(int32_t *p, size_t sz)

Each of these macros is used to pass an array of signed 32-bit integers. In the argument "p", specify the pointer that is set to an int32_t type array of at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of an INTE[2] type one-dimensional array of sz elements. The argument is passed by reference.

LOGI1(uint16_t *p)

This macro is used to pass an unsigned 16-bit integer. In the argument "p", specify the pointer that is set to a uint16_t type object. The actual argument passed to the subroutine is a LOGI[1] type variable without DIM. The argument is passed by reference.

LOGI(uint32_t *p)

LOGI2(uint32_t *p)

Each of these macros passes an unsigned 32-bit integer. In the argument "p", specify the pointer that is set to a uint32_t type object. The actual argument passed to the subroutine is a LOGI[2] type variable without DIM. The argument is passed by reference.

DLOGI1(uint16_t *p, size_t sz)

This macro passes an array of unsigned 16-bit integers. In the argument "p", specify the pointer that is set to a uint16_t type array of at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of a LOGI[1] type one-dimensional array of sz elements. The argument is passed by reference.

DLOGI(uint32_t *p, size_t sz)

DLOGI2(uint32_t *p, size_t sz)

Each of these macros passes an array of unsigned 32-bit integers. In the argument "p", specify the pointer that is set to a uint32_t type array which has at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of a LOGI[2] type one-dimensional array of sz elements. The argument is passed by reference.

REAL2(double *p)

VREAL2(double *p)

Each of these macros passes a double-precision floating-point number of a C program as a single-precision real number (without a unit) of an ATL program. In the argument "p", specify the pointer that is set to a double type object. The actual argument passed to the subroutine is a REAL[2] type variable without DIM. A REAL2 argument is passed by reference, and a VREAL2 argument is passed by value.

REAL(struct atreal *p)

REAL3(struct atreal *p)

VREAL(struct atreal *p)

VREAL3(struct atreal *p)

Each of these macros passes a double-precision floating-point number with a unit of a C program as a single-precision real number with a unit of an ATL program. In the argument "p", specify the pointer that is set to an object of the struct atreal structure. The actual argument passed to the subroutine is a REAL[3] type variable without DIM. REAL and REAL3 arguments are passed by reference, and VREAL and VREAL3 arguments are passed by value.

REAL4(double *p)

VREAL4(double *p)

Each of these macros passes a double-precision floating-point number of a C program as a double-precision real number (without a unit) of an ATL program. In the argument "p", specify the pointer that is set to a double type object. The actual argument passed to the subroutine is a REAL[4] type variable without DIM. A REAL4 argument is passed by reference, and a VREAL4 argument is passed by value.

DREAL2(double *p, size_t sz)

DVREAL2(double *p, size_t sz)

Each of these macros passes an array of double-precision floating-point numbers of a C program as an array of single-precision real numbers (without units) of an ATL program. In the argument "p", specify the pointer that is set to a double type array of at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of a REAL[2] type one-dimensional array of sz elements. A DREAL2 argument is passed by reference, and a DVREAL2 argument is passed by value.

DREAL(struct atreal *p, size_t sz)

DREAL3(struct atreal *p, size_t sz)

DVREAL(struct atreal *p, size_t sz)

DVREAL3(struct atreal *p, size_t sz)

Each of these macros passes an array of double-precision floating-point numbers with a unit of a C program as an array of single-precision real numbers with a unit of an ATL program. In the argument "p", specify the pointer that is set to an array of the struct atreal structure that has at least sz elements. The actual argument passed to the subroutine is an all-element DIM variable of a REAL[3] type one-dimensional array of sz elements. DREAL and DREAL3 arguments are passed by reference, and DVREAL and DVREAL3 arguments are passed by value.

DREAL4(double *p, size_t sz)

DVREAL4(double *p, size_t sz)

Each of these macros passes an array of double-precision floating-point numbers of a C program as an array of double-precision real numbers (without units) of an ATL program. In the argument "p", specify the pointer that is set to a double-type array of at least sz elements. The

actual argument passed to the subroutine is an all-element DIM variable of a REAL[4] type one-dimensional array of sz elements. A DREAL4 argument is passed by reference, and a DVREAL4 argument is passed by value.

STRING(char *p, size_t sz)

This macro passes a character string. In the argument "p", specify the pointer that is set to a char array of at least sz elements. The actual argument passed to the subroutine is a STRING variable of length "sz". The argument is passed by reference. When substituting a character string for the buffer p in an ATL program, it is the application's responsibility to terminate the string with a null character. Therefore, for an ATL program, the size of the buffer area available to store a character string is "sz - 1" characters, and the last byte should be reserved for a null character.

CINTE(int32_t v)

This macro passes a signed 32-bit integer. In the argument "v", specify int32_t type data. The actual argument passed to the subroutine is a single-precision integer constant. The argument is passed by value.

CLOGI(uint32_t v)

This macro is used to pass an unsigned 32-bit integer. In the argument "v", specify uint32_t type data. The actual argument passed to the subroutine is a single-precision logical number constant. The argument is passed by value.

CREAL(double v)

This macro passes a double-precision floating-point number of a C program as a single-precision real number (without a unit) of an ATL program. In the argument "v", specify double type data. The actual argument passed to the subroutine is a real number constant without a single-precision unit. The argument is passed by value.

Notes on Passing a Floating-point Number

Because the expression format of floating-point numbers differs between C and ATL programs, the expression is converted to a different one when a floating-point number is passed between these types of programs. The floating-point expression in C programs conforms to the IEEE 754 standard. Floating-point numbers in ATL programs are expressed as explained in Chapter 9, "Nontester Statement" in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#). These floating-point expressions are not compatible with each other. If the difference in the length of the mantissa is significant, the conversion results in an error. When the absolute value of the original number is v, the result of the conversion is approximated to the value obtained by adding a sign to the maximum expressible value in the closed interval [0, v]. Here, note that the value of "v" in the original floating-point expression may already include a rounding error, such as the case when expressing and infinite decimal in binary format.

The exponent definition field for the number of floating points used in C programs is also included in ATL programs. As a result, sometimes the number of floating points used in C programs cannot be expressed by the number of floating points used in ATL programs. If the expo-

ment for the number of floating points used in a C program is greater than the maximum exponent number used in an ATL program, the value converted from the C program to the ATL program is uncertain. If the exponent for the number of floating points used in a C program is smaller than the minimum exponent number used in an ATL program, the value converted from the C program to the ATL program will be 0.0.

- ➔ For more information on the struct atreal structure that represents a floating-point number with a double-precision unit for a C program, refer to Chapter 3, "Basic System" in [ASX/U-51 Hybrid Controller Users Manual\(No. : 8409598\)](#).
- ➔ For more information, refer to [1. 1. 38 "ATL_init" on page 1-76](#) and [1. 1. 27 "AT_strtor" on page 1-48](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_BAD_PROGNAME

The specified program name is invalid.

S_hybrid_PRO_NOT_LOADED

The ATL test plan program is not loaded.

S_hybrid_LABEL_NOT_FOUND

The specified label was not found.

S_hybrid_TEST_NOT_FOUND

The specified test number was not found.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Setting the I/O Voltage by an ATL Program

C program:

```
#include "AT/hybrid.h"
#include "stdlib.h"

int test_main(void)
{
    struct atreal      in[2];
    struct atreal      out[2];

    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }
    if(AT_strtorm(in, 2, "2.2V", "0.2V") == NULL ||
       AT_strtorm(out, 2, "2.0V", "0.5V") == NULL){
        AT_printf("Error: AT_strtorm\n");
        exit(1);
    }
    if(ATL_exec(ATL_GOSUB, "VIOSET", 4,
               CINT(1), DVREAL(in, 2),
               CINT(1), DVREAL(out, 2)) != 0){
        AT_printf("Error: ATL_exec: VIOSET\n");
        exit(1);
    }
    return (0);
}
```

ATL program:

```
PRO SAMPLE

VIOSET:
    ARGUMENT(INCH, ILV(2), OUTCH, OLV(2))
    IN(INCH)    = ILV(1), ILV(2)
    OUT(OUTCH) = OLV(1), OLV(2)
    GOTO CONTINUE

END
```

1. 1.30 ATL_get_char

Function: This library function obtains character data from a system variable.

Syntax

```
#include "AT/hybrid.h"

char *ATL_get_char
(
    int      svid,      /* system variable ID */
    char     *dest,     /* where to put data */
    size_t   size       /* number of characters to put */
);
```

Description

This function obtains character data from a system variable.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If the specified system variable does not contain character data, the function fails.

In the argument "dest", specify the pointer that is set to the char type area that stores the character string or character data set to the system variables. This area must be provided by the caller.

This function reads the character string or character data assigned to the specified system variable up to "size - 1" bytes into the area indicated by the argument "dest". The null character '\0' is then appended to the resulting character string.

➔ For more information, refer to [1. 1. 41 "ATL_set_char" on page 1-84](#).

Return Value

If the function ends successfully, it returns "dest".

If the function fails, the error information is assigned to "errno" and returns NULL.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

The specified system variable contains no character data.

1. 1.31 ATL_get_errinfo

Function: Obtain the error information of ATL.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_errinfo
(
    struct atlerrinfo *dest    /* where to put results */
);
```

Description

This function obtains a latest error information which has been occurred in the ATL program executed by the ATL_exec() and stores it in the area specified in the argument "dest".

In the argument "dest", specify the pointer that is set to the atlerrinfo structure as a storage destination of the error information. The area of this structure must be provided by the caller. If "dest" is NULL, this function fails.

Following members are available for atlerrinfo structure.

```
int32_t    e_code;
char       prname[AEI_PRNAME_MAX];
int32_t    testno;
int32_t    line1;
char       srcname[AEI_SRCNAME_MAX];
int32_t    rec_no;
int32_t    rec_line;
int32_t    line2;
int32_t    imcode;
int32_t    e_addr;
int32_t    e_psw;
int32_t    e_ac[AEI_NAC];
```


Meaning of member is as follow.

Member	Meaning
e_code	Indicates an ATL error code.
prname[]	Indicates the name of a program in which an error occurred.
testno	Indicates a test number when an error occurred.
line1	Indicates the number of lines where an error occurred, counted from the beginning.
srcname[]	Indicates a source file name of a program in which an error occurred.
rec_no	Indicates the number of records in a source file where an error occurred.
rec_line	Indicates the number of lines in a source file record where an error occurred.
line2	Indicates a line number in the horizontal direction connected with & symbol when an error occurred.
imcode	Indicates an intermediate language code when an error occurred.
e_addr	Indicates an address in the system program when an error occurred.
e_psw	Indicates a value of PSW when an error occurred.
e_ac[]	A value of ACn (n is an integer of 0 to 7) when an error occurred becomes a value of nth array element.

If this function is used when an error does not occur, e_code becomes -1 and all the others become 0.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

1. 1.32 ATL_get_inte

Function: This library function obtains the value of a system variable as an integer.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_inte
(
    int      svid,      /* system variable ID */
    int32_t  *dest      /* where to put data */
);
```

Description

This function obtains the value of a system variable as an integer.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If the system variable contains character type or character data, the function fails.

In the the argument "dest", specify the pointer that is set to an int32_t type object that stores the system variable values.

The value obtained by this function is the result of the following conversion issued according to the type of value of the system variable.

- If the value of the system variable is an integer or a logical number
If a double-precision integer or a logical number is assigned to the system variable, the 32 low-order bits are obtained. Otherwise, the original value is obtained as it is.
- If the value of the system variable is a real number
The real number assigned to the system variable is converted into an integer. The decimal fraction is discarded in this conversion. If the original real number cannot be expressed in int32_t, the resulting value is unspecified.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.33 ATL_get_intem

Function: This library function obtains the values of successive system variables as integers.

Syntax

```
#include "AT/hybrid.h"

(
int ATL_get_intem
    int      svid,      /* first system variable ID */
    int32_t  *dest,     /* pointer to array */
    size_t   ndata      /* number of data to get */
);
```

Description

This function obtains the values of successive system variables as integers.

➔ For more information on the definition of successive system variables, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#).

In the argument "svid", specify the identification code of the first system variable of the successive system variables. If any of the target system variables contains character data, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "dest", specify the pointer that is set to an array of `int32_t` that has at least as many elements as specified in the argument "ndata" to store the values of the system variables. The area must be provided by the caller.

This function reads values from "ndata" successive system variables beginning from the system variable "svid" into the array "dest" in sequence from the beginning. The values are converted in the same way as `ATL_get_inte()` is.

➔ For more information, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#) and [1. 1. 32 "ATL_get_inte" on page 1-64](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.34 ATL_get_logi

Function: This library function obtains the value of a system variable as a logical number.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_logi
(
    int      svid,      /* system variable ID */
    uint32_t *dest      /* where to put data */
);
```

Description

This function obtains the value of a system variable as a logical number.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If the system variable contains character type or character data, the function fails.

In the argument "dest", specify the pointer that is set to a uint32_t type area that stores the system variable values. This area must be provided by the caller.

The value obtained by this function is the result of the following conversion issued according to the type of value of the system variable.

- If the value of the system variable is an integer or a logical number
If a double-precision integer or a logical number is assigned to the system variable, the 32 low-order bits are obtained. Otherwise, the original value is obtained as it is.
- If the value of the system variable is a real number
The real number assigned to the system variable is converted into an unsigned integer. The decimal fraction is discarded in this conversion. If the original real number cannot be expressed in uint32_t, the resulting value is unspecified. A negative real number is converted into an unsigned integer represented by the same bit pattern as that obtained by converting the negative real number into a signed integer.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.35 ATL_get_logim

Function: This library function obtains the values of successive system variables as logical numbers.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_logim
(
    int          svid,      /* first system variable ID */
    uint32_t     *dest,     /* pointer to array */
    size_t       ndata      /* number of data to get */
);
```

Description

This function obtains values of successive system variables as logical numbers.

➔ For more information on the definition of successive system variables, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#).

In the argument "svid", specify the identification code of the first system variable of the successive system variables. If any of the target system variables contains character data, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "dest", specify the pointer that is set to an array of uint32_t that has at least as many elements as specified in the argument "ndata" to store the values of the system variables. The area must be provided by the caller.

This function reads values from "ndata" successive system variables beginning from the system variable "svid" into the array "dest" in sequence from the beginning. The values are converted in the same way as ATL_get_logi() is.

➔ For more information, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#) and [1. 1. 34 "ATL_get_logi" on page 1-68](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.36 ATL_get_real

Function: This library function obtains the value of a system variable as a real number.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_real
(
    int          svid,          /* system variable ID */
    struct atreal *dest         /* where to put data */
);
```

Description

This function obtains the value of a system variable as a real number.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If the system variable contains character type or character data, the function fails.

In the argument "dest", specify the pointer that is set to the struct atreal object that stores the system variable values. This area must be provided by the caller.

The value obtained by this function is the result of the following conversion issued according to the type of value of the system variable.

- If the value of the system variable is an integer
The integer assigned to the system variable is converted into a floating-point number. Therefore, the obtained value represents the corresponding integer expressed within the range of the precision of a floating-point number. The unit is UNIT_NONE.
- If the value of the system variable is a logical number
The logical number assigned to the system variable is assumed to be an unsigned integer and converted into a floating-point number. Therefore, the obtained value represents the corresponding unsigned integer expressed within the range of the precision of a floating-point number. The unit is UNIT_NONE.
- If the value of the system variable is a real number
The expression of the floating-point number in an ATL program is converted into the corresponding floating-point number in a C program. This conversion is the same as that performed for passing arguments using ATL_exec().
➔ For more information, refer to [1. 1. 29 "ATL_exec" on page 1-52](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.37 ATL_get_realm

Function: This library function obtains the values of successive system variables as real numbers.

Syntax

```
#include "AT/hybrid.h"

int ATL_get_realm
(
    int          svid,          /* first system variable ID */
    struct atreal *dest,        /* pointer to array */
    size_t       ndata          /* number of data to get */
);
```

Description

This function obtains the values of successive system variables as real numbers.

➔ For more information on the definition of successive system variables, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#).

In the argument "svid", specify the identification code of the first system variable of the successive system variables. If any of the target system variables contains character data, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "dest", specify the pointer that is set to an array of the struct atreal structure that has at least as many elements as specified in the argument "ndata" to store the values obtained from system variables. The area must be provided by the caller.

This function reads values from "ndata" successive system variables beginning from the system variable "svid" into the array "dest" in sequence from the beginning. The values are converted in the same way as ATL_get_real() is.

➔ For more information, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#) and [1. 1. 36 "ATL_get_real" on page 1-72](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.38 ATL_init

Function: This library function loads an ATL test plan program and prepares for device measurement in an ATL-separated program.

Syntax

```
#include "AT/hybrid.h"

int ATL_init
(
    const char *   pronaame    /* PRO name */
);
```

Description

This function loads the ATL test plan program into the tester processor and sets up the test system to measure devices in the ATL-separated program.

In the argument "pronaame", specify the name of the ATL test plan program to be loaded. The program name is not an object file name. Remove the '.pro' extension from the object file name, and specify the remainder of the name in uppercase letters. If the name includes an underscore '_' replace it with a period '.'.

The specified ATL test plan program is loaded only if it is not currently loaded on the program area of the test processor. If the program is currently loaded, it is not updated. To update the program, clear the program area by executing the "U" command. When the specified test plan program is loaded, the measurement conditions specified in the socket program are read and the test system is set up so that it can measure devices under the conditions specified in the socket program.

If this function is called more than once, the previous measurement conditions and measurement results remain unchanged and the ATL test plan program is switched each time the function is called. The label name and test number, which can be specified in the ATL_exec() function, are updated to those of the specified program each time the function is called.

The function fails if it is called from an event handler specified by the AT_event() function. It also fails if a runtime or system error have occurred in the previously executed ATL_init() or ATL_exec() function.

This function cannot be executed in the ATL-mixed program. Even if only one source file that configures a program is described with the ATL-mixed method, the program is evaluated as the ATL-mixed program.

➔ For more information, refer to [1. 1. 29 "ATL_exec" on page 1-52](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_INVALID_OPERATION

The function was called from a synchronous event handler.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_BAD_PROGNAME

The specified program name is invalid.

S_hybrid_PRO_FORMAT

The specified program is not an ATL test plan program.

S_hybrid_PRO_VERSION

The specified ATL test plan program cannot be used.

S_hybrid_ATL_MIXED_PROGRAM

The function was called from an ATL-mixed program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Loading the ATL Test Plan Program PRO SAMPLE

C program:

```
#include "AT/hybrid.h"
#include "stdlib.h"

int test_main(void)
{
    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }
    if(ATL_exec(ATL_GOSUB, "CHK", 0) != 0){
        AT_printf("Error: ATL_exec\n");
        exit(1);
    }
    return (0);
}
```

ATL program:

```
PRO SAMPLE

CHK:      WRITE "HERE IS CHK!",/
          GOTO CONTINUE

END
```


1. 1.39 ATL_init2

Function: This library function sets measurement conditions before measuring devices in the ATL-mixed program.

Syntax

```
#include "AT/hybrid.h"
#include "AT/atl.h"

int ATL_init2(void);
```

Description

This function sets measurement conditions, which are described in the ATL socket program, to the test system. This function also sets conditions that are required to execute the embedded ATL subroutine in the ATL-mixed program.

This function loads measurement conditions from the ATL socket program that is specified in `#pragma atl` socket described in the called source file. If no ATL socket program is specified in the source file, the source file is evaluated as if no ATL socket program exists.

If `#pragma atl` name is described in the source file that called this function, the program name is displayed in the control box. If `#pragma atl` name is not described, a program name, which is assigned by the compiler, is displayed in the control box.

When `ATL_init2()` is called twice or more, the program name that is displayed in the control box according to the specifications in the called source file and the number of devices for parallel testing are updated each time and the measurement conditions and measurement results remain unchanged.

The function fails if it is called from a source file in which no pragma function, which is related to ATL, is specified. This function also fails if a `DEFINE` variable, which is larger than the `DEFINE` variable of ATL defined in the source file that called this function, is defined in other source file. In addition, the pragma function, which is related to ATL, cannot be described in more than 256 source files. This function fails if it is described in more than 256 source files.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno."

ERRNO

`S_hybrid_UNAVAILABLE`

The hybrid controller cannot be used.

S_hybrid_INVALID_OPERATION

The function was called from a synchronous event handler.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_ATL_NOT_FOUND

The pragma, which is related to ATL, is not specified in the source file.

S_hybrid_DEFINE_OVERFLOW

A DEFINE variable, which is defined in another source file, exceeds the DEFINE variable of ATL which is defined in the source file that called this function.

S_hybrid_ATL_SETUP

Failed to prepare for the embedded ATL subroutine execution.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

1. 1.40 ATL_proc

Function: This library function executes embedded ATL subroutines in the ATL-mixed program.

Syntax

```
#include "AT/hybrid.h"
#include "AT/atl.h"

int ATL_proc
(
    atlproc_t    procaddr,    /* Embedded ATL subroutine address */
    int          nargs,       /* number of arguments to be passed */
    ...          /* arguments to the subroutine */
);
```

Description

This function executes the specified embedded ATL subroutine in the ATL-mixed program. The `ATL_init2()` function must be executed before this function.

For the argument "procaddr", specify the embedded ATL subroutine address. The embedded ATL subroutine address is given by the C compiler as an `atlproc_t` type variable that has the same name as the subroutine specified in `#pragma atl proc`. In addition, the embedded ATL subroutine must be defined in the source file that calls this function and defined before this function is called.

In the argument "nargs", specify the number of arguments to be passed to a subroutine. A maximum of 36 arguments can be passed to the subroutine. For the variable argument after "nargs", specify nargs arguments to be passed to the subroutine. The method of specifying the arguments is the same as that for the `ATL_exec()` function.

➔ For more information, refer to ["Passing Arguments to a Subroutine" on page 1-54](#) in [1. 29 "ATL_exec" on page 1-52](#)

This function returns to the caller when all the processes in the subroutine are complete or when the nest level of the internal subroutine is 0 and the GOTO CONTINUE statement is executed.

If a runtime or system error occurs while executing the subroutine, this function returns an error. This function also returns an error if a runtime or system error occurred in the ATL program which was executed before this function is called.

This function cannot be called from the asynchronous event handler.

➔ For more information, refer to [1. 1. 39 "ATL_init2" on page 1-79](#) and [1. 1. 29 "ATL_exec" on page 1-52](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno."

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_INVALID_OPERATION

The function was called from a synchronous event handler.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_ATL_NOT_INITIALIZED

The ATL_init2() function was not executed.

S_hybrid_ATL_NOT_FOUND

The function was called from a source file in which no pragma function, which is related to ATL, is specified.

S_hybrid_ILLEGAL_ATL_ADDRESS

The specified embedded ATL subroutine address is invalid.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of setting the I/O voltage

```
#include "AT/hybrid.h"
#include "AT/at1.h"
#include "stdlib.h"

#pragma atl name "SAMPLE"
#pragma atl socket "SSOC"

#pragma atl proc VIOSET(INCH,ILV(2),OUTCH,OLV(2)) "{
    IN(INCH)    = ILV(1), ILV(2)
    OUT(OUTCH) = OLV(1), OLV(2)
}"

int test_main(void)
{
    struct atreal    in[2];
    struct atreal    out[2];

    if(ATL_init2() != 0){
        AT_perror("ATL_init2()");
        exit(1);
    }
    if(AT_strtorm(in, 2, "2.2V", "0.2V") == NULL ||
       AT_strtorm(out, 2, "2.0V", "0.5V") == NULL){
        AT_perror("AT_strtorm()");
        exit(1);
    }
    if(ATL_proc(VIOSET, 4, CINTE(1), DVREAL(in, 2),
               CINTE(1), DVREAL(out, 2)) != 0){
        AT_perror("ATL_proc():VIOSET");
        exit(1);
    }
    return(0);
}
```

1. 1.41 ATL_set_char

Function: This library function sets character data to a system variable.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_char
(
    int          svid,          /* system variable ID */
    const char   *src          /* character data to set */
);
```

Description

This function sets character data to the specified system variable.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If the specified system variable does not accept the character data, the function fails.

In the argument "src", specify the pointer that is set to the character string to be set.

This function sets the character string specified in the argument "src", including the terminating null character '\0,' in the specified system variable up to the size of the system variable.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

The specified system variable does not accept character data.

1. 1.42 ATL_set_inte

Function: This library function sets an integer to a system variable.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_inte
(
    int      svid,      /* system variable ID */
    int32_t  src        /* data to set */
);
```

Description

This function sets an integer to the specified system variable.

In the argument "svid", specify the identification code of the target system variable. System variable identification codes are defined as macros as shown below. If a character-type system variable is specified, the function fails.

Identification Code	Targeted System Variable
SV_HB(int n)	HB0001 - HB0999 Variables common to all stations. HB0001 to HB0010 are unspecified 5W-type variables, and HB0011 to HB0999 are unspecified 3W-type variables. Specify an integer from 1 to 999 in the macro argument "n" to indicate the target system variable number.
SV_HBCOMM	HBCOMM 256-character character-type variable common to all stations
SV_DEVICE	DEVICE 8-character character-type variable for each station
SV_LOT	LOT 24-character character-type variable for each station
SV_SUBLOT	SUBLOT Integer 2W type variable for each station
SV_DUTIF	DUTIF 12-character character-type variable for each station
SV_OPERAT	OPERAT 24-character character-type variable for each station
SV_COMMEN	COMMEN 40-character character-type variable for each station
SV_WAFER	WAFER 24-character character-type variable for each station
SV_SWITCH	SWITCH Unspecified 3W-type variable for each station
SV_GLOB(int n)	GLOB1 - GLOB50 Variables for each station. GLOB1 to GLOB9 are unspecified 5W-type variables, and GLOB10 to GLOB50 are unspecified 3W-type variables. Specify an integer from 1 to 50 in the macro argument "n" to indicate the target system variable number.

Specify the integer value to be assigned to the variable in the argument "src". The specified value is subjected to sign extension depending on the size of the target system variable.

➔ For more information, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#) and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

Example

(1) Example of Setting the Value 256 to the System Variable HB0123

```
#include "AT/hybrid.h"
#include "stdlib.h"

int test_main(void)
{
    if(ATL_set_inte(SV_HB(123), 256) != 0){
        AT_printf("ATL_set_inte()\n");
        exit(1);
    }

    return (0);
}
```

1. 1.43 ATL_set_intem

Function: This library function sets integers to successive system variables.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_intem
(
    int          svid,      /* first system variable ID */
    int32_t      *src,      /* pointer to array */
    size_t       ndata      /* number of data to set */
);
```

Description

This function sets integers to the successive system variables. Successive system variables are HBN or GLOBN system variables which have successive numbers. The successive system variable with the smallest number is the first system variable of the successive system variables.

In the argument "svid", specify the identification code of the first system variable of the successive system variables.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If a character-type system variable is specified, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "src", specify the pointer that is set to an array of int32_t that has at least as many elements as specified in the argument "ndata".

This function sets values of the elements of array "src" in sequence from the beginning of the array into "ndata" system variables beginning from system variable "svid". These values are assigned in the same way as ATL_set_inte() is.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

Example

- (1) Example of Setting 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9 in Sequence to the System Variables HB0008 to HB0017

```
#include "AT/hybrid.h"
#include "stdlib.h"

int test_main(void)
{
    int    v[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    if(ATL_set_intem(SV_HB(8), v, 10) != 0){
        AT_printf("ATL_set_intem()\n");
        exit(1);
    }

    return (0);
}
```

1. 1.44 ATL_set_logi

Function: This library function sets a logical number to a system variable.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_logi
(
    int          svid,      /* system variable ID */
    uint32_t     src        /* data to set */
);
```

Description

This function sets a logical number to the specified system variable.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If a character-type system variable is specified, the function fails.

In the argument "src", specify the logical value to be assigned to the variable. If the target system variable is the 5W type, the specified value is set for the 32 low-order bits, and 0 is set for the 32 high-order bits.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.45 ATL_set_logim

Function: This library function sets logical numbers to successive system variables.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_logim
(
    int          svid,      /* first system variable ID */
    uint32_t     *src,      /* pointer to array */
    size_t       ndata      /* number of data to set */
);
```

Description

This function sets logical numbers to successive system variables.

➔ For more information on the definition of successive system variables, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#).

In the argument "svid", specify the identification code of the first system variable of the successive system variables. If a character-type system variable is specified, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "src", specify the pointer that is set to an array of uint32_t that has at least as many elements as specified in the argument "ndata".

This function sets values of the elements of array "src" in sequence from the beginning of the array into "ndata" system variables beginning from system variable "svid". These values are assigned in the same way as ATL_set_logi() is.

➔ For more information, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#) and [1. 1. 44 "ATL_set_logi" on page 1-90](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.46 ATL_set_real

Function: This library function sets a real number to a system variable.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_real
(
    int          svid,          /* system variable ID */
    struct atreal *src          /* data to set */
);
```

Description

This function sets a real number to the specified system variable.

In the argument "svid", specify the identification code of the target system variable.

➔ For more information on the identification code of system variables, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#).

If a character-type system variable is specified, the function fails.

In the argument "src", specify the pointer that is set to the struct atreal structure that contains the real number to be set.

The value set by this function is the result of the following conversion issued according to the type of the specified system variable.

- If the system variable is an unspecified type
The specified value is converted from the floating-point expression of a C program into that of an ATL program and then set. This conversion is the same as that for passing arguments between the ATL_exec() function and an ATL program.
➔ For more information, refer to [1. 1. 29 "ATL_exec" on page 1-52](#).
If this conversion fails, the function fails.
- If the system variable is an integer
The unit is ignored, and the specified value is truncated before it is set. However, if the value is not within the range that can be expressed with the target system variable, the set value is unspecified.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

ERANGE

Conversion of the floating-point expression failed.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

1. 1.47 ATL_set_realm

Function: This library function sets real numbers to successive system variables.

Syntax

```
#include "AT/hybrid.h"

int ATL_set_realm
(
    int          svid,          /* first system variable ID */
    struct atreal *src,         /* pointer to array */
    size_t       ndata         /* number of data to set */
);
```

Description

This function sets logical numbers to successive system variables.

→ For more information on the definition of successive system variables, refer to [1. 1. 43 "ATL_set_intem" on page 1-88](#).

In the argument "svid", specify the identification code of the first system variable of the successive system variables. If a character-type system variable is specified, the function fails. The function also fails if there are no "ndata" successive system variables beginning with the system variable "svid".

In the argument "src", specify the pointer that is set to an array of the struct atreal structure that has at least as many elements as specified in the argument "ndata".

This function sets values of the elements of array "src" in sequence from the beginning of the array into "ndata" system variables beginning from system variable "svid". These values are assigned in the same way as ATL_set_real() is

→ For more information, refer to [1. 1. 42 "ATL_set_inte" on page 1-85](#) and [1. 1. 46 "ATL_set_real" on page 1-93](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

ERANGE

Conversion of the floating-point expression failed.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_INVALID_OPERATION

A character-type system variable is specified.

2. *Tester Access Library*

This chapter describes the tester access library.

2. 1 ATL Statement Correspondence Table

ATL-51 Test Plan Program Reference Manual	ATL Statement	Function Name
Chapter 2. Measurement Results Processing	READ RESULT	AT_read_result()
	READ FAIL	AT_read_fail() AT_read_fail_test_cnt()
	READ CNT	AT_read_cnt()
	READ CTCNT	AT_read_ctcnt()
	READ WFCNT	AT_read_wfcnt()
	READ WFCTCNT	AT_read_wfctcnt()
	SET/RESET RESULT	AT_set_result() AT_reset_result()
	SET/RESET FAIL	AT_set_fail() AT_reset_fail()
	SET/RESET CNT	AT_set_cnt() AT_reset_cnt()
	SET/RESET WFCNT	AT_set_wfcnt() AT_reset_wfcnt()
	SET SORT	AT_set_sort()
	READ SORT	AT_read_sort()
	SET/RESET STN	AT_set_stn() AT_reset_stn()
Chapter 5. DC Parametric Tests	READ DCT PDATA	AT_read_dct_pdata() AT_read_dct_pdatam()
	READ DCT FAIL	AT_read_dct_fail() AT_read_dct_failm()
	READ DCT HFAIL	AT_read_dct_hfail() AT_read_dct_hfailm()
	READ DCT LFAIL	AT_read_dct_lfail() AT_read_dct_lfailm()
	READ DCT VS	AT_read_dct_vs()
	READ DCT DATA	AT_read_dct_data()
	READ TPIN	AT_read_tpin()
	READ FPIN	AT_read_fpin()

2. 1 ATL Statement Correspondence Table

ATL-51 Test Plan Program Reference Manual	ATL Statement	Function Name
Chapter 7. System Interface Control Statements	SET/RESET CATEGORY	AT_set_category() AT_reset_category()
	READ CATEGORY	AT_read_category()
	SET/RESET FKMODE	AT_set_fkmode() AT_reset_fkmode()
	READ FK	AT_read_fk()
	READ STN	AT_read_stn()
	READ TEST	AT_read_test()
	READ STNCNT	AT_read_stncnt()
	READ DUTCNT	AT_read_dutcnt()
	SEND EXT	AT_send_ext()
	READ EXT	AT_read_ext()
	SET DECISION	AT_set_decision()
	SET REJECTION	AT_set_rejection() AT_set_rejectionm()
	SET/RESET EXCLUSION	AT_set_exclusion() AT_set_exclusionm() AT_reset_exclusion() AT_reset_exclusionm()
	SET/RESET JDUT	AT_set_jdut() AT_set_jdutm() AT_reset_jdut()
	SET RDUT	AT_set_rdut()
	READ DDUT	AT_read_ddut()
	READ SDUT	AT_read_sdut()
	READ MDUT	AT_read_mdut()
	READ EDUT	AT_read_edut()
	READ CDUT	AT_read_cdut()
	READ GFDUT	AT_read_gfdut()
	READ PDUT	AT_read_pdut() AT_read_pdut_t()

2. 1 ATL Statement Correspondence Table

ATL-51 Test Plan Program Reference Manual	ATL Statement	Function Name
Chapter 7. System Interface Control Statements	READ FDUT	AT_read_fdut() AT_read_fdut_t()
	SET SDUT	AT_set_sdut() AT_set_sdutm()
	SET/RESET EXCLUSION2	AT_set_exclusion2() AT_set_exclusionm2() AT_reset_exclusion2() AT_reset_exclusionm2()
Chapter 8. ALPG Operation Setting and Functional Tests	READ FUNC	AT_read_func() AT_read_funcm() AT_read_func_block()
	READ CUMFUNC	AT_read_cumfunc() AT_read_cumfuncm() AT_read_cumfunc_block()
Chapter 9. Non-tester Statements	READ ERCODE	AT_read_ercode()
Chapter 10. Failure Analysis Memory	AFM MEMORY	AT_write_afm()
	READ AFM CONFIG COUNT	AT_read_afm_confcnt()
	READ AFM CONFIG DATA	AT_read_afm_confdat()
	READ AFM DUT SELECT	AT_read_afm_dutselect()
	READ AFM [MEMORY]	AT_read_afm_memory()
	READ AFM [FAIL ADDRESS]	AT_read_afm_failaddr()
	READ AFM [FAIL COUNT]	AT_read_afm_failcnt()
	READ AFM [Failure Cycle Count]	AT_read_afm_mcnt()
	READ CFM [MEMORY]	AT_read_cfm_memory()
	READ CFM [FAIL COUNT]	AT_read_cfm_failcnt()
Chapter 11. DBM/CBM	READ DBM	AT_read_dbm()
	READ CBM	AT_read_cbm()

2. 2 Utility - Function Table

Utility		Function Name
Parallel DFM Utility	PDFMSET	AT_set_pdfm()
	PDFMREAD	AT_read_pdfm()

2. 3 Notes on Multi-dimensional Arrays of C and ATL

Note that the relationship between the location of an array subscript and the dimensions in a multi-dimensional array are reversed between C and ATL. If an array is accessed in sequence based on memory locations, the rightmost subscript changes first in C but the leftmost subscript changes first in ATL.

The relationships of array subscripts between C and ATL are shown in the example given below for a two-dimensional array. This two-dimensional array has two 32-bit integer vectors, each of which has five elements. Values 0 to 9 are assumed to be assigned in memory address order. The data referenced in individual array subscripts is shown below.

C-program int32_t a[2][5];	ATL Program INTE[2] A(5,2)	Referenced Data
a[0][0]	A(1.1)	0
a[0][1]	A(2.1)	1
a[0][2]	A(3.1)	2
a[0][3]	A(4.1)	3
a[0][4]	A(5.1)	4
a[1][0]	A(1.2)	5
a[1][1]	A(2.2)	6
a[1][2]	A(3.2)	7
a[1][3]	A(4.2)	8
a[1][4]	A(5.2)	9

2. 4 Reference

2. 4. 1 AT_read_result

Function: This library function obtains various test results and states.

Syntax

```
#include "AT/hybrid.h"

int AT_read_result
(
    int      request,          /* kind of test result or condition */
    int      nelem,           /* number of data to read */
    uint32_t *vp              /* address to destination variable */
);
```

Description

This function reads the results of different kinds of tests and status information. In the argument "request", specify the type of data to be read. Use one of followings to specify.

Value Type	Description
1	Reads the cumulative results of tests previously performed using MEAS statements.
2	Reads the results of the test performed using the previously defined MEAS statement.
3	Reads the operation stop status of the pattern generator started using the previously defined MEAS MPAT or START MPAT statement.
6	Reads the cause of the last interruption from the tester.
7	Reads the type number indicating the type of the test previously performed.

In the argument "vp", set the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem". If a value other than 1, 2, 3, 6, or 7 is specified in the argument "request", an error occurs.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ RESULT statement.

➔ For more information on using this function, refer to the description of the READ RESULT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the DUT1 to DUT4 Test Results Performed by Using the Previously Defined MEAS Statement

```
int foo(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp[4];        /* Saving destination variables
                           Saves DUT1 test results in vp[0].
                           Saves DUT2 test results in vp[1].
                           Saves DUT3 test results in vp[2].
                           Saves DUT4 test results in vp[3].
                           */

    ret=AT_read_result(2,4,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 2 AT_read_fail

Function: This library function obtains the results of each test.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fail
(
    int      testno,    /* test number */
    int      nelem,     /* number of data to read */
    int32_t  *vp        /* address to destination variable */
);
```

Description

This function reads the results for each test. In the argument "testno", specify the number of the test whose results are to be read.0 Specify a value within 0 to 999 bits. In the argument "vp", specify a pointer that is set to an int32_t-type array which has at least as many elements as specified in the argument"nelem". The read data format is the same as that of data read using the READ FAIL statement.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ FAIL statement.

➔ For more information on using this function, refer to the description of the READ FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Results of a Four Device Parallel Test

```
int foo(void)
{
    int      ret;          /* Return value variable */
    int32_t vp[4];         /* Saving destination variables
                           Saves the results of DUT1 test 1 in vp[0].
                           Saves the results of DUT2 test 1 in vp[1].
                           Saves the results of DUT3 test 1 in vp[2].
                           Saves the results of DUT4 test 1 in vp[3].
                           */

    ret=AT_read_fail(1,4,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 3 AT_read_fail_test_cnt

Function: This library function obtains the number of tests that failed.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fail_test_cnt
(
    int      nelem,    /* number of data to read */
    int32_t  *vp       /* address to destination variable */
);
```

Description

This function reads data on the number of past tests that failed. In the argument "vp", specify a pointer that is set to an int32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

The argument meanings, the read data format, and notes are the same as those of the READ FAIL statement.

➔ For more information on using this function, refer to the description of the READ FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Data on the Number of Tests That Failed during a Four Device Parallel Test

```
int foo(void)
{
    int      ret;          /* Return value variable */
    int32_t vp[4];         /* Saving destination variables
                           Saves the data on the number of tests that failed
                           during DUT1 measurement in vp[0].
                           Saves the data on the number of tests that failed
                           during DUT2 measurement in vp[1].
                           Saves the data on the number of tests that failed
                           during DUT3 measurement in vp[2].
                           Saves the data on the number of tests that failed
                           during DUT4 measurement in vp[3].
                           */

    ret=AT_read_fail_test_cnt(4,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 4 AT_read_cnt

Function: This library function obtains the contents of different types of counters.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cnt
(
    int      cntno,      /* counter number */
    int      nelem,      /* number of data to read */
    uint32_t *vp         /* address to destination variable */
);
```

Description

This function reads values from different types of counters. The total counter, fail counter, counters for category 1 to 200, sort total counter, and sort 1 to 16 counters are provided for each DUT. When this function is executed, the sum total of the counters of all DUTs from DUT No. 1 is read. A sample number is assigned to each station, and the value of the station currently being used is read. Fail counters for tests 0 to 999 are provided for each station, and the sum total of all station counters is read. Specify a counter number in the argument "cntno".

➔ For more information on the value of "cntno", refer to the description of the READ CNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

In the argument "vp", set the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem". The read data format is the same as that of data read using the READ CNT statement.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

If the area for the specified fail counter has not been reserved, the function returns 1.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Data from Failed Counters for Tests 0 to 99

```
int foo(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp[100];      /* Saving destination variables
                           Saves fail counter data of test 0 in vp[0].
                           Saves fail counter data of test 1 in vp[1].
                           Saves fail counter data of test 2 in vp[2].
                           :
                           :
                           Saves fail counter data of test 99 in vp[99].
                           */

    ret=AT_read_cnt(100,100,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }
    return(0);
}
```


2. 4. 5 AT_read_ctcnt

Function: This library function reads values from different types of counters for each handler contactor.

Syntax

```
#include "AT/hybrid.h"

int AT_read_ctcnt
(
    int      ctcntno,    /* counter number */
    int      nelem,      /* number of data to read */
    uint32_t *vp         /* address to destination variable */
);
```

Description

This function reads values from different types of counters for each handler contactor. Specify a counter number in the argument "ctcntno".

➔ For more information on the value of ctcntno, refer to the description of the READ CTCNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

In the argument "vp", set the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem". The read data format is the same as that of data read using the READ CTCNT statement.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Values from the Total Fail Counter and Fail Counter for Each DUT As the Results of a Four Device Parallel Test

```
int fool(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp1[8];       /* Saving destination variables
                           Saves the contactor total counter value for DUT1 in
                           vp1[0].
                           Saves the contactor total counter value for DUT2 in
                           vp1[1].
                           Saves the contactor total counter value for DUT3 in
                           vp1[2].
                           Saves the contactor total counter value for DUT4 in
                           vp1[3].
                           Saves the contactor fail counter value for DUT1 in
                           vp1[4].
                           Saves the contactor fail counter value for DUT2 in
                           vp1[5].
                           Saves the contactor fail counter value for DUT3 in
                           vp1[6].
                           Saves the contactor fail counter value for DUT4 in
                           vp1[7].
                           */

    ret=AT_read_ctcnt(0,8,vp1);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

(2) Example of Reading Values from the Total Counter, Fail Counter, and Category Counter As the Results of a Single Device Test

```
int foo2(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp2[4];       /* Saving destination variables
                           Saves the contactor total counter value in vp2[0].
                           Saves the contactor fail counter value in vp2[1].
                           Saves the contactor category 1 counter value in
                           vp2[2].
                           Saves the contactor category 2 counter value in
                           vp2[2].
                           */

    ret=AT_read_ctcnt(0,4,vp2);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 6 AT_read_wfcnt

Function: This library function obtains values from different types of counters as the results of wafer testing.

Syntax

```
#include "AT/hybrid.h"

int AT_read_wfcnt
(
    int      wfcntno,    /* counter number */
    int      nelem,      /* number of data to read */
    uint32_t *vp         /* address to destination variable */
);
```

Description

This function reads the sum total of different types of counters in a wafer as the results of wafer testing. (Counter values totaled for each station can be read.) Use the function at the end of wafer testing. Specify a counter number in the argument "wfcntno".

➔ For more information on the value of "wfcntno", refer to the description of the READ WFCNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

In the argument "vp", set the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem". The read data format is the same as that of data read using the READ CNT statement.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Values from the Total Counter, Fail Counter, and Category Counter As the Results of a Multiple Device Parallel Test on a Single Station

```
int foo(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp[3];        /* Saving destination variables
                           Saves the wafer total counter value in vp[0].
                           Saves the wafer fail counter value in vp[1].
                           Saves the wafer category 1 counter value in vp[2].
                           */

    ret=AT_read_wfcnt(0.3,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

(2) Example of Reading Values from the Total Counter and Fail Counter As the Results of a Multiple Device Parallel Test on Two Stations

```
int foo(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp[4];        /* Saving destination variables
                           Saves the wafer total counter value (station 1) in
                           vp[0].
                           Saves the wafer fail counter value (station 1) in
                           vp[1].
                           Saves the wafer total counter value (station 2) in
                           vp[2].
                           Saves the wafer fail counter value (station 2) in
                           vp[3].
                           */

    ret=AT_read_wfcnt(0,4,vp);

    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 7 AT_read_wfctcnt

Function: This library function obtains values from different types of counters for each wafer contactor.

Syntax

```
#include "AT/hybrid.h"

int AT_read_wfctcnt
(
    int      wfctcntno, /* counter number */
    int      nelem,     /* number of data to read */
    uint32_t *vp        /* address to destination variable */
);
```

Description

This function reads different counter values for each wafer contactor as the results of wafer testing. Specify a counter number in the argument "wfctcntno".

➔ For more information on the value of "wfctcntno", refer to the description of the READ WFCTCNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

In the argument "vp", set the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem". The read data format is the same as that of data read using the READ WFCTCNT statement.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Values from the Total Fail Counter and Fail Counter for Each DUT As the Results of a Four Device Parallel Test

```
int fool(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp1[8];       /* Saving destination variables
                           Saves the contactor total counter value for DUT1 in
                           vp1[0].
                           Saves the contactor total counter value for DUT2 in
                           vp1[1].
                           Saves the contactor total counter value for DUT3 in
                           vp1[2].
                           Saves the contactor total counter value for DUT4 in
                           vp1[3].
                           Saves the contactor fail counter value for DUT1 in
                           vp1[4].
                           Saves the contactor fail counter value for DUT2 in
                           vp1[5].
                           Saves the contactor fail counter value for DUT3 in
                           vp1[6].
                           Saves the contactor fail counter value for DUT4 in
                           vp1[7].
                           */

    ret=AT_read_wfctcnt(0,8,vp1);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```


(2) Example of Reading Values from the Total Counter, Fail Counter, and Category Counter As the Results of a Single Device Test

```
int foo2(void)
{
    int      ret;          /* Return value variable */
    uint32_t vp2[4];       /* Saving destination variables
                           Saves the contactor total counter value in vp2[0].
                           Saves the contactor fail counter value in vp2[1].
                           Saves the contactor category 1 counter value in
                           vp2[2].
                           Saves the contactor category 2 counter value in
                           vp2[2].
                           */

    ret=AT_read_ctcnt(0,4,vp2);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4. 8 AT_set_result

Function: This library function sets the past to current test results to Fail.

Syntax

```
#include "AT/hybrid.h"

int AT_set_result
(
    int    state        /* test kind */
);
```

Description

This function sets the past to current test results to Fail. In the argument "state", specify the type of the tests to be set to Fail. Use one of followings to specify.

Value Type	Description
0	Ignored.
1	Sets the results of DC parametric tests to Fail.
2	Sets the results of functional tests to Fail.
3	Sets the results of both DC parametric tests and functional tests to Fail.

If the value specified in the argument "state" is not in a range of 0 to 3, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4. 9 AT_reset_result

Function: This library function sets past to current test results to Pass.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_result
(
    int    state        /* test kind */
);
```

Description

This function sets the past to current test results to Pass. In the argument "state", specify the type of the tests to be set to Pass. Use one of followings to specify.

Value Type	Description
0	Ignored.
1	Sets the results of DC parametric tests to Pass.
2	Sets the results of functional tests to Pass.
3	Sets the results of both DC parametric tests and functional tests to Pass.

If the value specified in the argument "state" is not in a range of 0 to 3, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.10 AT_set_fail

Function: This library function sets the test results to Fail.

Syntax

```
#include "AT/hybrid.h"

int AT_set_fail
(
    int      testno    /* test number */
);
```

Description

This function sets the test results of the specified test number to Fail. In the argument "testno", specify the test number to be set.0 Specify a value within 0 to 999 bits.

Note:

The argument meanings, the read data format, and notes are the same as those of the SET FAIL statement.

➔ For more information on using this function, refer to the description of the SET FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.11 AT_reset_fail

Function: This library function sets the test results to Pass.

Syntax

```
#include "AT/hybrid.h"

int AT_set_fail
(
    int      testno    /* test number */
);
```

Description

This function sets the test results of the specified test number to Pass. In the argument "testno", specify the test number to be set.0 Specify a value within 0 to 999 bits.

Note:

The argument meanings, the read data format, and notes are the same as those of the RESET FAIL statement.

➔ For more informatio on using this function, refer to the description of the RESET FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.12 AT_set_cnt

Function: This library function sets the contents of different types of counters.

Syntax

```
#include "AT/hybrid.h"

int AT_set_cnt
(
    int      cntno1,    /* counter number */
    uint32_t data      /* data for set    */
);
```

Description

This function defines a value in the specified total, fail, category, sample number, or test fail counter. Specify a counter number in the argument "cntno1".

➔ For more information on the value of "cntno1", refer to the description of the SET CNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

In the argument "data", specify the value to be set to the counter.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

If the area for the specified fail counter has not been reserved, the function returns 1.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Decrementing the Value in the Fail Counter by One

```
int foo(void)
{
    int      ret;          /* Return value variable */
    uint32_t data;         /* Saving destination variable */

    ret=AT_read_cnt(1,1,data);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    data=data-1;
    ret=AT_set_cnt(1,data);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4.13 AT_reset_cnt

Function: This library function initializes the contents of different types of counters.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_cnt
(
    int      cntno2    /* counter number */
);
```

Description

This function initializes the total counter, fail counter, category counter, sample number counter, and test fail counter. Specify a counter number in the argument "cntno2". Specify a value from 0 to 1316 or -1. To specify all counters, specify -1.

➔ For the relationship between the counter numbers 0 to 1316 and the counters, refer to the description of the RESET CNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

If the area for the specified fail counter has not been reserved, the function returns 1.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) When Clearing All Counters

```
int foo(void)
{
    int      ret;          /* Return value variable */

    ret=AT_reset_cnt(-1);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }
    return(0);
}
```

2. 4.14 AT_set_wfcnt

Function: This library function sets values to different types of wafer counters.

Syntax

```
#include "AT/hybrid.h"

int AT_set_wfcnt
(
    int      wfcntno1,    /* counter number */
    uint32_t data         /* data for set */
);
```

Description

This function sets values to different types of wafer counters, including the total, fail, and category counters. Specify a counter number in the argument "wfcntno1". Specify a value within 0 to 1300 bits.

➔ For the relationship between the counter numbers and the counters, refer to the description of the SET WFCNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

In the argument "data", specify the value to be set to the counter.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.15 AT_reset_wfcnt

Function: This library function initializes different types of wafer counters.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_wfcnt
(
    int      wfcntno2      /* counter number */
);
```

Description

This function initializes different wafer counters, including the total, fail, and category counters. Specify a counter number in the argument "wfcntno2". Specify a value from 0 to 1300 or -1. To specify all counter numbers, specify -1.

➔ For the relationship between the counter numbers 0 to 1300 and the counters, refer to the description of the RESET WFCNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.16 AT_set_sort

Function: This library function sets a sort number.

Syntax

```
#include "AT/hybrid.h"

int AT_set_sort
(
    int    sortno        /* sort number */
);
```

Description

This function specifies the sort number used for the handler for DUT sorting.

Specify a sort number in the argument "sortno". Specify one of the values from 0 to 16. If 0 is specified, the function does nothing. If the specified value is outside of the range of 0 to 16, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.17 AT_read_sort

Function: This library function obtains a sort number.

Syntax

```
#include "AT/hybrid.h"

int AT_read_sort
(
    int      nelem,      /* number of data to read */
    int32_t  *vp         /* address to destination variable */
);
```

Description

This function reads the sort numbers that have already been defined for each DUT.

In the argument "vp", specify a pointer that is set to an int32_t-type array which has at least as many elements as specified in the argument "nelem".

If the number of the DUT where reading starts is greater than the maximum number of the target DUTs, an error occurs.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ SORT statement.

➔ For more information on using this function, refer to the description of the READ RESULT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Sort Numbers of DUT1 to DUT5

```
int foo(void)
{
    int      ret;          /* Return value variable */
    int32_t vp[5];         /* Saving destination variables
                           Saves the DUT1 sort number in vp[0].
                           Saves the DUT2 sort number in vp[1].
                           Saves the DUT3 sort number in vp[2].
                           Saves the DUT4 sort number in vp[3].
                           Saves the DUT5 sort number in vp[4].
                           */

    ret=AT_read_sort(5,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}
```

2. 4.18 AT_set_stn

Function: This library function sets a station number.

Syntax

```
#include "AT/hybrid.h"

int AT_set_stn
(
    int    stnno/* station number */
);
```

Description

This function specifies the station number used to read or write system variables. Specify a station number in the argument "stnno". If the value specified in the argument "stnno" is outside of a range of 1 to 4, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.19 AT_reset_stn

Function: This library function resets a station number.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_stn(void);
```

Description

This function initializes the station numbers used to read or write system variables to those used first when the test plan program started.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.20 AT_read_dct_pdata

Function: This library function reads the results of DC parametric tests on a specific pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_pdata
(
    uint16_t    pinno,      /* pin data */
    int         nelem,      /* number of data to read */
    struct atreal *data     /* address to destination variable */
);
```

Description

This function reads the results of DC parametric tests on a specific pin.

Specify the type and number of the target pin for the tests in the argument "pinno". Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to an array of the struct atreal structure where the array has at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT PDATA statement.

➔ For more information on using this function, refer to the description of the READ DCT PDATA statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Data Obtained by a Four Device Parallel Test Using I/O Pin 33 (P33)

```
int foo(void)
{
    struct atreal data[4]; /* Saving destination variables */

    if (AT_read_dct_pdata(AT_P(33),4,data) !=0) {
        AT_printf("ERROR:AT_read_dct_pdata\n");
        exit(1);
    }
    return(0);
}
```

Measurement data is saved in individual "data" elements as shown below.

DUT1 measurement data in data[0]

DUT2 measurement data in data[1]

DUT3 measurement data in data[2]

DUT4 measurement data in data[3]

2. 4.21 AT_read_dct_pdatam

Function: This library function reads the results of DC parametric tests on multiple pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_pdatam
(
    int          npin,          /* number of pin data */
    uint16_t     pinno,        /* pin data */
    int          nelem,        /* number of data to read */
    struct atreal *data        /* address to destination variable */
);
```

Description

This function reads the results of DC parametric tests on multiple pins.

In the argument "npin", specify the number of pins whose data is read.

In the argument "pinno", specify the pointer that is set to an array, which has at least as many elements as specified in "npin", by using the pointer that stores the type and numbers of the target pins. Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to an array of the struct atreal structure where the array has at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT PDATA statement.

➔ For more information on using this function, refer to the description of the READ DCT PDATA statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Data Obtained by a Four Device Parallel Test Using I/O Pins 33 to 35 (P33-35)

```
int foo(void)
{
    uint16_t pindata[3];    /* Specifies the type and numbers of pins */
    struct atreal data[4][3]; /* Saving destination variables */

    pindata[0]=AT_P(33);
    pindata[1]=AT_P(34);
    pindata[2]=AT_P(35);

    if (AT_read_dct_pdatam(3,pindata,4,data)!=0){
        AT_printf("ERROR:AT_read_dct_pdatam\n");
        exit(1);
    }
    return(0);
}
```

Measurement data is saved in individual "data" elements as shown below.

```
DUT1 P33 measurement data in data[0][0]
DUT1 P34 measurement data in data[0][1]
DUT1 P35 measurement data in data[0][2]
:
DUT4 P33 measurement data in data[3][0]
DUT4 P34 measurement data in data[3][1]
DUT4 P35 measurement data in data[3][2]
```

2. 4.22 AT_read_dct_fail

Function: This library function reads the failure results of DC parametric tests on a specific pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_fail
(
    uint16_t  pinno, /* pin data */
    int       nelem, /* number of data to read */
    uint32_t  *data  /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the upper or lower limit specified in the LIMIT statement. It can read such data of a specific pin. This evaluation value is the result of an OR operation between the upper-limit failure count and lower-limit failure count. Specify the type and number of the target pin for the tests in the argument "pinno". Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT FAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Failure Results of a Four Device Parallel Test Using I/O Pin 33 (P33)

```
int foo(void)
{
    uint32_t data[4];          /* Saving destination variables */

    if (AT_read_dct_fail(AT_P(33),4,data)!=0){
        AT_printf("ERROR:AT_read_dct_fail\n");
        exit(1);
    }
    return(0);
}
```

If DUT1, DUT2, and DUT4 fail, measurement data is saved in individual data elements as shown below.

```
DUT1 measurement result 1 (FAIL) in data[0]
DUT2 measurement result 1 (FAIL) in data[1]
DUT3 measurement result 0 (PASS) in data[2]
DUT4 measurement result 1 (FAIL) in data[3]
```

2. 4.23 AT_read_dct_failm

Function: This library function reads the failure results of DC parametric tests on multiple pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_failm
(
    int      npin,      /* number of pin data */
    uint16_t pinno,     /* pin data */
    int      nelem,     /* number of data to read */
    uint32_t *data      /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the upper or lower limit specified in the LIMIT statement. It can read such data of multiple pins. This evaluation value is the result of an OR operation between the upper-limit failure count and lower-limit failure count. In the argument "pinno", specify the pointer that is set to an array, which has at least as many elements as specified in "npin", by using the pointer that stores the type and numbers of the target pins. Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT FAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT FAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

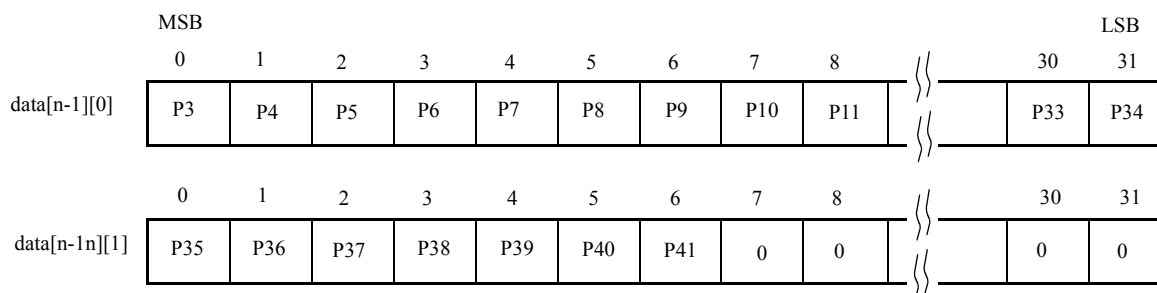
(1) Example of Reading the Failure Results of a Four Device Parallel Test Using I/O Pins 3 to 41 (P3-41)

```
int foo(void)
{
    uint16_t pindata[39];    /* Specifies the type and numbers of pins */
    uint32_t data[4][2];     /* Saving destination variables */

    for(i=3;i<=41;i++) pindata[i-3]=AT_P(i);
    /* Specifies a value from P3 to P41 */

    if (AT_read_dct_failm(39,pindata,8,data)!=0){
        AT_printf("ERROR:AT_read_dct_failm\n");
        exit(1);
    }
    return(0);
}
```


The data storage format is shown below. The bit associated with the failed pin is turned on. "n" represents a DUT number.



Failure information is saved as shown below.

Pin No.	DUT No.			
	DUT1	DUT2	DUT3	DUT4
P1 to 32	data[0][0]	data[1][0]	data[2][0]	data[3][0]
P33 to 40	data[0][1]	data[1][1]	data[2][1]	data[3][1]

2. 4.24 AT_read_dct_hfail

Function: This library function reads the upper-limit failure results of DC parametric tests on a specific pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_hfail
(
    uint16_t  pinno, /* pin data */
    int       nelem, /* number of data to read */
    uint32_t  *data  /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the upper limit specified in the LIMIT statement. It can read such data on a specific pin.

Specify the type and number of the target pin for the tests in the argument "pinno". Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT HFAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT HFAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Upper-limit Failure Results of a Four Device Parallel Test Using I/O Pin 33 (P33)

```
int foo(void)
{
    uint32_t data[4];          /* Saving destination variables */

    if (AT_read_dct_hfail(AT_P(33),4,data)!=0){
        AT_printf("ERROR:AT_read_dct_hfail\n");
        exit(1);
    }
    return(0);
}
```

If DUT1 and DUT2 fail because the upper limit is exceeded, measurement data is saved in individual data elements as shown below.

```
DUT1 measurement result 1 (upper-limit failure) in data[0]
DUT2 measurement result 1 (upper-limit failure) in data[1]
DUT3 measurement result 0 in data[2]
DUT4 measurement result 0 in data[3]
```

2. 4.25 AT_read_dct_hfailm

Function: This library function reads the upper-limit failure results of DC parametric tests on multiple pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_hfailm
(
    int          npin,      /* number of pin data */
    uint16_t     pinno,     /* pin data */
    int          nelem,     /* number of data to read */
    uint32_t     *data      /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the upper limit specified in the LIMIT statement. It can read such data on multiple pins.

In the argument "pinno", specify the pointer that is set to an array, which has at least as many elements as specified in "npin", by using the pointer that stores the type and numbers of the target pins. Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT HFAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT HFAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

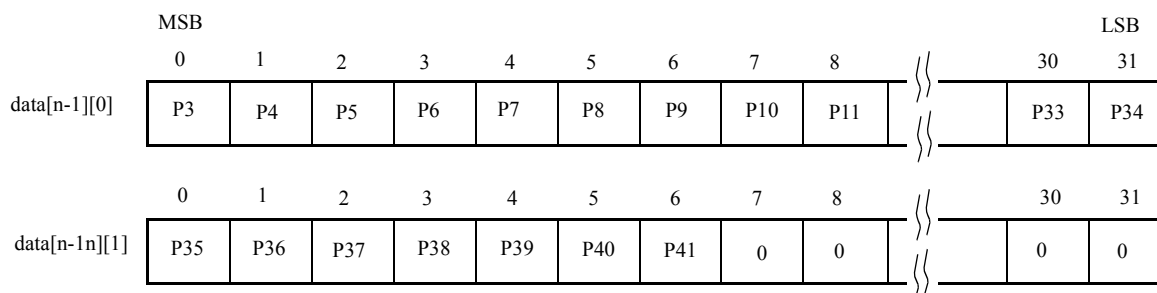
(1) Example of Reading the Upper-limit Failure Results of a Four Device Parallel Test Using I/O Pins 3 to 41 (P3-41)

```
int foo(void)
{
    uint16_t pindata[39];    /* Specifies the type and numbers of pins */
    uint32_t data[4][2];    /* Saving destination variables */

    for(i=3;i<=41;i++) pindata[i-3]=AT_P(i);
    /* Specifies a value from P3 to P41 */

    if (AT_read_dct_hfailm(39,pindata,8,data)!=0){
        AT_printf("ERROR:AT_read_dct_hfailm\n");
        exit(1);
    }
    return(0);
}
```

The data storage format is shown below. The bit associated with the upper-limit failed pin is turned on. "n" represents a DUT number.



Upper-limit failure information is saved as shown below.

Pin No.	DUT No.			
	DUT1	DUT2	DUT3	DUT4
P3 to 34	data[0][0]	data[1][0]	data[2][0]	data[3][0]
P35 to 41	data[0][1]	data[1][1]	data[2][1]	data[3][1]

2. 4.26 AT_read_dct_lfail

Function: This library function reads the lower-limit failure results of DC parametric tests on a specific pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_lfail
(
    uint16_t  pinno,    /* pin data */
    int       nelem,    /* number of data to read */
    uint32_t  *data     /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the lower limit specified in the LIMIT statement. It can read such data on a specific pin.

Specify the type and number of the target pin for the tests in the argument "pinno". Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT LFAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT LFAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the Lower-limit Failure Results of a Four Device Parallel Test Using I/O Pin 33 (P33)

```
int foo(void)
{
    uint32_t data[4];          /* Saving destination variables */

    if (AT_read_dct_lfail(AT_P(33),4,data)!=0){
        AT_printf("ERROR:AT_read_dct_lfail\n");
        exit(1);
    }
    return(0);
}
```

If the test for DUT4 fails because the lower limit is exceeded, measurement data is saved in individual data elements as shown below.

DUT1 measurement result 0 in data[0]

DUT2 measurement result 0 in data[1]

DUT3 measurement result 0 in data[2]

DUT4 measurement result 1 (lower-limit failure) in data[3]

2. 4.27 AT_read_dct_lfailm

Function: This library function reads the lower-limit failure results of DC parametric tests on multiple pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_lfailm
(
    int          npin,      /* number of pin data */
    uint16_t     pinno,     /* pin data */
    int          nelem,     /* number of data to read */
    uint32_t     *data      /* address to destination variable */
);
```

Description

This function reads the failure results of DC parametric tests exceeding the lower limit specified in the LIMIT statement. It can read such data on multiple pins.

In the argument "pinno", specify the pointer that is set to an array, which has at least as many elements as specified in "npin", by using the pointer that stores the type and numbers of the target pins. Use one of the following macros to specify the type and numbers of the pins:

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

If a pin not defined by a socket program is specified, an error occurs.

In the argument "nelem", specify the number of data items to be read.

Using the argument "data", set the pointer to the uint32_t-type array having at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT LFAIL statement.

➔ For more information on using this function, refer to the description of the READ DCT LFAIL statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

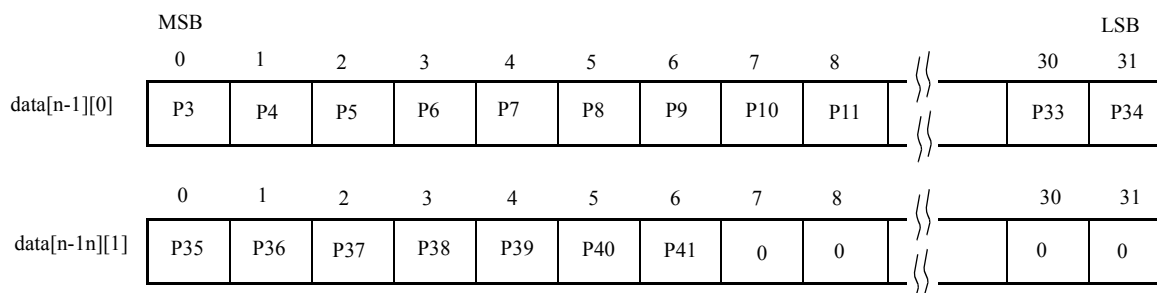
(1) Example of Reading the Lower-limit Failure Results of a Four Device Parallel Test Using I/O Pins 3 to 41 (P3-41)

```
int foo(void)
{
    uint16_t pindata[39];    /* Specifies the type and numbers of pins */
    uint32_t data[4][2];    /* Saving destination variables */

    for(i=3;i<=41;i++) pindata[i-3]=AT_P(i);
    /* Specifies a value from P3 to P41 */

    if (AT_read_dct_lfailm(39,pindata,8,data)!=0){
        AT_printf("ERROR:AT_read_dct_lfailm\n");
        exit(1);
    }
    return(0);
}
```

The data storage format is shown below. The bit associated with the lower-limit failed pin is turned on. "n" represents a DUT number.



Lower-limit failure information is saved as shown below.

Pin No.	DUT No.			
	DUT1	DUT2	DUT3	DUT4
P3 to 34	<code>data[0][0]</code>	<code>data[1][0]</code>	<code>data[2][0]</code>	<code>data[3][0]</code>
P35 to 41	<code>data[0][1]</code>	<code>data[1][1]</code>	<code>data[2][1]</code>	<code>data[3][1]</code>

2. 4.28 AT_read_dct_vs

Function: This library function reads VS measurement data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_vs
(
    uint16_t      vsno,    /* VS Number */
    int           nelem,   /* number of data to read */
    struct atreal *data    /* address to destination variable */
);
```

Description

This function reads the measurement data obtained by measuring the device power supply current (VS) during the DC parametric test.

In the argument "vsno", specify the unit number of the device power supply subject to measurement during the DC parametric test. Specify a unit number from 1 to 12.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to an array of the struct atreal structure where the array has at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT VS statement.

➔ For more information on using this function, refer to the description of the READ DCT VS statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the VS1 Data Obtained Using the DC Parametric Test during a Four Device Parallel Test

```
int foo(void)
{
    struct atreal data[4]; /* Saving destination variables */

    if (AT_read_dct_vs(1,4,data) !=0){
        AT_printf("ERROR:AT_read_dct_vs\n");
        exit(1);
    }
    return(0);
}
```

Measurement data is saved in individual "data" elements as shown below.

DUT1 measurement data in data[0]

DUT2 measurement data in data[1]

DUT3 measurement data in data[2]

DUT4 measurement data in data[3]

2. 4.29 AT_read_dct_data

Function: This library function reads DC measurement data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dct_data
(
    int          nelem, /* number of data to read          */
    struct atreal *data /* address to destination variable */
);
```

Description

This function reads the DC parametric test data obtained by measurement with the DC measuring unit.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to an array of the struct atreal structure where the array has at least as many elements as specified in the argument "nelem". However, note that element contents, which correspond to a DUT not targeted to the tests, do not change.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ DCT DATA statement.

➔ For more information on using this function, refer to the description of the READ DCT DATA statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading the DC Parametric Test Data Obtained from the DC Measuring Unit during a Four Device Parallel Test

```
int foo(void)
{
    struct atreal data[4]; /* Saving destination variables */

    if (AT_read_dct_data(4,data)!=0){
        AT_printf("ERROR:AT_read_dct_data\n");
        exit(1);    }
    return(0);
}
```

Measurement data is saved in individual "data" elements as shown below.

DUT1 measurement data in data[0]
DUT2 measurement data in data[1]
DUT3 measurement data in data[2]
DUT4 measurement data in data[3]

2. 4.30 AT_read_tpin

Function: This library function reads the pin numbers or VS numbers for which a DC parametric test was performed.

Syntax

```
#include "AT/hybrid.h"

int AT_read_tpin
(
    int          *number, /* address to number of data */
    int          nelem,   /* number of data to read */
    uint32_t     *data    /* address to destination variable */
);
```

Description

This function reads the number of the pin or the unit number of the device power supply (VS) for which the last DC parametric test was performed. Data of only the DUT with the number specified in AT_set_rdut() can be read. If AT_set_rdut() is not specified, data of DUT No. 1 can be read.

In the argument "number", specify the pointer that is set to the area into which the number of pin numbers or device power supply (VS) unit numbers are to be read.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

The argument meanings, the read data format, and notes are the same as those of the READ TPIN statement.

➔ For more information on using this function, refer to the description of the READ TPIN statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example in Which the Data Obtained Using the DC Parametric Test Performed on DUT No. 3 Is Read by a Program for a Four Device Parallel Test

```
int foo(void)
{
    int number;                /* Stores the number of pin numbers or device power
                               supply unit numbers */
    uint32_t data[10];         /* Saving destination variables */

    if (AT_set_rdut(3) != 0) {
        AT_printf("ERROR:AT_set_rdut\n");
        exit(1);
    }

    if (AT_read_tpin(&number, 10, data) != 0) {
        AT_printf("ERROR:AT_read_tpin\n");
        exit(1);
    }
    return(0);
}
```

2. 4.31 AT_read_fpin

Function: This library function reads the pin numbers or VS numbers that failed during a DC parametric test.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fpin
(
    int          *number, /* address to number of data */
    int          nelem,   /* number of data to read */
    uint32_t     *data    /* address to destination variable */
);
```

Description

This function reads the number of the pin or the unit number of the device power supply (VS) that failed during the last DC parametric test. Data of only the DUT with the number specified in AT_set_rdut() can be read. If AT_set_rdut() is not specified, data of DUT No. 1 can be read.

In the argument "number", specify the pointer that is set to an area into which the number of pin numbers or device power supply (VS) unit numbers are to be read.

In the argument "nelem", specify the number of data items to be read.

In the argument "data", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

The argument meanings, the read data format, and notes are the same as those of the READ FPIN statement.

➔ For more information on using this function, refer to the description of the READ FPIN statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example**(1) Example in Which the Data Obtained Using the DC Parametric Test Performed on DUT No. 3 Is Read by a Program for a Four Device Parallel Test**

```
int foo(void)
{
    int number;                /* Stores the number of pin numbers or device power
                               supply unit numbers */
    uint32_t data[10];         /* Saving destination variables */

    if (AT_set_rdut(3) != 0) {
        AT_printf("ERROR:AT_set_rdut\n");
        exit(1);
    }

    if (AT_read_fpin(&number, 10, data) != 0) {
        AT_printf("ERROR:AT_read_fpin\n");
        exit(1);
    }
    return(0);
}
```

2. 4.32 AT_set_category

Function: This library function sets a category lamp.

Syntax

```
#include "AT/hybrid.h"

int AT_set_category
(
    int    cateno        /* category number */
);
```

Description

This function specifies the category lamp that lights up at the end of a test. Specify the category number in the argument "cateno". Specify one of the values from 0 to 200. If 0 is specified, the function does nothing. If the number specified as the category number is outside of the range of 0 to 200, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.33 AT_reset_category

Function: This library function resets a category lamp.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_category
(
    int    cateno        /* category number */
);
```

Description

This function resets the setting of the category lamp that lights up at the end of a test. Specify the category number in the argument "cateno". Specify one of the values from 0 to 200. If 0 is specified, the function does nothing. If the number specified as the category number is outside of the range of 0 to 200, an error occurs.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.34 AT_read_category

Function: This library function obtains category data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_category
(
    int      nelem,      /* number of data to read */
    int32_t  *vp         /* address to destination variable */
);
```

Description

This function reads the category data defined by the program being executed.

In the argument "vp", specify a pointer that is set to an int32_t-type array which has at least as many elements as specified in the argument "nelem".

If the number of the DUT where reading starts is greater than the maximum number of the target DUTs, an error occurs.

Note:

The argument meanings, the read data format, and notes are the same as those of the READ CATEGORY statement.

➔ For more information on using this function, refer to the description of the READ CATEGORY statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Reading Categories 1 to 200 of DUT1 to DUT4

```

int foo(void)
{
    int ret;                /* Return value variable */
    int32_t vp[32];         /* Saving destination variables
                             Saves DUT1 categories 1 to 32 in vp[0].
                             Saves DUT1 categories 33 to 50 in vp[1].
                             :
                             Saves DUT4 categories 1 to 32 in vp[6].
                             Saves DUT4 categories 33 to 50 in vp[7].
                             -----
                             Saves DUT1 categories 51 to 82 in vp[8].
                             Saves DUT1 categories 83 to 100 in vp[9].
                             :
                             Saves DUT4 categories 51 to 82 in vp[14].
                             Saves DUT4 categories 83 to 100 in vp[15].
                             -----
                             Saves DUT1 categories 101 to 132 in vp[16].
                             Saves DUT1 categories 133 to 150 in vp[17].
                             :
                             Saves DUT4 categories 101 to 132 in vp[22].
                             Saves DUT4 categories 133 to 150 in vp[23].
                             -----
                             Saves DUT1 categories 151 to 182 in vp[24].
                             Saves DUT1 categories 183 to 200 in vp[25].
                             :
                             Saves DUT4 categories 151 to 182 in vp[30].
                             Saves DUT4 categories 183 to 200 in vp[31].*/

    ret=AT_read_category(32,vp);
    if (ret == 0)
    {
        Normal processing
    }
    else
    {
        Error processing
    }

    return(0);
}

```

2. 4.35 AT_set_fkmode

Function: This library function sets the function key mode.

Syntax

```
#include "AT/hybrid.h"

int AT_set_fkmode
(
    uint32_t  mdno    /* mode number */
);
```

Description

This function sets the function key mode. Specify a mode number in the argument "mdno". Specify a value within 32 bits.

➔ For the relationship between bits and modes, refer to the description of the SET_FKMODE statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.36 AT_reset_fkmode

Function: This library function resets the function key mode.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_fkmode
(
    uint32_t    mdno    /* mode number */
);
```

Description

This function resets the function key mode. Specify a mode number in the argument "mdno". Specify a value within 32 bits.

➔ For more information on the relationship between bits and modes, refer to the description of the RESET_FKMODE statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.37 AT_read_fk

Function: This library function reads the setting state of function keys.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fk
(
    uint32_t *vp      /* address to destination variable */
);
```

Description

This function reads the setting state of function keys in the control box. In the argument "vp", specify the pointer that is set to a uint32_t-type variable storing the definition state. The read data format is the same as for data read using the READ FK statement.

➔ For more information, refer to the description of the READ FK statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.38 AT_read_stn

Function: This library function reads the number of the station currently being used.

Syntax

```
#include "AT/hybrid.h"

int AT_read_stn(void);
```

Description

This function reads the number of the station currently being used.
Station numbers 1 and 2 can be read.

Return Value

If the function ends successfully, it returns the number of the station currently being used.
If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.39 AT_read_test

Function: This library function obtains the number of the test currently being executed.

Syntax

```
#include "AT/hybrid.h"

int AT_read_test(void);
```

Description

This function reads the number of the test being executed.

Test numbers range of 0 to 9999.

Return Value

If the function ends successfully, it returns the number of the test being executed.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.40 AT_read_stncnt

Function: This library function obtains the number of stations being used.

Syntax

```
#include "AT/hybrid.h"

int AT_read_stncnt(void);
```

Description

This function reads the number of stations being used.

Return Value

If the function ends successfully, it returns the number of stations being used.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.41 AT_read_dutcnt

Function: This library function obtains the number of DUTs currently being measured in parallel per station.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dutcnt(void);
```

Description

This function reads the number of DUTs currently measured in parallel per station.

Return Value

If the function ends successfully, it returns the number of DUTs currently measured in parallel per station.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.42 AT_send_gpib_function

Function: This library function controls the unit connected to the GPIB interface.

Syntax

```
#include "AT/hybrid.h"

int AT_send_gpib_function
(
    int          cont    /* control code */
);
```

Description

This function controls the unit connected to the GPIB interface.

Specify a control code in the argument "cont". Control codes are defined as macros, which are listed below.

Control Code	Description
AT_CLEAR	Outputs "SIC" to the GPIB interface to set it to initialize status.
AT_REMOTE	Sets GPIB to remote mode.
AT_LOCAL	Sets GPIB to local mode (releases it from remote mode).
AT_GTL	Sets the unit specified as a listener in local mode.
AT_SDC	Initializes a unit specified as a listener.
AT_GET	Triggers a unit specified as a listener.
AT_LLO	Prohibits the unit from returning to manual operation mode.
AT_DCL	Initializes all units on the bus.
AT_SPE	Enters serial polling mode to enable transmission of status bytes.
AT_SPD	Prohibits serial polling mode.
AT_UNL	Releases all listeners.
AT_UNT	Releases all talkers.

Each control code is ensured a non 0 value.

Note:

This function cannot be called from the asynchronous event handler.

➔ For more information on using this function, refer to the description of the SEND GPIB statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 43 "AT_send_gpib_data" on page 2-79](#), [2. 4. 44 "AT_read_gpib_srq" on page 2-81](#), and [2. 4. 45 "AT_read_gpib_data" on page 2-82](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.43 AT_send_gpib_data

Function: This library function transfers data to the listener unit (unit that receives data).

Syntax

```
#include "AT/hybrid.h"

int AT_send_gpib_data
(
    int      mla      /* listener unit address */
    char     *src      /* character data to set */
    size_t   size      /* number of data to set */
);
```

Description

This function defines the unit connected to the GPIB interface as the listener (unit that receives data) and transfers data to it.

Specify the unit address for argument "mla". Specify a value within 0 to 31 bits.

For argument "src", specify the pointer to a char-type array having as many elements as, or more elements than, specified by argument "size".

The function transfers "size" bytes of data pointed by "src" to the unit. The maximum number of bytes that can be transferred at a time is 1 MB.

This function cannot be called from the asynchronous event handler.

➔ For more information, refer to the description of the SEND GPIB DATA statement in [2. 4. 42 "AT_send_gpib_function" on page 2-77](#), [2. 4. 44 "AT_read_gpib_srq" on page 2-81](#), and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.44 AT_read_gpib_srq

Function: This library function obtains the current status of the GPIB service request line.

Syntax

```
#include "AT/hybrid.h"

int AT_read_gpib_srq(void);
```

Description

This function reads the current status of the GPIB service request line.

When a service request is issued, 1 is read. If it is not issued, 0 is read.

This function cannot be called from the asynchronous event handler.

➔ For more information, refer to the description of the READ GPIB SRQ statement in [2. 4. 42 "AT_send_gpib_function" on page 2-77](#), [2. 4. 43 "AT_send_gpib_data" on page 2-79](#), and *ATL-51 Test Plan Program Reference Manual(No. : 8431498)*.

Return Value

If the function ends successfully, it returns 0 or 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.45 AT_read_gpib_data

Function: This library function obtains data from the talker unit (unit which sends data).

Syntax

```
#include "AT/hybrid.h"

int AT_read_gpib_data
(
    int      mta      /* talker unit address */
    char     *dest     /* where to put data */
    size_t   size     /* number of characters to put */
    char     term     /* terminate character */
);
```

Description

This function defines the unit connected to the GPIB interface as the talker (unit which sends data) and reads data or status information.

Specify the unit address for argument "mta". Specify a value within 0 to 31 bits.

For argument "dest", specify the pointer to a char-type array having as many elements as, or more elements than, specified by argument "size".

For argument "term", specify the END data code. If an END data code does not need to be specified, specify AT_NTC.

This function ends processing when data at the End of Identify (EOI) bit has been input or "size" bytes of data have been read to the pointer indicated by "dest". However, when the same code as the code specified by "term" has been read, the function ends processing at that point. The maximum number of bytes that can be transferred at a time is 1 MB.

This function cannot be called from the asynchronous event handler.

➔ For more information, refer to the description of the READ GPIB DATA statement in [2. 4. 42 "AT_send_gpib_function" on page 2-77](#), [2. 4. 44 "AT_read_gpib_srq" on page 2-81](#), and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_NOT_ASYNC_CALLABLE

The function was called from an asynchronous event handler.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.46 AT_gpib_timeout

Function: This library function changes time-out time.

Syntax

```
#include "AT/hybrid.h"

int AT_gpib_timeout
(
    int    timeout    /* timeout in msec */
);
```

Description

This function changes the time-out monitoring time that applies to data transfer to or from the unit connected to the GPIB interface.

For argument "timeout", specify the time-out monitoring time in milliseconds.

➔ For more information, refer to the description of the TIME OUT statement in [2. 4. 42 "AT_send_gpib_function" on page 2-77](#), [2. 4. 43 "AT_send_gpib_data" on page 2-79](#), [2. 4. 44 "AT_read_gpib_srq" on page 2-81](#), and *ATL-51 Test Plan Program Reference Manual(No. : 8431498)*.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.47 AT_send_ext

Function: This library function writes to the tester bus.

Syntax

```
#include "AT/hybrid.h"

int AT_send_ext
(
    int32_t    addr,    /* tester address */
    int32_t    data     /* data to send */
);
```

Description

This function transfers the data specified in argument "data" to the tester address specified in argument "addr".

➔ For more information, refer to [2. 4. 48 "AT_read_ext" on page 2-86](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

2. 4.48 AT_read_ext

Function: This library function reads data from the tester bus.

Syntax

```
#include "AT/hybrid.h"

int AT_read_ext
(
    int32_t    addr,    /* tester address */
    int32_t    *buf     /* where to put data */
);
```

Description

This function reads data from the tester address specified in argument "addr" into the area specified in argument "buf".

➔ For more information, refer to [2. 4. 47 "AT_send_ext" on page 2-85](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

2. 4.49 AT_set_decision

Function: This library function updates CATEGORY, SORT, and COUNTER data.

Syntax

```
#include "AT/hybrid.h"

int AT_set_decision
(
    int    function /* macro name */
);
```

Description

This function updates CATEGORY and SORT data assigned to ATL CATEGORY and SORT TABLE during program execution.

CATEGORY, SORT, and COUNTER data can be updated only once.

If this function is used more than once, the program ends without any update of CATEGORY, SORT, and COUNTER data.

In the argument "function", specify the result of an OR operation of the individual bits for the following macros.

Macro	Description
D_CATEGORY	Updates the CATEGORY data assigned to the ATL CATEGORY TABLE.
D_SORT	Updates the SORT data assigned to the ATL SORT TABLE.
D_COUNTER	Updates COUNTER data (LOT COUNTER and WAFER COUNTER).

If two or more macros are specified in the argument "function", the "function" updates CATEGORY, SORT, and COUNTER, in this sequence.

➔ For more information, refer to the description of the SET DECISION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Updating CATEGORY Data

```
if(AT_set_decision(D_CATEGORY)==-1){  
    AT_printf("Error:AT_set_decision: \n");  
    exit(1);  
}
```

(2) Example of Updating CATEGORY, SORT, and COUNTER Data

```
if(AT_set_decision(D_CATEGORY|D_SORT|D_COUNTER)==-1){  
    AT_printf("Error:AT_set_decision: \n");  
    exit(1);  
}
```

2. 4.50 AT_set_rejection

Function: This library function excludes a DUT from test target.

Syntax

```
#include "AT/hybrid.h"

int AT_set_rejection
(
    int      argtype,    /* argument type */
    uint32_t dutno       /* number of data to set */
);
```

Description

This function excludes the applicable DUTs from testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET REJECTION statement.

➔ For more information on using this function, refer to the description of the SET REJECTION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 51 "AT_set_rejectionm" on page 2-91](#)

Return Value

When target DUTs are found as the result of executing this function, it returns 0.

When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Excluding DUT1 from Test Target

```
int foo(void)
{
    int          ret;
    uint32_t     dutno;

    dutno = 1;
    ret = AT_set_rejection(0,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.51 AT_set_rejectionm

Function: This library function excludes a DUT from test target.

Syntax

```
#include "AT/hybrid.h"

int AT_set_rejectionm
(
    int          argtype, /* argument type */
    int          count,   /* argument count */
    uint32_t     *dutno   /* number of data to set */
);
```

Description

This function excludes the applicable DUTs from testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

Specify the pointer indicating the target DUTs in the argument "dutno", where the pointer is set to a uint32_t-type array having at least as many elements as specified in "count".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET REJECTION statement.

➔ For more information on using this function, refer to the description of the SET REJECTION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 50 "AT_set_rejection" on page 2-89](#)

Return Value

When target DUTs are found as the result of executing this function, it returns 0.

When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Excluding DUT2 and DUT4 from Test Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x0000000a;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_set_rejectionm(1,16,dutno);
    if ( ret == 0 ) {
        Processing when the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.52 AT_set_jdut

Function: This library function specifies the target DUTs for test post-processing.

Syntax

```
#include "AT/hybrid.h"

int AT_set_jdut
(
    int          argtype, /* argument type */
    uint32_t     dutno    /* number of data to set */
);
```

Description

This function specifies the target DUT for test post-processing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

AT_set_jdut influences the following functions:

AT_set_rejection(), AT_set_rejectionm(), AT_set_exclusion(), AT_set_exclusionm(),
AT_reset_exclusion(), AT_reset_exclusionm(), AT_set_exclusion2(), AT_set_exclusionm2(),
AT_reset_exclusion2(), AT_reset_exclusionm2(), AT_set_sort(), AT_set_category(),
AT_reset_category(), AT_set_result(), AT_reset_result(), AT_set_fail(), AT_reset_fail(),
AT_set_cnt(), AT_reset_cnt(), AT_set_wfcnt(), AT_reset_wfcnt()

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET JDUT statement.

➔ For more information on using this function, refer to the description of the SET/RESET JDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 54 "AT_reset_jdut" on page 2-98](#) and [2. 4. 53 "AT_set_jdutm" on page 2-95](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Specifying DUT10

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 10;
    ret = AT_set_jdut(0,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```


2. 4.53 AT_set_jdutm

Function: This library function specifies the target DUTs for test post-processing.

Syntax

```
#include "AT/hybrid.h"

int AT_set_jdutm
(
    int      argtype,          /* argument type */
    int      count,            /* argument count */
    uint32_t *dutno            /* number of data to set */
);
```

Description

This function specifies the target DUTs for test post-processing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

Specify the pointer indicating the target DUTs in the argument "dutno", where the pointer is set to a uint32_t-type array having at least as many elements as specified in "count".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the "argtype", it means that the DUT number was specified in a bitwise manner.

AT_set_rdut influences the following functions:

AT_set_rejection(), AT_set_rejectionm(), AT_set_exclusion(), AT_set_exclusionm(),
AT_reset_exclusion(), AT_reset_exclusionm(), AT_set_exclusion2(), AT_set_exclusionm2(),
AT_reset_exclusion2(), AT_reset_exclusionm2(), AT_set_sort(), AT_set_category(),
AT_reset_category(), AT_set_result(), AT_reset_result(), AT_set_fail(), AT_reset_fail(),
AT_set_cnt(), AT_reset_cnt(), AT_set_wfcnt(), AT_reset_wfcnt()

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET JDUT statement.

➔ For more information on using this function, refer to the description of the SET/RESET JDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 52 "AT_set_jdut" on page 2-93](#) and [2. 4. 54 "AT_reset_jdut" on page 2-98](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Specifying DUT2 and DUT3

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000006;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_set_jdutm(1,16,dutno);
    if (ret == 0) [
        Processing that is performed if the function ends successfully
    ] else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.54 AT_reset_jdut

Function: This library function targets all DUTs for test post-processing.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_jdut(void);
```

Description

This function targets all DUTs for test post-processing.

AT_reset_jdut influences the following functions:

AT_set_rejection(), AT_set_rejectionm(), AT_set_exclusion(), AT_set_exclusionm(),
AT_reset_exclusion(), AT_reset_exclusionm(), AT_set_exclusion2(), AT_set_exclusionm2(),
AT_reset_exclusion2(), AT_reset_exclusionm2(), AT_set_sort(), AT_set_category(),
AT_reset_category(), AT_set_result(), AT_reset_result(), AT_set_fail(), AT_reset_fail(),
AT_set_cnt(), AT_reset_cnt(), AT_set_wfcnt(), AT_reset_wfcnt()

Note:

Precautions for use are the same as those of the SET/RESET JDUT statement.

➔ For more information on using this function, refer to the description of the SET/RESET JDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 52 "AT_set_jdut" on page 2-93](#) and [2. 4. 53 "AT_set_jdutm" on page 2-95](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.55 AT_set_rdut

Function: This library function specifies the DUT number at which reading begins.

Syntax

```
#include "AT/hybrid.h"

int AT_set_rdut
(
    int dutno /* number of data to set */
);
```

Description

This function specifies the number of the DUT where reading starts for test post-processing. In the argument "dutno", specify the number of the DUT where reading starts.

AT_set_rdut influences the following functions:

AT_read_dct(), AT_read_func(), AT_read_cumfunc(), AT_read_category(), AT_read_cnt(), AT_read_ctcnt(), AT_read_result(), AT_read_fail(), AT_read_fail_test_cnt(), AT_read_sort(), AT_read_cnt(), AT_read_ctcnt(), AT_read_wfcnt(), AT_read_wfctcnt()

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET RDUT statement.

➔ For more information on using this function, refer to the description of the SET RDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

2. 4.56 AT_read_ddut

Function: This library function reads DDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_ddut
(
    int      nelem,    /* number of data to read */
    uint32_t *p        /* pointer to array */
);
```

Description

This function reads the data of DUTs specified for parallel measurement (DDUTs) in the program, and saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight DDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};
    if (AT_read_ddut(nelem,p) != 0){
        AT_printf("Error:AT_read_ddut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.57 AT_read_sdut

Function: This library function reads SDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_sdut
(
    int      nelem,    /* number of data to read */
    uint32_t *p        /* pointer to array */
);
```

Description

This function reads the data of target DUTs (SDUTs), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight SDUT Data Items

```
int foo(void)
{
    int          nelem=8;
    uint32_t      p[8]={0};

    if (AT_read_sdut(nelem,p) != 0){
        AT_printf("Error:AT_read_sdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.58 AT_read_mdut

Function: This library function reads MDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_mdut
(
    int          nelem,    /* number of data to read */
    uint32_t     *p        /* pointer to array */
);
```

Description

This function reads data of test target DUTs (MDUTs), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight MDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};

    if (AT_read_mdut(nelem,p) != 0){
        AT_printf("Error:AT_read_mdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.59 AT_read_edut

Function: This library function reads EDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_edut
(
    int      nelem,    /* number of data to read */
    uint32_t *p        /* pointer to array */
);
```

Description

This function reads the data of target DUTs (EDUTs), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight EDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};

    if (AT_read_edut(nelem,p) != 0){
        AT_printf("Error:AT_read_edut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.60 AT_read_cdut

Function: This library function reads CDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cdut
(
    int      nelem,    /* number of data to read */
    uint32_t *p        /* pointer to array */
);
```

Description

This function reads the data of the target DUTs (CDUT), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight CDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};

    if (AT_read_cdut(nelem,p) != 0){
        AT_printf("Error:AT_read_cdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.61 AT_read_pdut

Function: This library function reads PDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_pdut
(
    int          nelem,    /* number of data to read */
    uint32_t     *p        /* pointer to array */
);
```

Description

This function reads the data of DUTs that passed the preceding test (PDUT), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight PDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};

    if (AT_read_pdut(nelem,p) != 0){
        AT_printf("Error:AT_read_pdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.62 AT_read_pdut_t

Function: This library function reads PDUT data when a test number is specified.

Syntax

```
#include "AT/hybrid.h"

int AT_read_pdut_t
(
    int      testno, /* test no. */
    int      nelem,  /* number of data to read */
    uint32_t *p      /* pointer to array */
);
```

Description

This function reads the data of DUTs, that passed the test specified in "testno" (PDUT), to the data at the address specified in "p".

In the argument "testno", specify the number of the test to be read.

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight Items of PDUT Data Corresponding to Test Number 1

```
int foo(void)
{
    int          testno=1;
    int          nelem=8;
    uint32_t     p[8]={0};

    if (AT_read_pdut_t(testno,nelem,p) != 0){
        AT_printf("Error:AT_read_pdut_t: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.63 AT_read_fdut

Function: This library function reads FDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fdut
(
    int          nelem,    /* number of data to read */
    uint32_t     *p        /* pointer to array */
);
```

Description

This function reads the data of DUTs that failed during the preceding test (FDUT), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight FDUT Data Items

```
int foo(void)
{
    int      nelem=8;
    uint32_t p[8]={0};

    if (AT_read_fdut(nelem,p) != 0){
        AT_printf("Error:AT_read_fdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.64 AT_read_fdut_t

Function: This library function reads FDUT data when a test number is specified.

Syntax

```
#include "AT/hybrid.h"

int AT_read_fdut_t
(
    int      testno, /* test no. */
    int      nelem,  /* number of data to read */
    uint32_t *p      /* pointer to array */
);
```

Description

This function reads the data of DUTs that failed during the test specified in "testno" (FDUT), and it saves the data at the address specified in "p".

In the argument "testno", specify the number of the test to be read.

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight Items of FDUT Data Corresponding to Test Number 1

```
int foo(void)
{
    int          testno=1;
    int          nelem=8;
    uint32_t     p[8]={0};

    if (AT_read_fdut_t(testno,nelem,p) != 0){
        AT_printf("Error:AT_read_fdut_t: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.65 AT_read_gfdut

Function: This library function reads GFDUT.

Syntax

```
#include "AT/hybrid.h"

int AT_read_gfdut
(
    int      nelem,    /* number of data to read */
    uint32_t *p        /* pointer to array */
);
```

Description

This function reads the data of DUTs whose previous test results show a failure (GFDUT), and it saves the data at the address specified in "p".

In the argument "nelem", specify the number of array elements where read data is stored.

In the argument "p", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

➔ For the read data format, refer to the description of the READ mDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading Eight GFDUT Data Items

```
int foo(void)
{
    int          nelem=8;
    uint32_t      p[8]={0};

    if (AT_read_gfdut(nelem,p) != 0){
        AT_printf("Error:AT_read_gfdut: \n");
        exit(1);
    }
    return 0;
}
```

2. 4.66 AT_set_sdut

Function: This library function updates the target DUTs.

Syntax

```
#include "AT/hybrid.h"

int AT_set_sdut
(
    int      argtype, /* argument type */
    uint32_t dutno    /* number of data to set */
);
```

Description

This function specifies the target DUT(s).

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET SDUT statement.

➔ For more information on using this function, refer to the description of the SET SDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 67 "AT_set_sdutm" on page 2-122](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Updating DUT5 to DUT8

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 0x000000f0;
    ret = AT_set_sdut(1,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.67 AT_set_sdutm

Function: This library function updates the target DUTs.

Syntax

```
#include "AT/hybrid.h"

int AT_set_sdutm
(
    int          argtype, /* argument type */
    int          count,   /* argument count */
    uint32_t     *dutno   /* number of data to set */
);
```

Description

This function specifies the target DUTs.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

Specify the pointer indicating the target DUTs in the argument "dutno", where the pointer is set to a uint32_t-type array having at least as many elements as specified in "count".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET SDUT statement.

➔ For more information on using this function, refer to the description of the SET SDUT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 66 "AT_set_sdut" on page 2-120](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Specifying DUT1 and DUT7

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000041;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_set_sdutm(1,16,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.68 AT_set_exclusion

Function: This library function excludes a specific DUT from the target temporarily.

Syntax

```
#include "AT/hybrid.h"

int AT_set_exclusion
(
    int      argtype, /* argument type */
    uint32_t dutno    /* number of data to set */
);
```

Description

This function temporarily excludes a specific DUT(s) from the target.

While the electrical connection stays open, the specified DUTs are excluded from GO or NOGO testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 69 "AT_set_exclusionm" on page 2-126](#), [2. 4. 70 "AT_reset_exclusion" on page 2-129](#), [2. 4. 72 "AT_set_exclusion2" on page 2-134](#), [2. 4. 73 "AT_set_exclusionm2" on page 2-136](#), and [2. 4. 75 "AT_reset_exclusionm2" on page 2-141](#)

Return Value

When target DUTs are found as the result of executing this function, it returns 0.

When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Temporarily Excluding DUT3 from Test Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 3;
    ret = AT_set_exclusion(0,dutno);
    if ( ret == 0 ) {
        Processing when the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.69 AT_set_exclusionm

Function: This library function excludes a specific DUT from the target temporarily.

Syntax

```
#include "AT/hybrid.h"

int AT_set_exclusionm
(
    int      argtype, /* argument type */
    int      count,   /* argument count */
    uint32_t *dutno   /* number of data to set */
);
```

Description

This function temporarily excludes a specific DUT from the target.

While the electrical connection stays open, the specified DUTs are excluded from GO or NOGO testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

In the argument "dutno", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in "count", by using the pointer that stores the target DUTs.

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

2. 4. 68 "AT_set_exclusion" on page 2-124, 2. 4. 70 "AT_reset_exclusion" on page 2-129, 2. 4. 71 "AT_reset_exclusionm" on page 2-131, 2. 4. 72 "AT_set_exclusion2" on page 2-134, 2. 4. 73 "AT_set_exclusionm2" on page 2-136, 2. 4. 74 "AT_reset_exclusion2" on page 2-139, and 2. 4. 75 "AT_reset_exclusionm2" on page 2-141

Return Value

When target DUTs are found as the result of executing this function, it returns 0.

When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Excluding DUT10 from the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000200;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_set_exclusionm(1,16,dutno);
    if (ret == 0) {
        Processing when the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.70 AT_reset_exclusion

Function: This library function restores a DUT that was temporarily excluded from the target.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_exclusion
(
    int          argtype, /* argument type */
    uint32_t     dutno    /* number of data to set */
);
```

Description

This function restores DUTs to testing that were temporarily excluded from testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 68 "AT_set_exclusion" on page 2-124](#), [2. 4. 69 "AT_set_exclusionm" on page 2-126](#), [2. 4. 71 "AT_reset_exclusionm" on page 2-131](#), [2. 4. 72 "AT_set_exclusion2" on page 2-134](#), [2. 4. 73 "AT_set_exclusionm2" on page 2-136](#), [2. 4. 74 "AT_reset_exclusion2" on page 2-139](#), and [2. 4. 75 "AT_reset_exclusionm2" on page 2-141](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Restoring DUT15 Back to the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 15;
    ret = AT_reset_exclusion(0,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.71 AT_reset_exclusionm

Function: This library function restores a DUT that was temporarily excluded from the target.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_exclusionm
(
    int      argtype, /* argument type */
    int      count,   /* argument count */
    uint32_t *dutno   /* number of data to set */
);
```

Description

This function restores DUTs to testing that were temporarily excluded from testing.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

In the argument "dutno", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in "count", by using the pointer that stores the target DUTs.

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 68 "AT_set_exclusion" on page 2-124](#), [2. 4. 69 "AT_set_exclusionm" on page 2-126](#), [2. 4. 70 "AT_reset_exclusion" on page 2-129](#), [2. 4. 72 "AT_set_exclusion2" on page 2-134](#), [2. 4. 73 "AT_set_exclusionm2" on page 2-136](#),

[2. 4. 74 "AT_reset_exclusion2" on page 2-139](#), and [2. 4. 75 "AT_reset_exclusionm2" on page 2-141](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Restoring DUT10 and DUT11 Back to the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000600;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_reset_exclusionm(1,16,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.72 AT_set_exclusion2

Function: This library function excludes a specific DUT from the target temporarily.

Syntax

```
#include "AT/hybrid.h"

int AT_set_exclusion2
(
    int      argtype, /* argument type */
    uint32_t dutno    /* number of data to set */
);
```

Description

This function temporarily excludes a specific DUT from the target.

The specified DUTs are excluded from testing and disconnected electrically.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION2 statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 68 "AT_set_exclusion" on page 2-124](#), [2. 4. 69 "AT_set_exclusionm" on page 2-126](#), [2. 4. 70 "AT_reset_exclusion" on page 2-129](#), [2. 4. 71 "AT_reset_exclusionm" on page 2-131](#), [2. 4. 73 "AT_set_exclusionm2" on page 2-136](#), [2. 4. 74 "AT_reset_exclusion2" on page 2-139](#), and [2. 4. 75 "AT_reset_exclusionm2" on page 2-141](#)

Return Value

When target DUTs are found as the result of executing this function, it returns 0. When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Excluding DUT5 from the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 5;
    ret = AT_set_exclusion2(0,dutno);
    if ( ret == 0 ) {
        Processing when the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.73 AT_set_exclusionm2

Function: This library function excludes a specific DUT from the target temporarily.

Syntax

```
#include "AT/hybrid.h"

int AT_set_exclusionm2
(
    int      argtype, /* argument type */
    int      count,   /* argument count */
    uint32_t *dutno   /* number of data to set */
);
```

Description

This function temporarily excludes a specific DUT from the target.

The specified DUTs are excluded from testing and disconnected electrically.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

In the argument "dutno", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in "count", by using the pointer that stores the target DUTs.

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION2 statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

2. 4. 68 "AT_set_exclusion" on page 2-124, 2. 4. 69 "AT_set_exclusionm" on page 2-126, 2. 4. 70 "AT_reset_exclusion" on page 2-129, 2. 4. 71 "AT_reset_exclusionm" on page 2-131, 2. 4. 72 "AT_set_exclusion2" on page 2-134, 2. 4. 74 "AT_reset_exclusion2" on page 2-139, and 2. 4. 75 "AT_reset_exclusionm2" on page 2-141

Return Value

When target DUTs are found as the result of executing this function, it returns 0. When no target DUTs are found as the result of executing this function, it returns 1.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Excluding DUT10 and DUT11 from the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000600;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_set_exclusionm2(1,16,dutno);
    if ( ret == 0 ) {
        Processing when the target DUTs are found
    } else if (ret == 1) {
        Processing that is performed if no target DUTs are found
    } else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.74 AT_reset_exclusion2

Function: This library function restores a DUT that was temporarily excluded from the target.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_exclusion2
(
    int      argtype, /* argument type */
    uint32_t dutno    /* number of data to set */
);
```

Description

This function restores DUTs to testing that were temporarily excluded from testing.

The target DUTs are reconnected electrically.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the target DUT(s) in the argument "dutno".

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 68 "AT_set_exclusion" on page 2-124](#), [2. 4. 69 "AT_set_exclusionm" on page 2-126](#), [2. 4. 70 "AT_reset_exclusion" on page 2-129](#), [2. 4. 71 "AT_reset_exclusionm" on page 2-131](#), [2. 4. 72 "AT_set_exclusion2" on page 2-134](#), [2. 4. 73 "AT_set_exclusionm2" on page 2-136](#), and [2. 4. 75 "AT_reset_exclusionm2" on page 2-141](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Restoring DUT30 Back to the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno;

    dutno = 30;
    ret = AT_reset_exclusion2(0,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    }
    else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.75 AT_reset_exclusionm2

Function: This library function restores a DUT that was temporarily excluded from the target.

Syntax

```
#include "AT/hybrid.h"

int AT_reset_exclusionm2
(
    int      argtype,    /* argument type */
    int      count,      /* argument count */
    uint32_t *dutno      /* number of data to set */
);
```

Description

This function restores DUTs to testing that were temporarily excluded from testing.

The target DUTs are reconnected electrically.

Specify the method of specifying "dutno" in the argument "argtype". Specify 0 or 1.

Specify the number of "dutno" array elements in the argument "count".

In the argument "dutno", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in "count", by using the pointer that stores the target DUTs.

When you specify 0 in the argument "argtype", it means that the DUT number was specified directly. When you specify 1 in the argument "argtype", it means that the DUT number was specified in a bitwise manner.

Note:

The meaning of data specified in each argument, the data format, and notes in use are the same as those of the SET/RESET EXCLUSION statement.

➔ For more information on using this function, refer to the description of the SET/RESET EXCLUSION2 statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

2. 4. 68 "AT_set_exclusion" on page 2-124, 2. 4. 69 "AT_set_exclusionm" on page 2-126, 2. 4. 70 "AT_reset_exclusion" on page 2-129, 2. 4. 71 "AT_reset_exclusionm" on page 2-131, 2. 4. 72 "AT_set_exclusion2" on page 2-134, 2. 4. 73 "AT_set_exclusionm2" on page 2-136, and 2. 4. 74 "AT_reset_exclusion2" on page 2-139

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of Restoring DUT8 Back to the Target

```
int foo(void)
{
    int      ret;
    uint32_t dutno[16];

    dutno[0] = 0x00000080;
    dutno[1] = 0x0;
    dutno[2] = 0x0;
    dutno[3] = 0x0;
    dutno[4] = 0x0;
    dutno[5] = 0x0;
    dutno[6] = 0x0;
    dutno[7] = 0x0;
    dutno[8] = 0x0;
    dutno[9] = 0x0;
    dutno[10] = 0x0;
    dutno[11] = 0x0;
    dutno[12] = 0x0;
    dutno[13] = 0x0;
    dutno[14] = 0x0;
    dutno[15] = 0x0;
    ret = AT_reset_exclusionm2(1,16,dutno);
    if ( ret == 0 ) {
        Processing that is performed if the function ends successfully
    }
    else {
        Processing performed if the function fails
    }

    return(0);
}
```

2. 4.76 AT_read_func

Function: This library function reads the functional test failure data of individual pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_func
(
    int      request,    /* kind of STRB reading fail data */
    uint16_t pinno,      /* pin kind and pin number */
    int      destcnt,    /* number of data to read */
    uint32_t *dest       /* where to put data */
);
```

Description

This function reads the functional test failure data of the specified pins. Only a single pin number can be specified.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to "n" of READ FUNCn.

In the argument "pinno", specify the type and number of the pin whose functional test failure data is read. Use one of the following macros to specify the type and number of each target pin.

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ FUNC statement.

➔ For more information on using this function, refer to the description of the READ FUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 77 "AT_read_funcm" on page 2-146](#) and [2. 4. 78 "AT_read_func_block" on page 2-149](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Pin 33 in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Functional Test Failures Occurred at Pin 33

```
#define DUTNO 128

int foo(void)
{
    uint32_t dest[DUTNO];
    int i;

    if(AT_read_func(1, AT_P(33), DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_func()\n");
        exit(1);
    }
    for(i = 0; i < DUTNO; i++) {
        if((dest[i] & 0x80000000) != 0)
            AT_printf("FAIL PIN:P33 DUT:%3d\n", i + 1);
    }

    return(0);
}
```

2. 4.77 AT_read_funcm

Function: This library function reads the functional test failure data of individual pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_funcm
(
    int      request, /* kind of STRB reading fail data */
    int      pincnt,  /* number of pin to specify */
    uint16_t *pinno,  /* array of pin kind and pin number */
    int      destcnt, /* number of data to read */
    uint32_t *dest    /* where to put data */
);
```

Description

This function reads the functional test failure data of the specified pins. More than one pin number can be specified.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to "n" of READ_FUNCn.

In the argument "pincnt", specify the type and the number of pins whose failure data is read.

In the argument "pinno", specify the pointer indicating the type and numbers of the pins whose functional test failure data is read. The pointer must be set to a uint16_t-type array having at least as many elements as specified in the argument "pincnt". Use one of the following macros to specify the type and number of each target pin.

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ_FUNC statement.

➔ For more information on using this function, refer to the description of the READ_FUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 76 "AT_read_func" on page 2-144](#) and [2. 4. 78 "AT_read_func_block" on page 2-149](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO**EINVAL**

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

S_atlerr_XXXXThe error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Pins 33 to 36 in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Functional Test Failures Occurred at Each Pin

```
#define PINNO 4
#define DUTNO 128

int foo(void)
{
    uint16_t pinno[PINNO];
    uint32_t dest[DUTNO];
    int i, j;

    for(i = 0; i < PINNO; i++)
        pinno[i] = AT_P(i + 33);

    if(AT_read_funcm(1, PINNO, pinno, DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_funcm()\n");
        exit(1);
    }
    for(j = 0; j < PINNO; j++) {
        for(i = 0; i < DUTNO; i++) {
            if((dest[i] & (0x80000000 >> j)) != 0)
                AT_printf("FAIL PIN:P%d DUT:%3d\n", j + 33, i + 1);
        }
    }

    return(0);
}
```

2. 4.78 AT_read_func_block

Function: This library function reads the functional test failure data of individual pins.

Syntax

```
#include "AT/hybrid.h"

int AT_read_func_block
(
    int      request, /* kind of STRB reading fail data */
    int      blkno,   /* block number */
    int      destcnt, /* number of data to read */
    uint32_t *dest    /* where to put data */
);
```

Description

This function reads the functional test failure data of pins of the specified block.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to "n" of READ FUNCn.

In the argument "blkno", specify the block number whose functional test failure data is read.

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ FUNC statement.

➔ For more information on using this function, refer to the description of the READ FUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 76 "AT_read_func" on page 2-144](#) and [2. 4. 77 "AT_read_funcm" on page 2-146](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Block Number 1 (pins 33 to 64) in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Functional Test Failures Occurred at Each Pin

```
#define PINNO    32
#define DUTNO    128

int foo(void)
{
    uint32_t  dest[DUTNO];
    int       i, j;

    if(AT_read_func_block(1, 1, DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_func_block()\n");
        exit(1);
    }
    for(j = 0; j < PINNO; j++) {
        for(i = 0; i < DUTNO; i++) {
            if((dest[i] & (0x80000000 >> j)) != 0)
                AT_printf("FAIL  PIN:P%d DUT:%3d\n", j + 33, i + 1);
        }
    }

    return(0);
}
```


2. 4.79 AT_read_cumfunc

Function: This library function reads failure data that is accumulated for each pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cumfunc
(
    int      request, /* kind of STRB reading fail data */
    uint16_t pinno,   /* pin kind and pin number */
    int      destcnt, /* number of data to read */
    uint32_t *dest     /* where to put data */
);
```

Description

This function reads failure data that is accumulated for the specified pin numbers. Only a single pin number can be specified.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to n of READ CUMFUNCn.

In the argument "pinno", specify the type and number of the pin whose cumulative failure data is read. Use one of the following macros to specify the type and number of each target pin.

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ CUMFUNC statement.

➔ For more information on using this function, refer to the description of the READ CUMFUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Related Items

[2. 4. 80 "AT_read_cumfuncm" on page 2-153](#)
and [2. 4. 81 "AT_read_cumfunc_block" on page 2-156](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Pin 33 in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Accumulated Failure Data Was Found at Pin 33

```
#define DUTNO 128

int foo(void)
{
    uint32_t dest[DUTNO];
    int i;

    if(AT_read_cumfunc(1, AT_P(33), DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_cumfunc()\n");
        exit(1);
    }
    for(i = 0; i < DUTNO; i++) {
        if((dest[i] & 0x80000000) != 0)
            AT_printf("FAIL PIN:P33 DUT:%3d\n", i + 1);
    }

    return(0);
}
```

2. 4.80 AT_read_cumfuncm

Function: This library function reads failure data that is accumulated for each pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cumfuncm
(
    int      request, /* kind of STRB reading fail data */
    int      pincnt,  /* number of pin to specify */
    uint16_t *pinno,  /* array of pin kind and pin number */
    int      destcnt, /* number of data to read */
    uint32_t *dest    /* where to put data */
);
```

Description

This function reads failure data that is accumulated for the specified pin numbers. More than one pin number can be specified.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to n of READ CUMFUNCn.

In the argument "pincnt", specify the type and the number of pins whose failure data is read.

In the argument "pinno", specify a pointer to an area storing the type and numbers of the pins whose cumulative failure data is read. The pointer must be set to a uint16_t-type array having at least as many elements as specified in the argument "pincnt". Use one of the following macros to specify the type and number of each target pin.

Macro	Type of Pin
AT_P(uint16_t pin)	I/O pin
AT_PC(uint16_t pin)	Comparator pin
AT_PD(uint16_t pin)	Driver pin

Specify a uint16_t-type pin number in macro argument "pin".

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ CUMFUNC statement.

➔ For more information on using this function, refer to the description of the READ CUMFUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Related Items

[2. 4. 79 "AT_read_cumfunc" on page 2-151](#) and [2. 4. 81 "AT_read_cumfunc_block" on page 2-156](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO**EINVAL**

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Pins 33 to 36 in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Accumulated Failure Data Was Found at Each Pin

```
#define PINNO 4
#define DUTNO 128

int foo(void)
{
    uint16_t pinno[PINNO];
    uint32_t dest[DUTNO];
    int i, j;

    for(i = 0; i < PINNO; i++)
        pinno[i] = AT_P(i + 33);

    if(AT_read_cumfuncm(1, PINNO, pinno, DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_cumfuncm()\n");
        exit(1);
    }
    for(j = 0; j < PINNO; j++) {
        for(i = 0; i < DUTNO; i++) {
            if((dest[i] & (0x80000000 >> j)) != 0)
                AT_printf("FAIL PIN:P%d DUT:%3d\n", j + 33, i + 1);
        }
    }

    return(0);
}
```

2. 4.81 AT_read_cumfunc_block

Function: This library function reads failure data that is accumulated for each pin.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cumfunc_block
(
    int      request, /* kind of STRB reading fail data */
    int      blkno,   /* block number */
    int      destcnt, /* number of data to read */
    uint32_t *dest     /* where to put data */
);
```

Description

This function reads failure data that is accumulated for pins of the specified block number.

In the argument "request", specify the type of STRB used for reading failure data. It corresponds to the value assigned to n of READ CUMFUNCn.

In the argument "blkno", specify the block number whose cumulative failure data is read.

In the argument "destcnt", specify the number of items of data to be read.

In the argument "dest", specify the pointer that is set to a uint32_t-type array, which has at least as many elements as specified in the argument "destcnt", by using the pointer where read data will be stored. If read data overflows, or if, in other words, the number of "destcnt" elements is exceeded, excess data is not saved.

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ CUMFUNC statement.

➔ For more information on using this function, refer to the description of the READ CUMFUNC statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

Related Items

[2. 4. 79 "AT_read_cumfunc" on page 2-151](#) and [2. 4. 80 "AT_read_cumfuncm" on page 2-153](#)

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) Example of Reading the Logical Sum of All STRB Failure Data of Block Number 1 (pins 33 to 64) in a 128 Device Parallel Test, and Then Displaying the Numbers of the DUTs Where Accumulated Failure Data Was Found at Each Pin

```
#define PINNO    32
#define DUTNO    128

int foo(void)
{
    uint32_t  dest[DUTNO];
    int       i, j;

    if(AT_read_cumfunc_block(1, 1, DUTNO, dest) != 0) {
        AT_printf("Error:AT_read_cumfunc_block()\n");
        exit(1);
    }
    for(j = 0; j < PINNO; j++) {
        for(i = 0; i < DUTNO; i++) {
            if((dest[i] & (0x80000000 >> j)) != 0)
                AT_printf("FAIL  PIN:P%d DUT:%3d\n", j + 33, i + 1);
        }
    }

    return(0);
}
```

2. 4.82 AT_read_ercode

Function: This library function reads error codes.

Syntax

```
#include "AT/hybrid.h"

int AT_read_ercode
(
    int32_t *vp      /* ATL error code to read */
);
```

Description

This function reads the error code of the preceding error in the ATL program.

If no error has occurred, the read value is -1.

In the argument "vp", specify the address of the variable in which the error code is stored.

➔ For more information, refer to the description of the READ ERCODE statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

2. 4.83 AT_write_afm

Function: This library function writes AFM or PM data.

Syntax

```
#include "AT/hybrid.h"

int AT_write_afm
(
    int            mode,          /* mode to write */
    struct fm_rwparam *args,      /* conditions to write */
    int            ndata,         /* number of data to write */
    uint32_t       *dest          /* pointer to write data */
);
```

Description

This function writes the data specified in the argument "dest" in address fail memory (AFM) and pattern memory (PM) according to the definitions of arguments "mode" and "args".

The arguments are explained separately below.

The meaning of each argument, the data format, and notes in use are the same as those of the AFM memory statement.

➔ Refer to the description of the AFM MEMORY statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#) when using this function.

◆ Mode

This argument specifies the mode in which data is written to AFM or PM.

Use one of the pre-defined macros listed below to set the write mode and specify the type of memory, AFM or PM, to which to write data.

Two write modes can be selected: normal mode and high-speed mode. The high-speed mode is not available if the target bit structure consists of 2, 9, 16, or 18 bits. The data write format is different between the normal and high-speed modes.

Macro for Normal Mode	Description
FM_FT	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.

Macro for Normal Mode	Description
FM_FOSB(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AF	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_MASK	Writes data to the AFM specified for the MASK mode (FMX>MASK) in the AFM ACTION statement.
FM_MASKSB1	Writes data to the AFM specified for the MASKSB1 mode (FMX>MASKSB1 or FMX>MASKSB(#1)) in the AFM ACTION statement.
FM_MASKSB2	Writes data to the AFM specified for the MASKSB2 mode (FMX>MASKSB2 or FMX>MASKSB(#2)) in the AFM ACTION statement.
FM_MASKSB129	Writes data to the AFM specified for the MASKSB129 mode (FMX>MASKSB129 or FMX>MASKSB(#4)) in the AFM ACTION statement.

Macro for Normal Mode	Description
FM_MASKSB130	Writes data to the AFM specified for the MASKSB130 mode (FMX>MASKSB130 or FMX>MASKSB(#8)) in the AFM ACTION statement.
FM_MASKSB(int v)	Writes data to the AFM specified for the MASKSB(v) mode (FMX>MASKSB(v)) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_EXP	Writes data to the PM specified for the EXP mode (PM>EXP) in the AFM ACTION statement.

Macro for High-speed Mode	Description
FM_FTH	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1H	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2H	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129H	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130H	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSBH(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZH	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1H	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2H	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129H	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130H	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSBH(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FOH	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1H	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2H	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129H	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130H	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.
FM_FOSBH(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

Macro for High-speed Mode	Description
FM_AFH	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1H	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2H	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129H	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130H	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSBH(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BFH	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1H	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2H	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129H	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130H	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSBH(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_MASKH	Writes data to the AFM specified for the MASK mode (FMX>MASK) in the AFM ACTION statement.
FM_MASKSB1H	Writes data to the AFM specified for the MASKSB1 mode (FMX>MASKSB1 or FMX>MASKSB(#1)) in the AFM ACTION statement.
FM_MASKSB2H	Writes data to the AFM specified for the MASKSB2 mode (FMX>MASKSB2 or FMX>MASKSB(#2)) in the AFM ACTION statement.
FM_MASKSB129H	Writes data to the AFM specified for the MASKSB129 mode (FMX>MASKSB129 or FMX>MASKSB(#4)) in the AFM ACTION statement.
FM_MASKSB130H	Writes data to the AFM specified for the MASKSB130 mode (FMX>MASKSB130 or FMX>MASKSB(#8)) in the AFM ACTION statement.

Macro for High-speed Mode	Description
FM_MASKSBH(int v)	Writes data to the AFM specified for the MASKSB(v) mode (FMX>MASKSB(v)) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_EXPH	Writes data to the PM specified for the EXP mode (PM>EXP) in the AFM ACTION statement.

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The fm_rwparam structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Specifies the direction in which data is written in the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Writes data in the X direction first, then Y direction. FM_APLINK_YX: Writes data in the Y direction first, then X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) on the X-axis along which data is written. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0x3ffff$ within the range specified in the AFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) on the Y-axis along which data is written. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0x3ffff$ within the range specified in the AFM ADDRESS statement.
int dutno	Specifies the number of the target DUT for writing data. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program, or the following pre-defined macro. FM_DUTALL: All DUTs are targeted. However, if FM_EXP or FM_EXPH is specified in the argument "mode", "dutno" is invalid.
int bitno	Specifies the target bits for writing data. Specify one of the following pre-defined macros. FM_BLK1: Data is read from BLK1. FM_BLK2: Data is read from BLK2. FM_BLK3: Data is written to BLK3. FM_BLK4: Data is read from BLK4. FM_BLK5: Data is written to BLK5. FM_BLK6: Data is read from BLK6. FM_BLK7: Data is written to BLK7. FM_BLK8: Data is written to BLK8.
int option[2]	Specifies 0

◆ Ndata

This argument specifies the number of items of data to be written.

◆ Dest

This argument specifies the pointer to the variable to which to write data.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Writing 16 Items of Data for DUT1, BLK1 in the X Direction First, Then Y Direction, in Which the Data Is Written to the Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the AFM Specified for the FT Mode

```
int foo(void)
{
    int      ret;
    uint32_t  dest[16];
    struct fm_rwparam  args = {0};

    dest[0]   = 0x8000;
    dest[1]   = 0x4000;
    dest[2]   = 0x2000;
    dest[3]   = 0x1000;
    dest[4]   = 0x0800;
    dest[5]   = 0x0400;
    dest[6]   = 0x0200;
    dest[7]   = 0x0100;
    dest[8]   = 0x0080;
    dest[9]   = 0x0040;
    dest[10]  = 0x0020;
    dest[11]  = 0x0010;
    dest[12]  = 0x0008;
    dest[13]  = 0x0004;
    dest[14]  = 0x0002;
    dest[15]  = 0x0001;
    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = FM_BLK1;

    ret = AT_write_afm(FM_FT, &args, 16, dest);

    if ( ret == 0 ) {
        AT_printf("write ok!\n"); /* success */
    }
    else {
        AT_printf("write ng!\n"); /* failure */
    }

    return(0);
}
```

2. 4.84 AT_read_afm_confcnt

Function: This library function reads the number of groups into which DUTs are divided at AFM allocation and the number of DUTs per group.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_confcnt
(
    int      modcnt, /* number of afm action mode */
    int      pgssel, /* condition of afm pgssel */
    int      cnttyp, /* kind of data to read */
    uint32_t *dest   /* where to put data */
);
```

Description

This function reads the number of groups into which DUTs are divided when allocated in address fail memory (AFM) under the conditions specified in arguments "modcnt" and "pgssel". It can also read the number of DUTs per group.

The arguments are explained separately below.

The meaning of each argument and notes in use are the same as those of the READ AFM CONFIG Count statement.

➔ For more information on using this function, refer to the description of the READ AFM CONFIG COUNT statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

◆ Modcnt

This argument specifies the number of modes (modes except PM>EXP, PM>DBM(m), BBM>MASK, BBM>MASK, and CLR(w)>MASK) to be allocated to AFM using the AFM ACTION statement.

Specify 1 or 2.

◆ Pgsel

This argument specifies PG in the AFM PGSEL statement.¹ Specify a value from 1 to 4.

Value Type	Description
1	The AFM PGSEL statement specifies MAIN. MAIN is not in two-pass double-speed mode or double-speed mode.
2	The AFM PGSEL statement specifies MAIN or SUB is not in the double-speed mode.
3	The AFM PGSEL statement specifies MAIN. MAIN is in double-speed mode.
4	The AFM PGSEL statement specifies MAIN or SUB and is in the double-speed mode.

◆ Cnttyp

This argument specifies the type of data to be read. Specify 1 or 2.

Value Type	Description
1	Reads the number of groups into which DUTs are divided.
2	Reads the number of DUTs per group.

◆ Dest

This argument specifies a pointer to the variable from which data to be read is stored.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Reading the Number of Groups into Which DUTs Are Divided When They Are Allocated in AFM (AFM ACTION = FT>FMX, AFM PGSEL = MAIN, SUB)

```
int foo(void)
{
    int      ret;
    uint32_t dest;

    ret = AT_read_afm_confcnt(1, 2, 1, &dest);

    if ( ret == 0 ) {
        AT_printf("group count = %d\n", dest); /* success */
    }
    else {
        AT_printf("read error!\n");           /* failure */
    }

    return(0);
}
```

2. 4.85 AT_read_afm_confdat

Function: This library function reads the numbers of DUTs to be allocated in AFM.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_confdat
(
    int      modcnt, /* number of afm action mode */
    int      pgssel, /* condition of afm pgssel */
    int      confno, /* group number to read */
    int      dattyp, /* type of data to read */
    int      ndata,  /* number of array element */
    uint32_t *dest   /* where to put data */
);
```

Description

This function reads the numbers of DUTs to be allocated in address fail memory (AFM) under the conditions specified in the arguments "modcnt" and "pgssel".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM CONFIG Data statement.

➔ Refer to the description of the READ AFM CONFIG DATA statement in [2. 4. 84 "AT_read_afm_confcnt" on page 2-169](#), [2. 4. 86 "AT_read_afm_dutselect" on page 2-175](#), and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

◆ Modcnt

This argument specifies the number of modes (modes except PM>EXP, PM>DBM(m), BBM>MASK, BBM>MASK, and CLR(w)>MASK) to be allocated to AFM using the AFM ACTION statement.

Specify 1 or 2.

◆ Pgsel

This argument specifies PG in the AFM PGSEL statement.¹ Specify a value from 1 to 4.

Value Type	Description
1	The AFM PGSEL statement specifies MAIN. MAIN is not in two-pass double-speed mode or double-speed mode.
2	The AFM PGSEL statement specifies MAIN or SUB is not in the double-speed mode.
3	The AFM PGSEL statement specifies MAIN. MAIN is in double-speed mode.
4	The AFM PGSEL statement specifies MAIN or SUB and is in the double-speed mode.

◆ Confno

This argument specifies the number of the group for which information is read.

Specify a number from 1 to the number of groups formed.

AT_read_afm_confcnt() can be used to read the number of groups formed.

◆ Dattyp

This argument specifies the data read format.¹ Specify a value from 1 to 3.

Value Type	Description
1	Reads DUT numbers.
2	Reads DUT numbers according to bit correspondence.
3	Reads the numbers of target DUTs, according to bit correspondence.

◆ Ndata

This argument specifies the number of elements of the variable from which data is read.

◆ Argument *dest

This argument specifies a pointer to the variable from which data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) The Following Sample Code Reads the DUT Numbers of Group 1 if DUTs Are Allocated in AFM with AFM ACTION=FT>FMX, AFM PGSEL=MAIN, and SUB.

```
int foo(void)
{
    int      ret;
    uint32_t dest[512];

    ret = AT_read_afm_confdat(1, 2, 1, 1, 512, &dest);
    if ( ret == 0 ) {
        AT_printf("read ok!\n");          /* success */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }
    return(0);
}
```


2. 4.86 AT_read_afm_dutselect

Function: This library function reads the numbers of DUTs to be allocated in AFM.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_dutselect
(
    int      dattyp, /* type of data to read */
    int      ndata,  /* number of array element */
    uint32_t *dest   /* where to put data */
);
```

Description

This function reads the numbers of DUTs currently allocated in address fail memory (AFM).

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM DUTSELECT statement.

➔ For more information, refer to the description of the READ AFM DUTSELECT statement in [2. 4. 85 "AT_read_afm_confdat" on page 2-172](#) and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

◆ Dattyp

This argument specifies the data read format. Specify a value from 1 to 3.

Value Type	Description
1	Reads DUT numbers.
2	Reads DUT numbers according to bit correspondence.
3	Reads the numbers of target DUTs, according to bit correspondence.

◆ Ndata

This argument specifies the number of elements of the variable from which data is read.

◆ Dest

This argument specifies a pointer to the variable from which data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading the Numbers of DUTs Currently Allocated in AFM

```
int foo(void)
{
    int          ret;
    uint32_t     dest[512];

    ret = AT_read_afm_dutselect(1, 512, &dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");           /* success */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }

    return(0);
}
```

2. 4.87 AT_read_afm_memory

Function: This library function reads AFM or PM data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_memory
(
    int            mode,          /* mode to read */
    struct fm_rwparam *args,      /* conditions to read */
    int            ndata,         /* number of data to read */
    uint32_t       *dest         /* where to put data */
);
```

Description

This function reads data from the address fail memory (AFM) and pattern memory (PM) specified in the arguments "mode" and "args" and writes it to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM Memory statement.

➔ For more information on using this function, refer to the description of the READ AFM MEMORY statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

◆ Mode

This argument specifies the mode in which data is read from AFM or PM.

Use a pre-defined macro to set the read mode and specify the type of memory, AFM or PM, from which data is read.

Two read modes can be selected: normal mode and high-speed mode. The high-speed mode is not available if the target bit structure consists of 2, 9, 16, or 18 bits.

The pre-defined macros that can be specified are the same as those for AT_write_afm().

➔ For more information, refer to [2. 4. 83 "AT_write_afm" on page 2-159](#).

The data read format is different between the normal and high-speed modes.

The function can also read AFM data of the two different modes specified in the AFM ACTION statement performing an OR operation on the data of the different modes.

To do so, specify one of the following macros.

Specify normal mode macros in the arguments "mode1" and "mode2" of each macro.

However, FM_MASK, FM_MASKSB1, FM_MASKSB2, FM_MASKSB129, FM_MASKSB130, FM_MASKSB(v), and FM_EXP cannot be specified.

FM_MASKSB(v) and FM_EXP cannot be specified.

Macro for OR Operation and Reading	Description
FM_OR(mode1,mode2)	Executes an OR operation on AFM data of the modes corresponding to the macros specified for "mode1" and "mode2", and reads the result of the OR operation in normal mode.
FM_ORH(mode1,mode2)	Executes an OR operation on AFM data of the modes corresponding to the macros specified for "mode1" and "mode2", and reads the result of the OR operation at high-speed mode.

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The fm_rwparam structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Specifies the direction in which data is read from the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Reads data in the X direction first, then the Y direction. FM_APLINK_YX: Reads data in the Y direction first, then the X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) of the X-axis along which data is read. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) of the Y-axis along which data is read. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
int dutno	Specifies the number of the target DUT for reading data. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program. However, if FM_EXP or FM_EXPH is specified in the argument "mode", "dutno" is invalid.
int bitno	Specifies the target bits for reading data. Specify a value from 0 to 143 or the following pre-defined macro. FM_BLK1: Data is read from BLK1. FM_BLK2: Data is read from BLK2. FM_BLK3: Data is written to BLK3. FM_BLK4: Data is read from BLK4. FM_BLK5: Data is written to BLK5. FM_BLK6: Data is read from BLK6. FM_BLK7: Data is written to BLK7. FM_BLK8: Data is written to BLK8. FM_ORDATA: An OR operation is executed on all bits.
int option[2]	Specifies 0

◆ Ndata

This argument specifies the number of items of data to be read.

◆ Dest

This argument specifies a pointer to the variable from which data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Reading 16 Items of Data for DUT1 and BLK1 in the X-Y Direction in Which the Data Is Read from the Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the AFM Specified for the FT Mode

```
int foo(void)
{
    int          ret;
    uint32_t      dest[16];
    struct fm_rwparam  args = {0};

    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = FM_BLK1;

    ret = AT_read_afm_memory(FM_FT, &args, 16, dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");          /* success */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }

    return(0);
}
```

2. 4.88 AT_read_afm_failaddr

Function: This library function reads AFM fail addresses.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_failaddr
(
    int            mode,          /* mode to read */
    struct fm_rwparam *args,      /* conditions to read */
    int            skipcnt,      /* number of skip fail */
    int            ndata,         /* number of fail address to read*/
    uint32_t       *dest         /* where to put data */
);
```

Description

This function searches the address fail memory (AFM) specified in the arguments "mode" and "args" for fail addresses, and it writes the fail addresses to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM Fail Address Search statement.

➔ Refer to the description of the READ AFM FAIL ADDRESS SEARCH statement in [2. 4. 83 "AT_write_afm" on page 2-159](#) and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

◆ Mode

This argument specifies the mode in which the function searches the AFM for fail addresses and reads them.

Use one of the pre-defined macros listed below to set the search mode and search the target AFM.

Two search modes can be selected: normal search mode and advanced search mode. The normal search mode is not available if one of X12 to X17 or Y12 to Y17 is specified in the AFM ADDRESS statement.

The address data read format is different between the normal and advanced search modes.

Macros for Normal Search Mode	Description
FM_FT	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.

Macros for Normal Search Mode	Description
FM_FOSB(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AF	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

Macros for Advanced search Mode	Description
FM_FTA	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1A	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2A	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129A	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130A	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSBA(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZA	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1A	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2A	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129A	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130A	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSBA(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FOA	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1A	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2A	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129A	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130A	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.

Macros for Advanced search Mode	Description
FM_FOSBA(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AFA	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1A	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2A	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129A	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130A	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSBA(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BFA	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1A	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2A	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129A	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130A	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSBA(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

The macros listed below can be used to search AFM data of the two different modes specified in the AFM ACTION statement while combining them in an OR operation and to read fail addresses.

To do so, specify one of the following macros.

Specify normal search mode macros in the arguments "mode1" and "mode2" of each macro.

Macro for OR Operation and Reading	Description
FM_OR(mode1, mode2)	Searches AFM data of the modes corresponding to the macros specified in "mode1" and "mode2", combines them in an OR operation, and reads the results in normal search mode.
FM_ORA(mode1, mode2)	Searches AFM data of the modes corresponding to the macros specified in "mode1" and "mode2", combines them in an OR operation, and reads the results in advanced search mode.

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The `fm_rwparam` structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Specifies the direction in which to search for fail addresses in the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Searches data in the X direction first, then the Y direction. FM_APLINK_YX: Searches data in the Y direction first, then the X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) of the X-axis along which to search for fail addresses. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) of the Y-axis along which to search for fail addresses. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
int dutno	Specifies the number of the target DUT when searching for fail addresses. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program.
int bitno	Specifies the target bits for searching for fail addresses. Specify a value from 0 to 143 or the following pre-defined macro. FM_ORDATA: An OR operation is executed on all bits.
int option[2]	Specifies 0

◆ Skipcnt

This argument specifies the number of fail addresses to be skipped.
Specify a value from 0 to 67108864.

◆ Ndata

This argument specifies the number of fail addresses to be read.

◆ Dest

This argument specifies a pointer to the variable into which fail addresses are read.

The variable dest must be a unit32_t-type array with at least as many elements as the argument "ndata" in Normal Search mode, or at least three times as many elements as the argument "ndata" in Advanced Search mode.

Return Value

This function returns 0 if the read fail address is the argument "ndata" or greater. It returns 1 if the read fail address is less than the argument "ndata".

If the function fails, the error information is assigned to "errno" and returns -1.

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Searching for and Reading Four Fail Addresses for DUT1 Bit 0 in the X Direction First, Then Y Direction, in Which the Searching and Reading Are Done in an Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the AFM Specified for the FT Mode

```
int foo(void)
{
    int          ret;
    uint32_t      dest[4];
    struct fm_rwparam  args = {0};

    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = 0;

    ret = AT_read_afm_failaddr(FM_FT, &args, 0, 4, dest);

    if ( ret == 0 ) {
        AT_printf("fail address >= 4\n"); /* The number of fail addresses is 4 or
                                           less */
    }
    else if ( ret == 1 ) {
        AT_printf("fail address < 4\n"); /* The number of fail addresses is less
                                           than 4 */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }
    return(0);
}
```


2. 4.89 AT_read_afm_failcnt

Function: This library function reads the number of AFM failures.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_failcnt
(
    int            mode,          /* mode to read */
    struct fm_rwparam *args,      /* conditions to read */
    int            ndata,         /* number of data to read */
    uint32_t       *dest         /* where to put data */
);
```

Description

This function counts fail cells in the address fail memory (AFM) specified in the arguments "mode" and "args", and writes the count to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM Fail Count statement.

➔ For more information, refer to the description of the READ AFM FAIL COUNT statement in [2. 4. 83 "AT_write_afm" on page 2-159](#), [2. 4. 88 "AT_read_afm_failaddr" on page 2-182](#), and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

◆ Mode

This argument specifies the mode in which the function counts fail cells in AFM and reads the count.

Use one of the following pre-defined macros to set the count mode and specify the target AFM.

Two count modes can be selected: normal count mode and advanced count mode. The count data read format is different between the normal and advanced count modes.

Macro for Normal Count Mode	Description
FM_FT	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.

Macro for Normal Count Mode	Description
FM_FOSB(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AF	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

Macros for Advanced Count Mode	Description
FM_FTA	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1A	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2A	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129A	Writes data to the AFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130A	Writes data to the AFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSBA(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZA	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1A	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2A	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129A	Writes data to the AFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130A	Writes data to the AFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSBA(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FOA	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1A	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2A	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129A	Writes data to the AFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130A	Writes data to the AFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.
FM_FOSBA(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

Macros for Advanced Count Mode	Description
FM_AFA	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1A	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2A	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129A	Writes data to the AFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130A	Writes data to the AFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSBA(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BFA	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1A	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2A	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129A	Writes data to the AFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130A	Writes data to the AFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSBA(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

The following macros can be used to count AFM data items of the two different modes specified in the AFM ACTION statement while combining them in an OR operation, and to read the number of failures.

To do so, specify one of the following macros.

Specify normal count mode macros in the arguments "mode1" and "mode2" of each macro.

Macro for OR Operation and Reading	Description
FM_OR(mode1, mode2)	Counts AFM data items of the modes corresponding to the macros specified in "mode1" and "mode2" while combining them in an OR operation, and reads the results in normal count mode.
FM_ORA(mode1, mode2)	Counts AFM data items of the modes corresponding to the macros specified in "mode1" and "mode2" while combining them in an OR operation, and reads the results in advanced count mode.

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The fm_rwparam structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Specifies the direction in which the function counts fail addresses in the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Counts data in the X direction first, then the Y direction. FM_APLINK_YX: Counts data in the Y direction first, then the X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) of the X-axis along which fail addresses are counted. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) of the Y-axis along which fail addresses are counted. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0x3fff$ within the range specified in the AFM ADDRESS statement.
int dutno	Specifies the number of the target DUT for counting failures. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program, or the following pre-defined macro. FM_DUTALL: All DUTs are targeted.
int bitno	Specifies the target bits for counting failures. Specify a value from 0 to 143 or the following pre-defined macro. FM_ORDATA: An OR operation is executed on all bits. FM_BITN_M: The applicable bits are in a range from the read start bit (option[0]) to the read end bit (option[1]) specified in the argument "option".
int option[2]	Specifies the read start and end bits if FM_BITN_M is specified in the argument "bitno". Specify a value from 1 to 143. option[0]: Specifies the read start bit. option[1]: Specifies the read end bit.

◆ Ndata

This argument specifies the number of elements of the variable from which data is read.

◆ Dest

This argument specifies a pointer to the variable into which the data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Counting Failures for DUT1, BIT0 in the X Direction First, Then Y Direction, in Which Counting Is Done in an Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the AFM Specified for the FT Mode and after That, the Count Is Read

```
int foo(void)
{
    int                ret;
    uint32_t           dest;
    struct fm_rwparam  args = {0};

    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = 0;

    ret = AT_read_afm_failcnt(FM_FT, &args, 1, &dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");           /* success */
    }
    else {
        AT_printf("read ng!\n");           /* failure */
    }

    return(0);
}
```

2. 4.90 AT_read_afm_mcnt

Function: This library function reads the number of AFM test failures.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_mcnt
(
    int          mode,          /* mode to read */
    struct fm_rwparam *args,    /* conditions to read */
    int          ndata,         /* number of data to read */
    uint32_t     *dest          /* where to put data */
);
```

Description

This function counts the number of failures in the address fail memory (AFM) specified in the arguments "mode" and "args", and reads the counts to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ AFM Failure Cycle Count statement.

➔ For more information, refer to the description of the READ AFM Failure Cycle Count statement in [2. 4. 83 "AT_write_afm" on page 2-159](#), [2. 4. 88 "AT_read_afm_failaddr" on page 2-182](#), and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

mode

This argument specifies the mode in which the function counts the number of failures in AFM and reads the count.

Use one of the following pre-defined macros to specify the target AFM.

Macro for Count Mode	Description
FM_FT	Writes data to the AFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the AFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the AFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the AFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the AFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the AFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the AFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the AFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the AFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the AFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the AFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB(int v)	Writes data to the AFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AF	Writes data to the AFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the AFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the AFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the AFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the AFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.

Macro for Count Mode	Description
FM_BFSB1	Writes data to the AFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the AFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the AFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

args

This argument specifies a pointer to the `fm_rwparam` structure.

The fm_rwparam structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Available due to the compatibility with other AT_read_afm functions but unavailable in this function.
uint32_t xsta uint32_t xspa	
uint32_t ysta uint32_t yspa	
int dutno	Specifies the number of the target DUT for counting failures. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program, or the following pre-defined macro. FM_DUTALL: All DUTs are targeted.
int bitno	Specifies the target bits for counting failures. Specify a value from 0 to 143 or the following pre-defined macro. FM_BITN_M: The applicable bits are in a range from the read start bit (option[0]) to the read end bit (option[1]) specified in the argument "option".
int option[2]	Specifies the read start and end bits if FM_BITN_M is specified in the argument "bitno". Specify from 1 to 143. option[0]: Specifies the read start bit. option[1]: Specifies the read end bit.

ndata

This argument specifies the number of elements of the variable from which data is read.

dest

This argument specifies a pointer to the variable into which the data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno."

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Specifying ALL to AFM DUT and BIT0-3 to the Target Bits, and Reading the Number of Failures As the Results of a 16 Device Parallel Test (Execute this function immediately after the test.)

```
int foo(void)
{
    int          ret;
    uint32_t      dest[64];
    struct fm_rwparam  args = {0};

    args.dutno    = FM_DUTALL;
    args.bitno    = FM_BITN_M;
    args.option[0] = 0;
    args.option[1] = 3;

    ret = AT_read_afm_failcnt(FM_FT, &args, 64, &dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");          /* success */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }

    return(0);
}
```

The dest array stores the following results:

```
dest[0]    <- The number of failures for BIT0 of DUT1
dest[1]    <- The number of failures for BIT1 of DUT1
dest[2]    <- The number of failures for BIT2 of DUT1
dest[3]    <- The number of failures for BIT3 of DUT1
dest[4]    <- The number of failures for BIT0 of DUT2
dest[5]    <- The number of failures for BIT1 of DUT2
dest[6]    <- The number of failures for BIT2 of DUT2
:
:
dest[59]   <- The number of failures for BIT3 of DUT15
dest[60]   <- The number of failures for BIT0 of DUT16
dest[61]   <- The number of failures for BIT1 of DUT16
dest[62]   <- The number of failures for BIT2 of DUT16
dest[63]   <- The number of failures for BIT3 of DUT16
```

2. 4.91 AT_read_cfm_memory

Function: This library function reads CFM data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cfm_memory
(
    int            mode,          /* mode to read */
    struct fm_rwparam *args,      /* conditions to read */
    int            ndata,         /* number of data to read */
    uint32_t       *dest          /* where to put data */
);
```

Description

This function reads data from the compact fail memory (CFM) specified in the arguments "mode" and "args", and it writes the data to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ CFM Memory statement.

➔ For more information on using this function, refer to the description of the READ CFM MEMORY statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

◆ Mode

This argument specifies the mode in which the function reads data from CFM.

Use one of the following pre-defined macros to specify the CFM from which data is read.

Macro	Description
FM_FT	Writes data to the CFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the CFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the CFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129	Writes data to the CFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130	Writes data to the CFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the CFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the CFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the CFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the CFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129	Writes data to the CFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130	Writes data to the CFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the CFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the CFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the CFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the CFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129	Writes data to the CFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130	Writes data to the CFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.
FM_FOSB(int v)	Writes data to the CFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

Macro	Description
FM_AF	Writes data to the CFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the CFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the CFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129	Writes data to the CFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130	Writes data to the CFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the CFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the CFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1	Writes data to the CFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the CFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129	Writes data to the CFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130	Writes data to the CFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the CFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_MASK	Writes data to the AFM specified for the MASK mode (FMX>MASK) in the AFM ACTION statement.
FM_MASKSB1	Writes data to the CFM specified for the MASKSB1 mode (FMX>MASKSB1 or FMX>MASKSB(#1)) in the AFM ACTION statement.
FM_MASKSB2	Writes data to the CFM specified for the MASKSB2 mode (FMX>MASKSB2 or FMX>MASKSB(#2)) in the AFM ACTION statement.
FM_MASKSB129	Writes data to the CFM specified for the MASKSB129 mode (FMX>MASKSB129 or FMX>MASKSB(#4)) in the AFM ACTION statement.
FM_MASKSB130	Writes data to the CFM specified for the MASKSB130 mode (FMX>MASKSB130 or FMX>MASKSB(#8)) in the AFM ACTION statement.

Macro	Description
FM_MASKSB(int v)	Writes data to the CFM specified for the MASKSB(v) mode (FMX>MASKSB(v)) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The fm_rwparam structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int aplink	Specifies the direction in which data is read in the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Reads data in the X direction first, then the Y direction. FM_APLINK_YX: Reads data in the Y direction first, then the X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) of the X-axis along which data is read. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0xffff$ within the range specified in the CFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) of the Y-axis along which data is read. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0xffff$ within the range specified in the CFM ADDRESS statement.
int dutno	Specifies the number of the target DUT for reading data. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program.
int bitno	Specifies the target bits for reading data. Specify a value from 0 to 143 or the following pre-defined macro. FM_BLK1: Data is read from BLK1. FM_BLK2: Data is read from BLK2. FM_BLK3: Data is written to BLK3. FM_BLK4: Data is read from BLK4. FM_BLK5: Data is written to BLK5. FM_BLK6: Data is read from BLK6. FM_BLK7: Data is written to BLK7. FM_BLK8: Data is written to BLK8. FM_ORDATA: An OR operation is executed on all bits.
int option[2]	Specifies 0

◆ Ndata

This argument specifies the number of items of data to be read.

◆ Dest

This argument specifies a pointer to the variable from which data to be read is stored.

The argument "dest" must be the uint32_t-type array having at least as many elements as specified in the argument "ndata".

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Reading 16 Items of Data for DUT1 and BLK1 in the X-Y Direction in Which the Data Is Read from the Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the CFM Specified for the FT Mode

```
int foo(void)
{
    int      ret;
    uint32_t      dest[16];
    struct fm_rwparam  args = {0};

    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = FM_BLK1;

    ret = AT_read_cfm_memory(FM_FT, &args, 16, dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");          /* success */
    }
    else {
        AT_printf("read ng!\n");          /* failure */
    }

    return(0);
}
```

2. 4.92 AT_read_dbm

Function: This library function reads DBM data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_dbm
(
    int      unit,      /* unit number */
    int      kind,      /* kind number */
    uint32_t saddr,     /* start address */
    uint32_t taddr,     /* stop address */
    int      blkno,     /* block number */
    int      nelem,     /* number of data to read */
    uint32_t *vp        /* buffer of data to read */
);
```

Description

This function reads data from data buffer memory (DBM).

Specify the unit number in the argument "unit". Specify any of DBM_UNIT1, DBM_UNIT2, DBM_UNIT3, and DBM_UNIT4 defined in the header file. Specify DBM data type in the argument "kind". Specify any of DBM_PAT, DBM_CPE, and DBM_DRE defined in the header file. Specify the start address and the end address in the arguments "saddr" and "taddr", respectively. Specify a value of 0 to the maximum. The maximum address value varies depending on the system. Specify "saddr" and "taddr" so that they satisfy $saddr \leq taddr$. In the argument "nelem", specify the number of items of data to be read. Specify "nelem" so that it satisfies $nelem = taddr - saddr + 1$. If they are not equal, as many items as the smaller number are read. In the argument "vp", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ DBM statement.

➔ For more information on using this function, refer to the description of the READ DBM statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading 16 Items of BLK1 DRE Pattern Data of UNIT1 from Addresses 0 to 15

```
int sample(void)
{
    uint32_t      vp[16];

    if(AT_read_dbm(DBM_UNIT1,DBM_DRE,0,15,DBM_BLK1,16,vp) == -1){
        AT_printf("Error: AT_read_dbm()\n");
        exit(1);
    }

    return(0);
}
```


2. 4.93 AT_read_cfm_failcnt

Function: This library function reads the number of CFM failures.

Syntax

```
#include "AT/hybrid.h"

int AT_read_afm_failcnt
(
    int            mode,          /* mode to read */
    struct fm_rwparam *args,      /* conditions to read */
    uint32_t       *dest         /* where to put data */
);
```

Description

This function counts fail cells in the compact fail memory (CFM) specified in the arguments "mode" and "args", and it writes the count to the argument "dest".

The arguments are explained separately below.

The each argument meaning, data format, and notes in use are the same as those of the READ CFM Fail Count statement.

➔ For more information, refer to the description of the READ CFM FAIL COUNT statement in [2. 4. 91 "AT_read_cfm_memory" on page 2-206](#) and *ATL-51 Test Plan Program Reference Manual(No. : 8431498)*.

◆ Argument "mode"

This argument specifies the mode in which the function counts fail cells in CFM and reads the count.

Use one of the following pre-defined macros to set the count mode and specify the target CFM.

Macro	Description
FM_FT	Writes data to the CFM specified for the FT mode (FT>FMX) in the AFM ACTION statement.
FM_FTSB1	Writes data to the CFM specified for the FTSB1 mode (FTSB1>FMX or FTSB(#1)>FMX) in the AFM ACTION statement.
FM_FTSB2	Writes data to the CFM specified for the FTSB2 mode (FTSB2>FMX or FTSB(#2)>FMX) in the AFM ACTION statement.
FM_FTSB129	Writes data to the CFM specified for the FTSB129 mode (FTSB129>FMX or FTSB(#4)>FMX) in the AFM ACTION statement.
FM_FTSB130	Writes data to the CFM specified for the FTSB130 mode (FTSB130>FMX or FTSB(#8)>FMX) in the AFM ACTION statement.
FM_FTSB(int v)	Writes data to the CFM specified for the FTSB(v) mode (FTSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FZ	Writes data to the CFM specified for the FZ mode (FZ>FMX) in the AFM ACTION statement.
FM_FZSB1	Writes data to the CFM specified for the FZSB1 mode (FZSB1>FMX or FZSB(#1)>FMX) in the AFM ACTION statement.
FM_FZSB2	Writes data to the CFM specified for the FZSB2 mode (FZSB2>FMX or FZSB(#2)>FMX) in the AFM ACTION statement.
FM_FZSB129	Writes data to the CFM specified for the FZSB129 mode (FZSB129>FMX or FZSB(#4)>FMX) in the AFM ACTION statement.
FM_FZSB130	Writes data to the CFM specified for the FZSB130 mode (FZSB130>FMX or FZSB(#8)>FMX) in the AFM ACTION statement.
FM_FZSB(int v)	Writes data to the CFM specified for the FZSB(v) mode (FZSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_FO	Writes data to the CFM specified for the FO mode (FO>FMX) in the AFM ACTION statement.
FM_FOSB1	Writes data to the CFM specified for the FOSB1 mode (FOSB1>FMX or FOSB(#1)>FMX) in the AFM ACTION statement.
FM_FOSB2	Writes data to the CFM specified for the FOSB2 mode (FOSB2>FMX or FOSB(#2)>FMX) in the AFM ACTION statement.
FM_FOSB129	Writes data to the CFM specified for the FOSB129 mode (FOSB129>FMX or FOSB(#4)>FMX) in the AFM ACTION statement.
FM_FOSB130	Writes data to the CFM specified for the FOSB130 mode (FOSB130>FMX or FOSB(#8)>FMX) in the AFM ACTION statement.

Macro	Description
FM_FOSB(int v)	Writes data to the CFM specified for the FOSB(v) mode (FOSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_AF	Writes data to the CFM specified for the AF mode (AF>FMX) in the AFM ACTION statement.
FM_AFSB1	Writes data to the CFM specified for the AFSB1 mode (AFSB1>FMX or AFSB(#1)>FMX) in the AFM ACTION statement.
FM_AFSB2	Writes data to the CFM specified for the AFSB2 mode (AFSB2>FMX or AFSB(#2)>FMX) in the AFM ACTION statement.
FM_AFSB129	Writes data to the CFM specified for the AFSB129 mode (AFSB129>FMX or AFSB(#4)>FMX) in the AFM ACTION statement.
FM_AFSB130	Writes data to the CFM specified for the AFSB130 mode (AFSB130>FMX or AFSB(#8)>FMX) in the AFM ACTION statement.
FM_AFSB(int v)	Writes data to the CFM specified for the AFSB(v) mode (AFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".
FM_BF	Writes data to the CFM specified for the BF mode (BF>FMX) in the AFM ACTION statement.
FM_BFSB1	Writes data to the CFM specified for the BFSB1 mode (BFSB1>FMX or BFSB(#1)>FMX) in the AFM ACTION statement.
FM_BFSB2	Writes data to the CFM specified for the BFSB2 mode (BFSB2>FMX or BFSB(#2)>FMX) in the AFM ACTION statement.
FM_BFSB129	Writes data to the CFM specified for the BFSB129 mode (BFSB129>FMX or BFSB(#4)>FMX) in the AFM ACTION statement.
FM_BFSB130	Writes data to the CFM specified for the BFSB130 mode (BFSB130>FMX or BFSB(#8)>FMX) in the AFM ACTION statement.
FM_BFSB(int v)	Writes data to the CFM specified for the BFSB(v) mode (BFSB(v)>FMX) in the AFM ACTION statement. Specify a value from 0x1 to 0xf in the argument "v".

◆ Args

This argument specifies a pointer to the fm_rwparam structure.

The `fm_rwparam` structure is defined as shown below. Specify data for each member.

```
struct fm_rwparam{
    struct fm_addr adrs;
    int dutno;
    int bitno;
    int option[2];
}
struct fm_addr{
    int aplink;
    uint32_t xsta;
    uint32_t xspa;
    uint32_t ysta;
    uint32_t yspa;
}
```

Member	Description
int ap_link	Specifies the direction in which the function counts fail addresses in the address range specified by "xsta", "xspa", "ysta", and "yspa". Specify one of the following pre-defined macros. FM_APLINK_XY: Counts data in the X direction first, then the Y direction. FM_APLINK_YX: Counts data in the Y direction first, then the X direction.
uint32_t xsta uint32_t xspa	Specify the start address (xsta) and end address (xspa) of the X-axis along which fail addresses are counted. Specify a value satisfying $0x0 \leq xsta \leq xspa \leq 0xffff$ within the range specified in the CFM ADDRESS statement.
uint32_t ysta uint32_t yspa	Specify the start address (ysta) and end address (yspa) of the Y-axis along which fail addresses are counted. Specify a value satisfying $0x0 \leq ysta \leq yspa \leq 0xffff$ within the range specified in the CFM ADDRESS statement.
int dutno	Specifies the number of the target DUT for counting failures. For the DUT number, specify a number in a range from 1 to the maximum number defined for parallel measurement in the socket program.
int bitno	Specifies the target bits for counting failures. Specify a value from 0 to 143 or the following pre-defined macro. FM_ORDATA: An OR operation is executed on all bits.
int option[2]	Specifies 0

◆ Dest

This argument specifies a pointer to the variable into which the data to be read is stored.

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

- (1) Example of Counting Failures for DUT1, BIT0 in the X Direction First, Then Y Direction, in Which Counting Is Done in an Area Ranging from X Addresses 0 to 3 and Y Addresses 0 to 3 in the AFM Specified for the FT Mode and after That, the Count Is Read

```
int foo(void)
{
    int      ret;
    uint32_t dest;
    struct fm_rwparam args = {0};

    args.adrs.aplink = FM_APLINK_XY;
    args.adrs.xsta = 0x0;
    args.adrs.xspa = 0x3;
    args.adrs.ysta = 0x0;
    args.adrs.yspa = 0x3;
    args.dutno = 1;
    args.bitno = 0;

    ret = AT_read_cfm_failcnt(FM_FT, &args, &dest);

    if ( ret == 0 ) {
        AT_printf("read ok!\n");           /* success */
    }
    else {
        AT_printf("read ng!\n");           /* failure */
    }

    return(0);
}
```

2. 4.94 AT_read_cbm

Function: This library function reads CBM data.

Syntax

```
#include "AT/hybrid.h"

int AT_read_cbm
(
    int      unit,      /* unit number */
    int      kind,      /* kind number */
    uint32_t saddr,     /* start address */
    uint32_t taddr,     /* stop address */
    int      nelem,     /* number of data to read */
    uint32_t *vp        /* buffer of data to read */
);
```

Description

This function reads data from command buffer memory (CBM).

Specify the unit number in the argument "unit". Specify any of CBM_UNIT1, CBM_UNIT2, CBM_UNIT3, and CBM_UNIT4 defined in the header file. Specify CBM data type in the argument "kind". Specify either CBM_1 or CBM_2 defined in the header file. Specify the start address and the end address in the arguments "saddr" and "taddr", respectively. Specify a value from 0x0 to 0x7fff. Specify "saddr" and "taddr" so that they satisfy $saddr \leq taddr$. In the argument "nelem", specify the number of items of data to be read. Specify "nelem" so that it satisfies $nelem = taddr - saddr + 1$. If they are not equal, as many items as the smaller number are read. In the argument "vp", specify the pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument "nelem".

Note:

The each argument meaning, the read data format, and notes in use are the same as those of the READ CBM statement.

➔ For more information on using this function, refer to the description of the READ CBM statement in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to "errno".

ERRNO

EINVAL

An invalid argument is specified.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

Example

(1) Example of Reading 16 Items of UNIT1 CBM1 Pattern Data from Addresses 0 to 15

```
int sample(void)
{
    uint32_t  vp[16];

    if(AT_read_cbm(CBM_UNIT1,CBM_1,0,15,16,vp) == -1){
        AT_printf("Error: AT_read_cbm()\n");
        exit(1);
    }

    return(0);
}
```

2. 4.95 AT_set_pdfm

Function: Sets or resets the Parallel DFM utility.

Syntax

```
#include "AT/hybrid.h"

int AT_set_pdfm
(
    int32_t      *errnum, /* error number */
    int32_t      grpno   /* number of data to set */
);
```

Description

This function specifies whether to allocate multiple DUTs in DFM or reset the allocation.

For the argument grpno, specify 1 to allocate or 0 to reset the allocation.

If the function ends successfully, the argument errnum returns 0. If the function fails, it returns the error number from the Parallel DFM utility.

Note:

The argument meanings, the read data format, and notes are the same as those for the PDFMSET utility.

➔ For more information on this function, refer to the Parallel DFM utility in [T5500 Series Tester Utility Programs Reference Manual\(No. : 8256295\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to errno.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_UTLERR

An error occurred in this function.

➔ For more information, refer to the argument errnum.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

(1) Example of a simultaneous DFM obtaining setting

```
int sample(void)
{
    int      ret;
    int32_t  errnum;

    ret = AT_set_pdfm(&errnum, 1);
    if ((ret != 0) || (errnum != 0)) {
        AT_printf("Error:AT_set_pdfm \n");
        exit(1);
    }

    return(0);
}
```

2. 4.96 AT_read_pdfm

Function: Reads the fail information which is obtained in the DFM for every DUT I/O channel block.

Syntax

```
#include "AT/hybrid.h"

int AT_read_pdfm
(
    int32_t      *errnum, /* error number */
    int32_t      untsl,   /* number of unit select */
    int32_t      dutno,   /* number of dut select */
    int32_t      pnbk,    /* number of pin block select */
    int32_t      *rdcnt,  /* number of read data count */
    int32_t      ndata,    /* number of array element */
    uint32_t      rddat    /* where to put data */
);
```

Description

This function reads DUT pin pass/fail information which is obtained in the DFM.

The argument untsl is used to select the memory to be read. Specify 1 for PIN-DFM and 2 for DFM.

The argument dutno is used to select the DUT numbers to be read. Specify a value from 1 up to the number set for the parallel test which is used in the socket program.

The argument pnbk is used to select the pin block to be read. Specify 1, 2, 3, or 4 for the 33-pin block, 97-pin block, 161-pin block, or 225-pin block, respectively.

The argument rdcnt stores the number of effective data items which are read.

For the argument ndata, specify the number of array elements to be stored.

For the argument rddat, specify a pointer that is set to a uint32_t-type array which has at least as many elements as specified in the argument ndata.

If the function ends successfully, the argument errnum returns 0. If the function fails, it returns an error number from the Parallel DFM utility.

Note:

The argument meanings, the read data format, and notes are the same as those for the PDFMSET utility.

➔ For more information on this function, refer to the Parallel DFM utility in [T5500 Series Tester Utility Programs Reference Manual\(No. : 8256295\)](#).

Return Value

If the function ends successfully, it returns 0.

If the function fails, it returns -1 with error information assigned to errno.

ERRNO

EINVAL

An invalid argument is specified.

S_hybrid_UNAVAILABLE

The hybrid controller cannot be used.

S_hybrid_RTE_OCCURRED

An error occurred in the previously executed ATL program.

S_hybrid_UTLERR

An error occurred in this function.

➔ For more information, refer to the argument errnum.

S_atlerr_XXXX

The error indicated by ATL error code XXXX occurred.

Example

- (1) The example reads the 2048 data items from the DUT1 33-pin block which are stored in the PIN-DFM

```
int sample(void)
{
    int      ret;
    int32_t  errnum;
    int32_t  rdcnt;
    uint32_t rddat[2048];

    ret = AT_read_pdfm(&errnum, 1, 1, 1, &rdcnt, 2048, rddat);
    if ((ret != 0) || (errnum != 0)) {
        AT_printf("Error:AT_read_pdfm \n");
        exit(1);
    }

    return(0);
}
```

3. *ATL Library*

This chapter describes utility programs which can be used by the hybrid controller.

3. 1 Useful Utility Programs

3. 1. 1 CLOCKGET

Function: This utility program reads the CPU time.

Syntax

```
ASM CLOCKGET (SEC, NSEC)
```

Argument: SEC

Specify a variable to store data in seconds.

Specify an integer type 2 word variable or logical type 2 word variable.

NSEC

Specify a variable to store data in nanoseconds.

Specify an integer type 2 word variable or logical type 2 word variable.

Description

The CLOCKGET utility program is called from the ATL program and used to read the current CPU time to variables, and to measure execution time.

The CPU time is divided into integer and fraction parts and stored in the SEC and NSEC arguments, respectively. The CPU time is calculated by using the following formula: $(SEC + NSEC \times 10^{-9})$. The range of the data stored in NSEC is: $0 \leq NSEC < 1,000,000,000$.

The CPU time can be read by using either a ATL program or a C program.

Therefore, both ATL and C programs are used to measure the execution time accurately.

- ➔ For more information on the library functions `AT_clock_gettime()` and `AT_diff_timespec()` that are to be provided as the C program interface, refer to [1. 1. 3 "AT_clock_gettime" on page 1-4](#) and [1. 1. 4 "AT_diff_timespec" on page 1-7](#).
- ➔ For a program sample using this function, refer to [1. 1. 3 "AT_clock_gettime" on page 1-4](#).

Error Condition

If this utility program satisfies one of the error conditions described in below, a runtime error occurs and the utility program is terminated.

Error Condition	Runtime Error Code
The number of arguments does not match.	3055
An integer- or logical-type two-word variable is not used for the argument SEC or NSEC.	3000
This statement cannot be used for the current hardware.	12173

Index

[A]

AT_atexit.....	1-1	AT_perror.....	1-41
AT_calloc.....	1-3	AT_printErrno	1-43
AT_clock_gettime	1-4	AT_printf.....	1-42
AT_diff_timespec	1-7	AT_putchar.....	1-44
AT_event	1-9	AT_puts.....	1-45
AT_evlongjmp.....	1-14	AT_read_afm_confcnt	2-169
AT_evmask	1-17	AT_read_afm_confdat	2-172
AT_evwait.....	1-20	AT_read_afm_dutselect.....	2-175
AT_fork.....	1-23	AT_read_afm_failaddr	2-182
AT_forkkill.....	1-26	AT_read_afm_failcnt.....	2-191
AT_forkwait	1-27	AT_read_afm_mcnt.....	2-200
AT_free	1-29	AT_read_afm_memory.....	2-177
AT_free_named	1-30	AT_read_category	2-68
AT_get_channel.....	1-31	AT_read_cbm	2-220
AT_getchar.....	1-35	AT_read_cdut	2-108
AT_get_pin.....	1-33	AT_read_cfm_failcnt.....	2-215
AT_gets.....	1-36	AT_read_cfm_memory.....	2-206
AT_gpib_timeout.....	2-84	AT_read_cnt.....	2-13
ATL_exec	1-52	AT_read_ctcnt	2-15
ATL_get_char.....	1-60	AT_read_cumfunc	2-151
ATL_get_errinfo	1-62	AT_read_cumfunc_block	2-156
ATL_get_inte	1-64	AT_read_cumfuncm.....	2-153
ATL_get_intem.....	1-66	AT_read_dbm	2-213
ATL_get_logi.....	1-68	AT_read_dct_data	2-60
ATL_get_logim.....	1-70	AT_read_dct_fail.....	2-43
ATL_get_real.....	1-72	AT_read_dct_failm	2-45
ATL_get_realm.....	1-74	AT_read_dct_hfail.....	2-48
ATL_init.....	1-76	AT_read_dct_hfailm.....	2-50
ATL_init2.....	1-79	AT_read_dct_lfail.....	2-53
ATL_proc.....	1-81	AT_read_dct_lfailm	2-55
ATL_set_char	1-84	AT_read_dct_pdata.....	2-39
ATL_set_inte.....	1-85	AT_read_dct_pdatam	2-41
ATL_set_intem	1-88	AT_read_dct_vs	2-58
ATL_set_logi.....	1-90	AT_read_ddut.....	2-100
ATL_set_logim	1-91	AT_read_dutcnt.....	2-76
ATL_set_real	1-93	AT_read_edut.....	2-106
ATL_set_realm.....	1-95	AT_read_ercode	2-158
AT_malloc.....	1-37	AT_read_ext.....	2-86
AT_malloc_named	1-38	AT_read_fail.....	2-9
		AT_read_fail_test_cnt.....	2-11

AT_read_fdut	2-114	AT_set_exclusionm	2-126
AT_read_fdut_t	2-116	AT_set_exclusionm2	2-136
AT_read_fk	2-72	AT_set_fail	2-26
AT_read_fpin	2-64	AT_set_fkmode	2-70
AT_read_func	2-144	AT_set_jdut	2-93
AT_read_func_block	2-149	AT_set_jdutm	2-95
AT_read_funcm	2-146	AT_set_pdfm	2-222
AT_read_gfdut	2-118	AT_set_rdut	2-99
AT_read_gpib_data	2-82	AT_set_rejection	2-89
AT_read_gpib_srq	2-81	AT_set_rejectionm	2-91
AT_read_mdut	2-104	AT_set_result	2-24
AT_read_pdfm	2-224	AT_set_sdut	2-120
AT_read_pdut	2-110	AT_set_sdutm	2-122
AT_read_pdut_t	2-112	AT_set_sort	2-34
AT_read_result	2-7	AT_set_stn	2-37
AT_read_sdut	2-102	AT_set_wfcnt	2-32
AT_read_sort	2-35	AT_strerror	1-47
AT_read_stn	2-73	AT_strtor	1-48
AT_read_stncnt	2-75	AT_strtorm	1-50
AT_read_test	2-74	AT_write_afm	2-159
AT_read_tpin	2-62		
AT_read_wfcnt	2-18		
AT_read_wfctcnt	2-21		
AT_reset_category	2-67		
AT_reset_cnt	2-30		
AT_reset_exclusion	2-129		
AT_reset_exclusion2	2-139		
AT_reset_exclusionm	2-131		
AT_reset_exclusionm2	2-141		
AT_reset_fail	2-27		
AT_reset_fkmode	2-71		
AT_reset_jdut	2-98		
AT_reset_result	2-25		
AT_reset_stn	2-38		
AT_reset_wfcnt	2-33		
AT_scanf	1-46		
AT_send_ext	2-85		
AT_send_gpib_data	2-79		
AT_send_gpib_function	2-77		
AT_set_category	2-66		
AT_set_cnt	2-28		
AT_set_decision	2-87		
AT_set_exclusion	2-124		
AT_set_exclusion2	2-134		

[C]

CLOCKGET	3-1
----------------	-----